Rechnerstrukturen Teil II

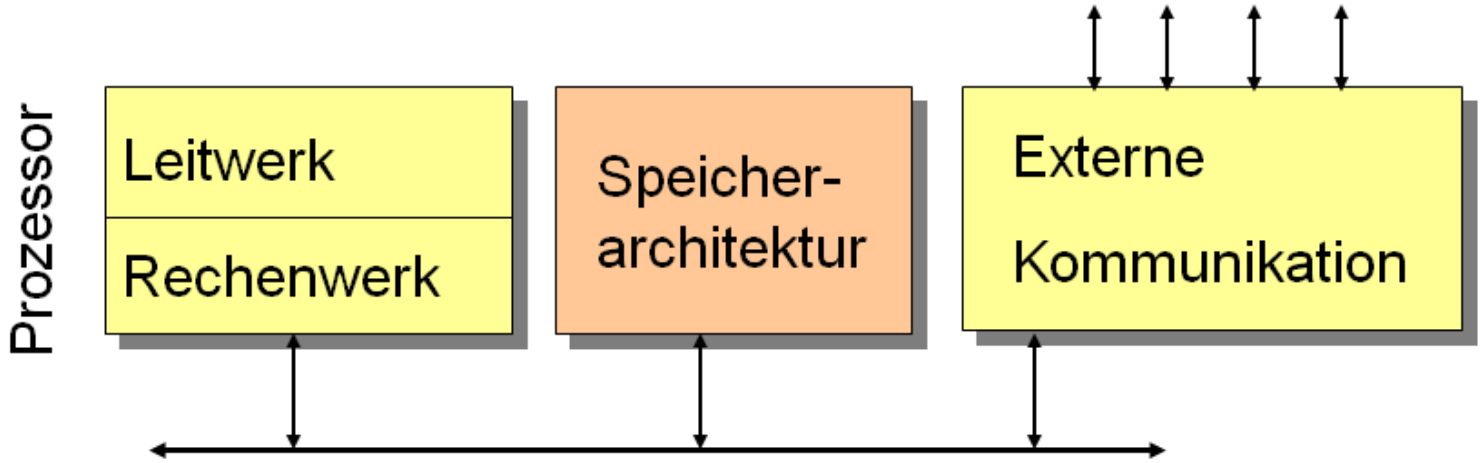
2.4 Speicherarchitektur

Dr.-Ing. Christian Hakert / Dr. rer. nat. Mario Günzel

TU Dortmund, Fakultät für Informatik, Lehrstuhl 12

by Decker

Kontext



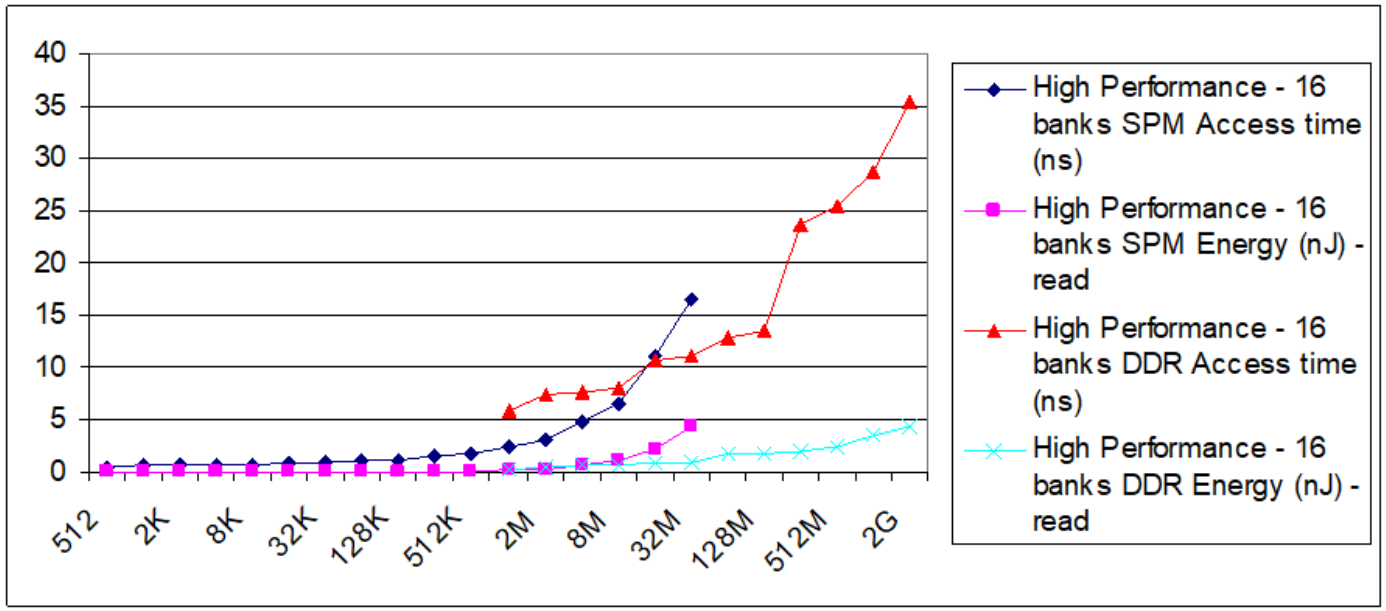
Die Wissenschaft Informatik befasst sich mit der Darstellung, Speicherung, Übertragung und Verarbeitung von Informationen

[Gesellschaft für Informatik]

Entwurfsziele

- Möglichst große Kapazität
- Möglichst kleine Zugriffszeiten
  - Geringe Zeit bis zur Verfügbarkeit eines Speicherinhalts (kleine latency)
  - Kleiner zeitlicher Abstand zwischen Übertragungen, hoher Durchsatz (großer throughput)
- Persistente (nicht-flüchtige) Speicherung
- Geringe Energieaufnahme
- Hohe Datensicherheit
- Geringer Preis
- Physikalisch klein

Konflikt für *einen* Speicher: Entweder hohe Kapazität oder schnell und energieeffizient

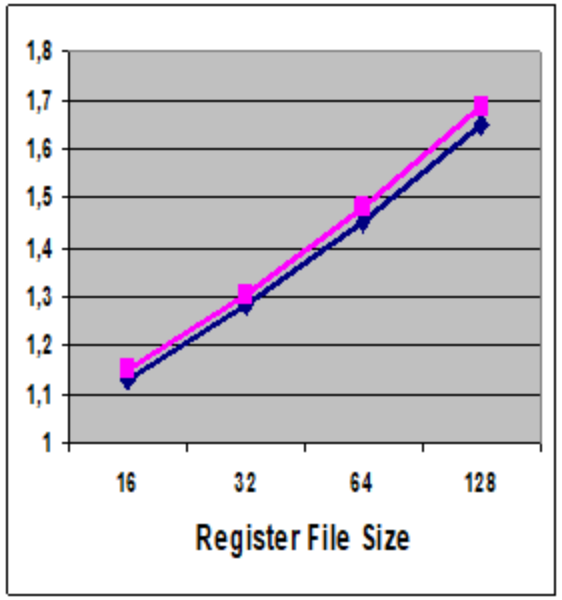


Quelle: Oliviera Jovanovic, TU Dortmund 2011

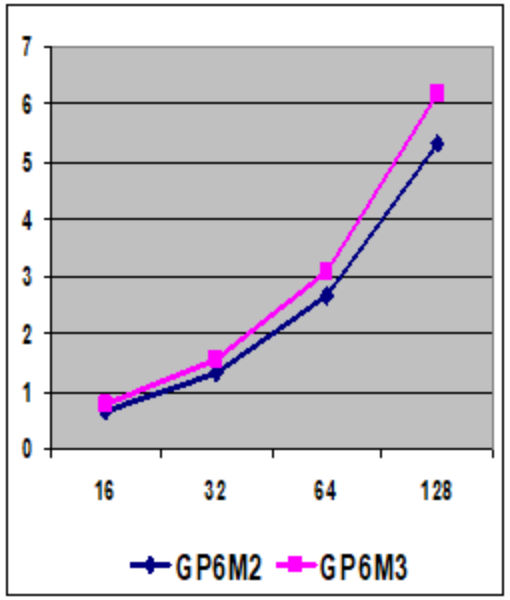
- 16 bit lesen
- Größe in Bytes
- 65nm für SRAM
- 80nm für DRAM

„Alles“ ist groß für große Speicher

Zykluszeit (ns)*



Fläche ($\lambda^2 x 10^6$)

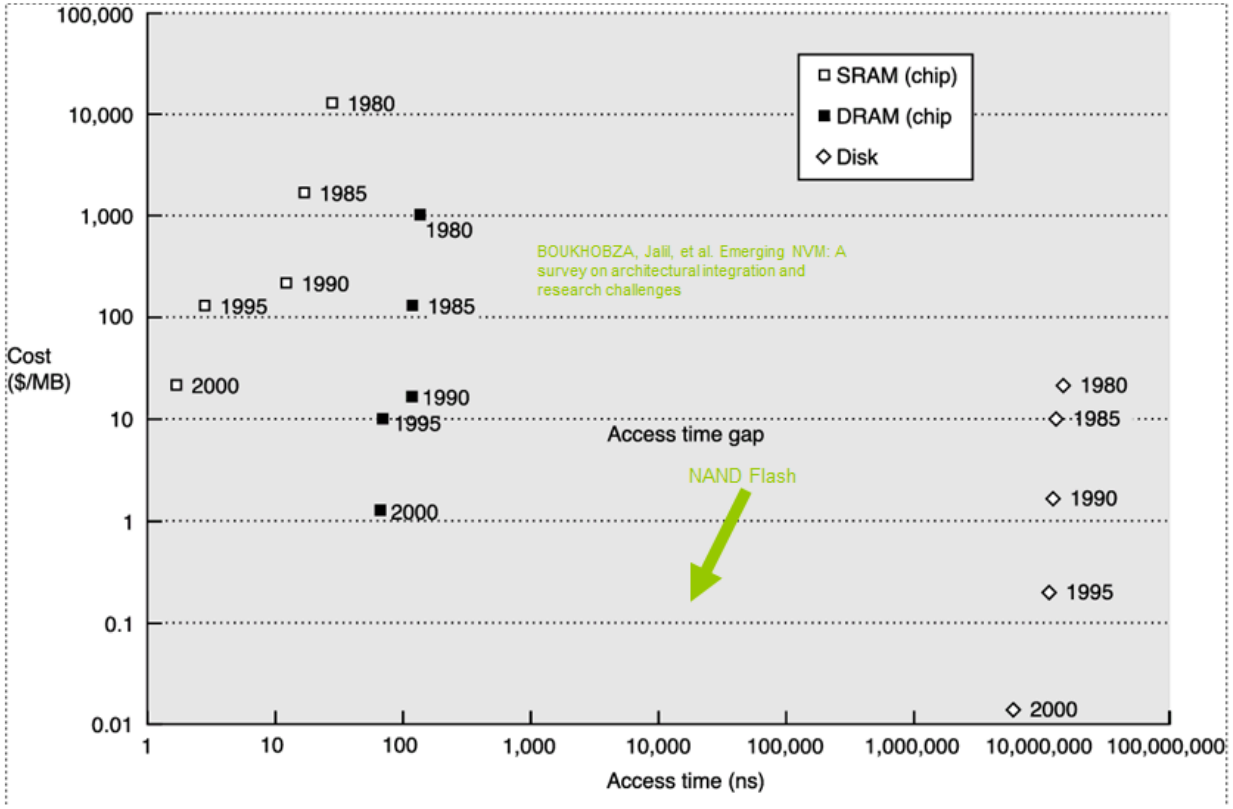


El. Leistung (W)



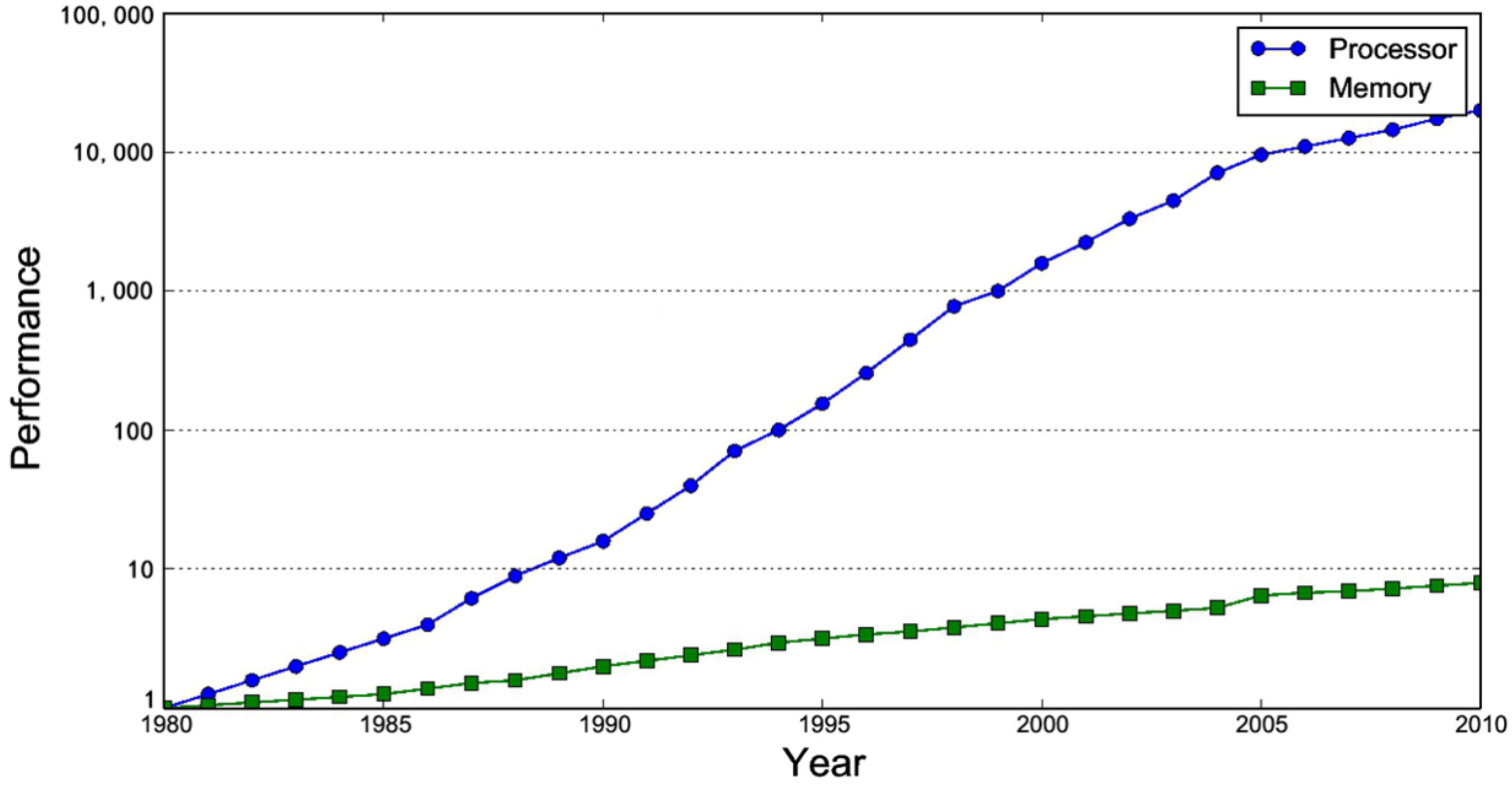
* Monolithic register file; Rixner’s et al. model [HPCA’00], Technology of 0.18 mm; VLIW configurations for a certain number of ports („GPxMyREGz where: x={6}, y={2, 3} and z={16, 32, 64, 128“}; Based on slide by and ©: Harry Valero, U. Barcelona, 2001

Die Realität: Kosten/Mbyte und Zugriffszeiten für verschiedene Speichermedien



[Hennessy/Patterson, Computer Architecture, 3. Aufl.] © Elsevier Science (USA), 2003, All rights reserved

Der Speicher-„Flaschenhals“ (Memory wall)



Rate of improvement in processor performance outpaces the rate of improvement in memory performance

⇒ Memory performance limits overall system performance

Speicherhierarchie

- Große Speicher sind langsam und benötigen viel Energie
- Anwendungen verhalten sich üblicherweise lokal
- Wir versuchen so abzulegen:
  - häufig benötigte Speicherinhalte in kleinen Speichern,
  - seltener benötigte Inhalte in großen Speichern.
- Einführung einer „Speicherhierarchie“
- Durch Kombination im Mittel die Illusion eines großen Speichers mit kleinen Zugriffszeiten.
- Enge Grenze für Zugriffszeit ist meist nicht zu garantieren

- Bei Realzeitsystemen: spezielle Überlegungen

Der Konflikt und die Idee sind nicht neu

“Ideally one would desire an indefinitely large memory capacity such that any particular … word … would be immediately available - i.e. in a time which is … shorter than the operation time of a fast electronic multiplier. … It does not seem possible physically to achieve such a capacity. We are therefore forced to recognize the possibility of constructing a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible.”

A.W. Burks, H.H. Goldstine, and J. von Neumann: Preliminary Discussion of the Logical Design of an Electronic Computing Element, 1946

Mögliche Stufen der Speicherhierarchie und derzeit eingesetzte Technologien

Tertiärspeicher (Optisch, Bänder, Cloud) (persistent)
Sekundärspeicher (Platten, Flash) (persistent)
Plattencaches (DRAM, Flash)
Haupt- bzw. Primärspeicher (DRAM)
Caches (SRAM) & TLBs
Register (SRAM)

Schlüssel zur Einführung der Hierarchie: Speicherverwaltung

Unterscheidung zwischen den Adressen, welche der Prozessor generiert und jenen, mit denen der physikalische Speicher adressiert wird.

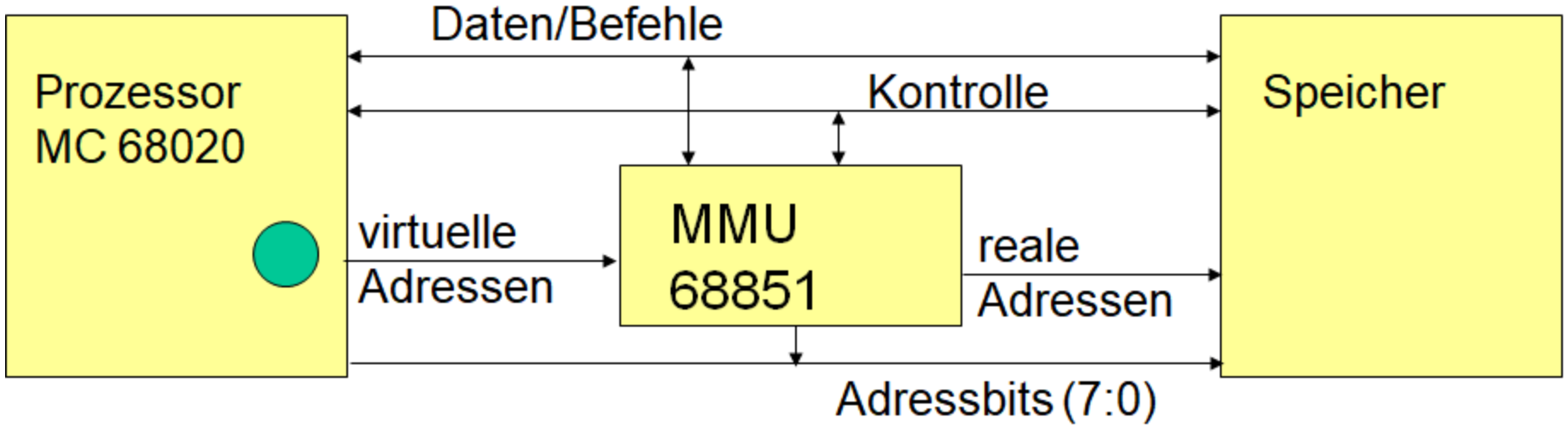
Def.: **Prozessadressen** bzw. **virtuelle Adressen** sind die effektiven Adressen nach Ausführung aller dem Assemblerprogrammierer sichtbaren Adressmodifikationen, d.h. die von einem Prozess dem Rechner angebotenen Adressen.

Def.: **Maschinenadressen, reale Adressen** bzw. **physikalische Adressen** sind die zur Adressierung der realen Speicher verwendeten Adressen.

Memory Management Unit (MMU)

MMU ist für das Umrechnen der Adressen verantwortlich

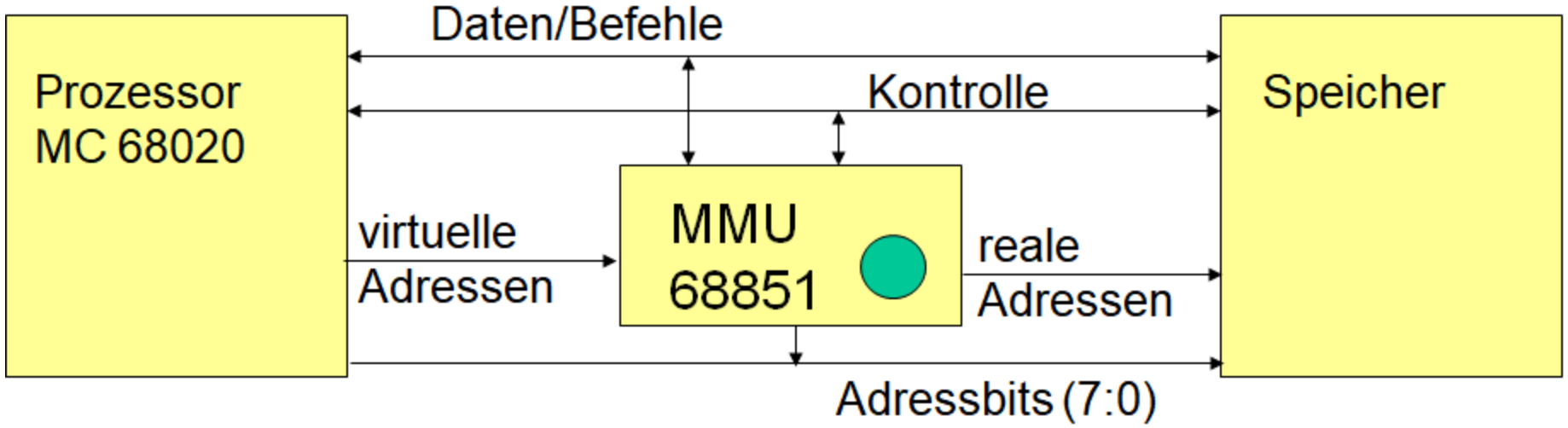
Beispiel **Motorola**:



Memory Management Unit (MMU)

MMU ist für das Umrechnen der Adressen verantwortlich

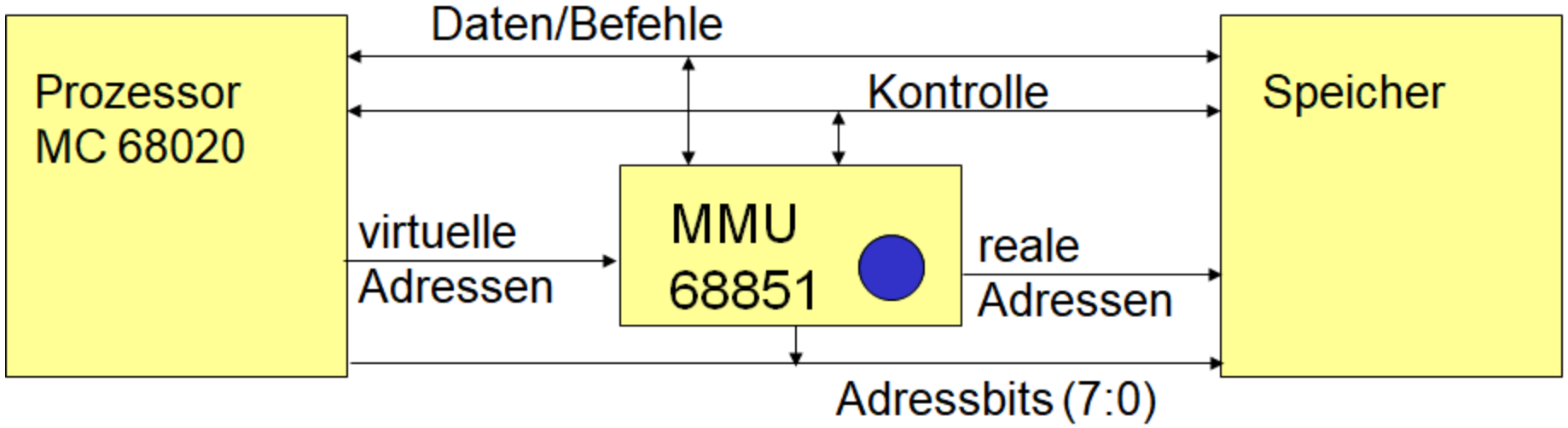
Beispiel **Motorola**:



Memory Management Unit (MMU)

MMU ist für das Umrechnen der Adressen verantwortlich

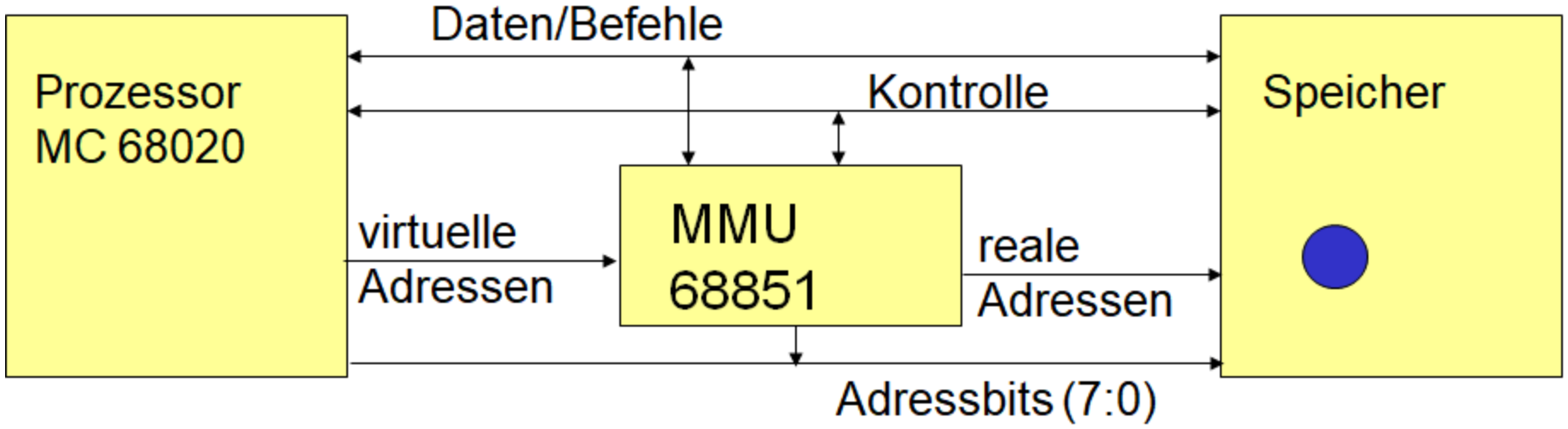
Beispiel **Motorola**:



Memory Management Unit (MMU)

MMU ist für das Umrechnen der Adressen verantwortlich

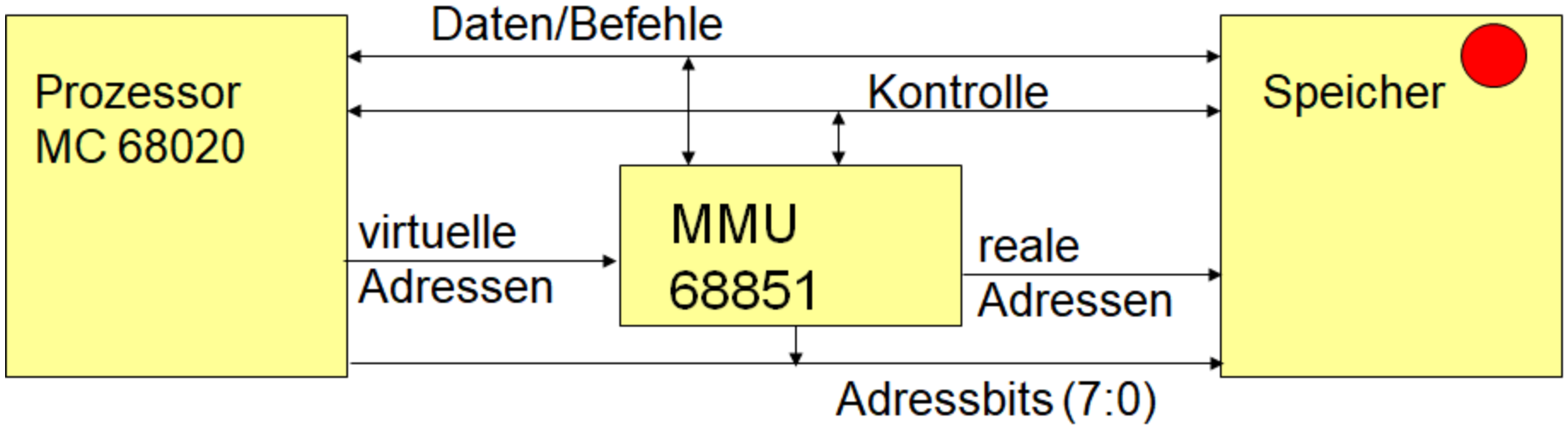
Beispiel **Motorola**:



Memory Management Unit (MMU)

MMU ist für das Umrechnen der Adressen verantwortlich

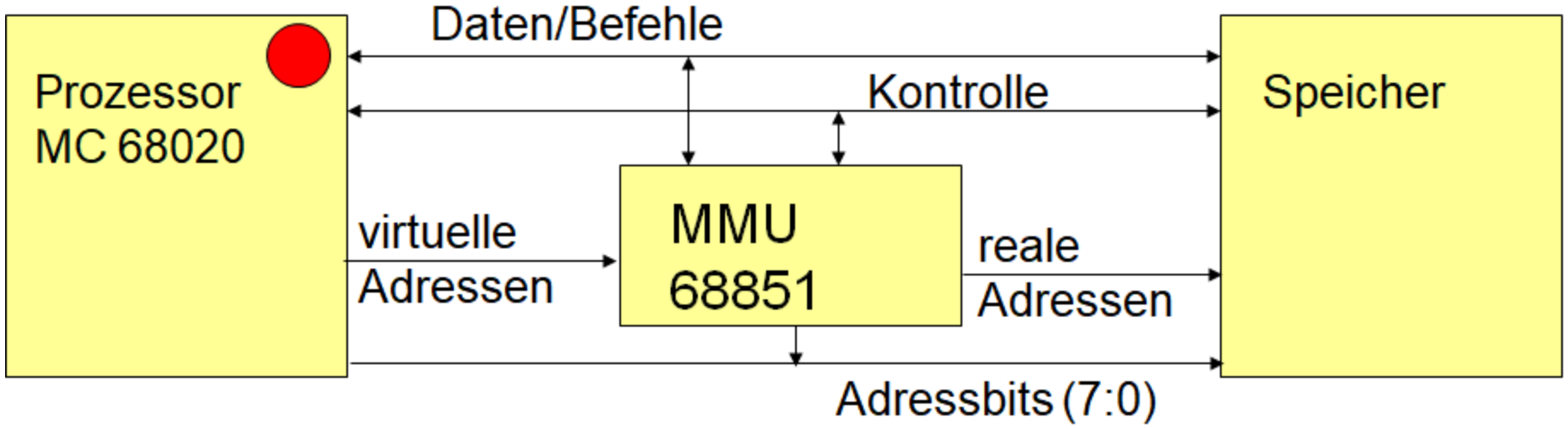
Beispiel **Motorola**:



Memory Management Unit (MMU)

MMU ist für das Umrechnen der Adressen verantwortlich

Beispiel **Motorola**:

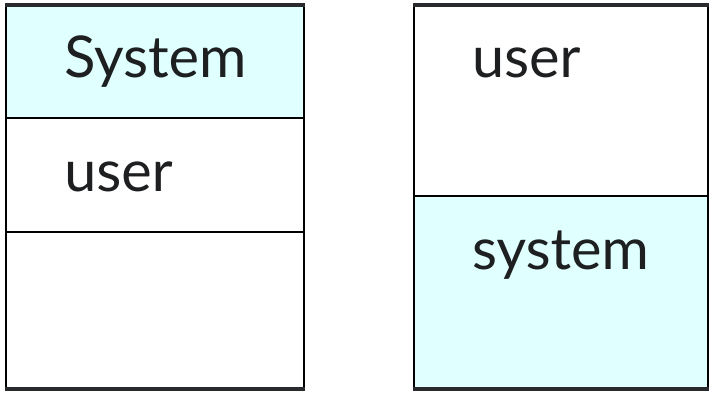


Identität

- Bei einfachen Systemen sind reale und virtuelle Adressen identisch (“memory management without swapping or paging”)
- Adressraum durch real vorhandenen Speicher beschränkt
- Kommt v.a. bei eingebetteten Systemen (Fahrzeugelektronik, alte Handys usw.) vor, weniger bei PCs und Workstations

Sehr einfache Systeme:

- Aufteilung in Systembereich und Benutzerprogramm



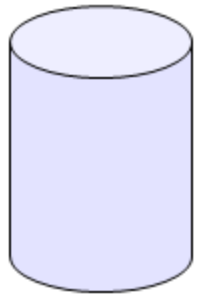
Einteilung in Laufbereiche (core partitions)

Aus Aufwandsgründen statt flexibler Speicherzuteilung an Prozesse feste Partitionierung des Speichers

Compiler adressieren Befehle ab Adresse 0.

Beim Laden muss auf alle Adressen Anfangsadresse der Partition addiert werden → Identifikation von Adressen im übersetzten Programm!

Betriebssystem
Partition 1
Partition 2
Partition 3
Partition 4

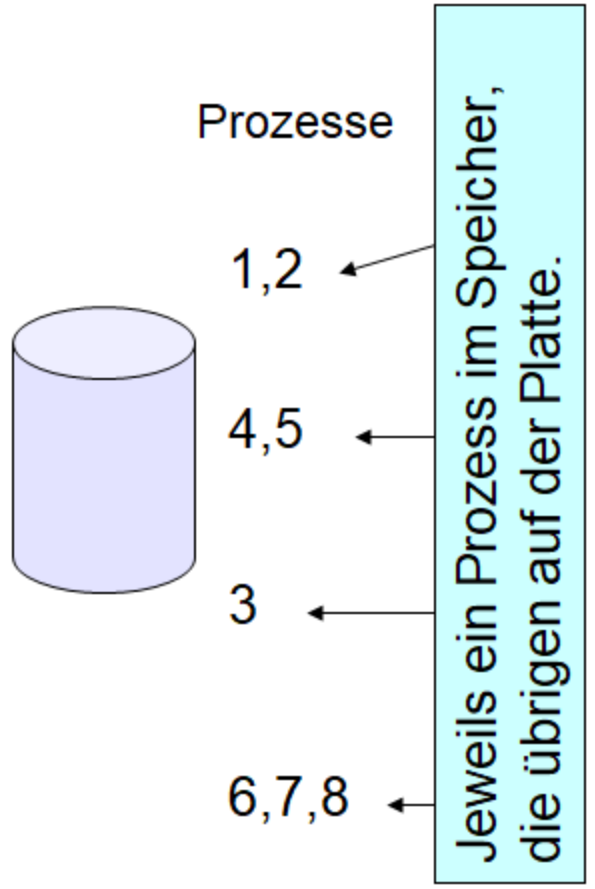


Einteilung in Laufbereiche (core partitions)

Aus Aufwandsgründen statt flexibler Speicherzuteilung an Prozesse feste Partitionierung des Speichers

Compiler adressieren Befehle ab Adresse 0.
Beim Laden muss auf alle Adressen Anfangsadresse der Partition addiert werden
→ Identifikation von Adressen im übersetzten Programm!

Betriebssystem
Partition 1
Partition 2
Partition 3
Partition 4

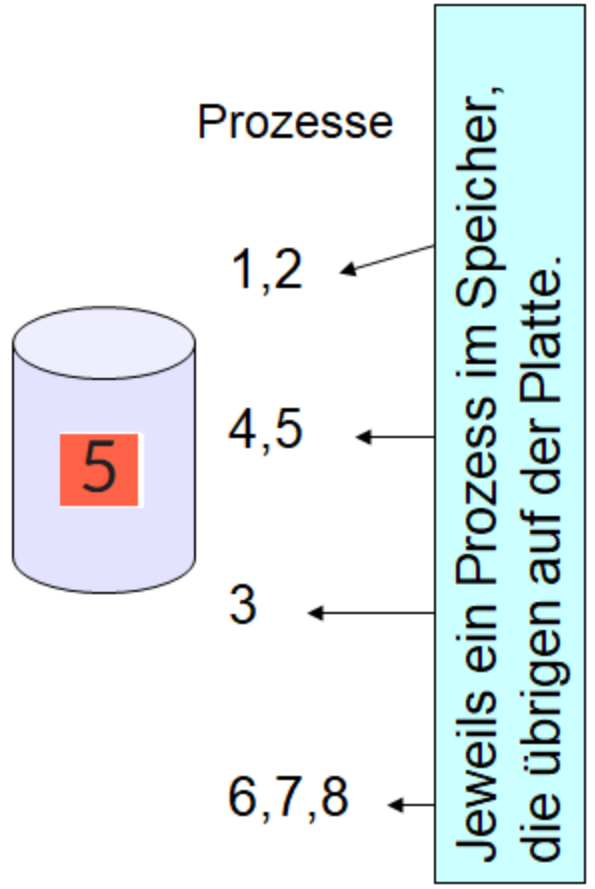


Einteilung in Laufbereiche (core partitions)

Aus Aufwandsgründen statt flexibler Speicherzuteilung an Prozesse feste Partitionierung des Speichers

Compiler adressieren Befehle ab Adresse 0.
Beim Laden muss auf alle Adressen Anfangsadresse der Partition addiert werden
→ Identifikation von Adressen im übersetzten Programm!

Betriebssystem
Partition 1
Partition 2 4
Partition 3
Partition 4

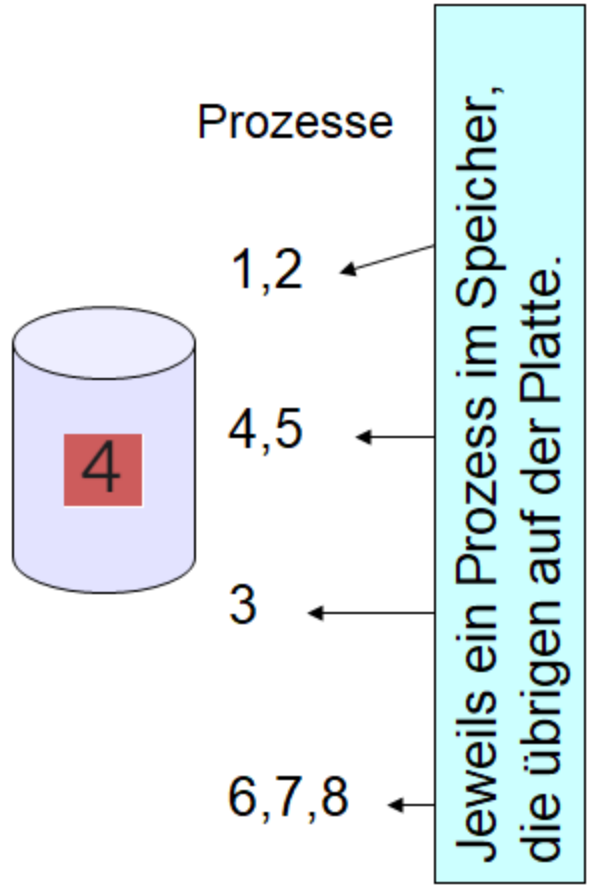


Einteilung in Laufbereiche (core partitions)

Aus Aufwandsgründen statt flexibler Speicherzuteilung an Prozesse feste Partitionierung des Speichers

Compiler adressieren Befehle ab Adresse 0.
Beim Laden muss auf alle Adressen Anfangsadresse der Partition addiert werden
→ Identifikation von Adressen im übersetzten Programm!

Betriebssystem
Partition 1
Partition 2 5
Partition 3
Partition 4

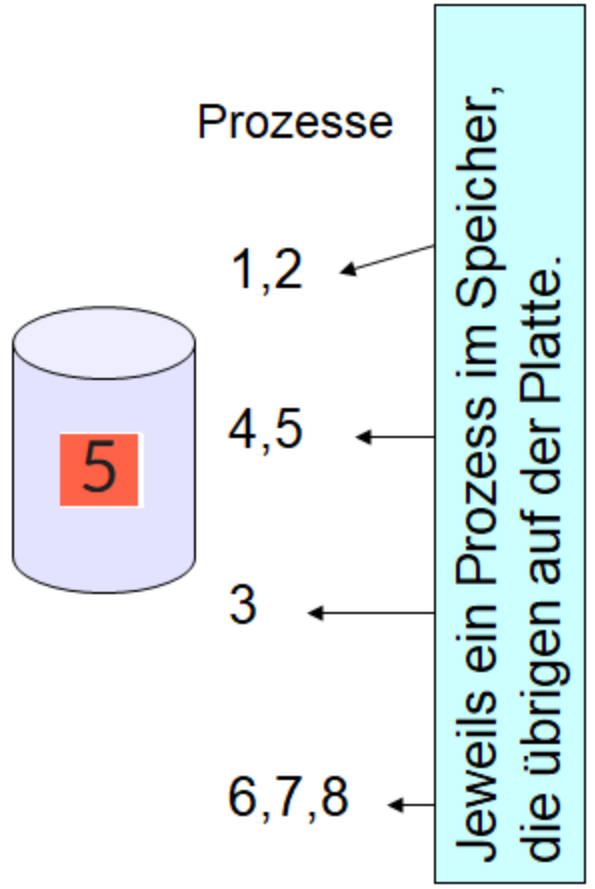


Einteilung in Laufbereiche (core partitions)

Aus Aufwandsgründen statt flexibler Speicherzuteilung an Prozesse feste Partitionierung des Speichers

Compiler adressieren Befehle ab Adresse 0.
Beim Laden muss auf alle Adressen Anfangsadresse der Partition addiert werden
→ Identifikation von Adressen im übersetzten Programm!

Betriebssystem
Partition 1
Partition 2 4
Partition 3
Partition 4



Speicherschutz durch Grenzregister möglich

Z.Tl. nur Zugriff auf durch Grenzregister begrenzten Speicherbereich erlaubt (Speicherschutz); Grenzregister werden bei context switch umgeladen.

Betriebssystem	Prozesse
Partition 1	1,2
Partition 2	4,5
Partition 3	3
Partition 4	6,7,8

Speicherschutz durch Grenzregister möglich

Z.Tl. nur Zugriff auf durch Grenzregister begrenzten Speicherbereich erlaubt (Speicherschutz); Grenzregister werden bei context switch umgeladen.

Erweiterung: Memory protection unit (MPU)

Betriebssystem	Prozesse
Partition 1	1,2
Partition 2	4,5
Partition 3	3
Partition 4	6,7,8

Was tun, wenn Prozesse mehr Speicher benötigen? (1)

Man könnte Partitionen verschmelzen, wenn Prozesse mit dem Speicher der zugeteilten Partition nicht auskommen. Aus Aufwandsgründen selten realisiert.

Betriebssystem	Prozesse
Partition 1	1,2
Partition 2	4,5
Partition 3	3
Partition 4	6,7,8

Was tun, wenn Prozesse mehr Speicher benötigen? (1)

Man könnte Partitionen verschmelzen, wenn Prozesse mit dem Speicher der zugeteilten Partition nicht auskommen. Aus Aufwandsgründen selten realisiert.

Betriebssystem	Prozesse
Partition 1	1,2
Partition 2	4,5
Partition 3	3 → Platte
Partition 4	6,7,8

Was tun, wenn Prozesse mehr Speicher benötigen? (2)

Verschieben von Prozessen im Speicher nicht möglich, da Adressen angepasst werden müssten, andere Informationen nicht. Im Speicher sind Adressen und andere Informationen ununterscheidbar

Aus demselben Grund kann nur der gesamte Inhalt einer Partition auf die Platte ausgelagert werden (**swapping**) und an dieselbe Stelle zurückgeschrieben werden.

Betriebssystem
Partition 1 `0x010101FF` (pointer (Adresse)) `0x23040501` (integer)
Partition 2
Partition 3 `0x010101FF` (pointer (Adresse)) `0x23040501` (integer)
Partition 4

Beurteilung

Nachteile:

Sehr starke „externe Fragmentierung“

(Speicherbereiche bleiben ungenutzt). Adressraum durch real vorhandenen Speicher beschränkt.

Man muss den Speicherbedarf von Applikationen recht genau kennen, um eine möglichst passende Speichergröße zuzuweisen.

Risiko des Prozessabbruchs, wenn Speicher nicht ausreicht.

→ Gründe für bessere Verfahren.

Segmentadressierung (1)

Zusammenhängenden Speicherbereichen (für Befehle, Daten, und den stack) im Prozessadressenraum werden **zusammenhängende Bereiche im realen Adressraum** zugeordnet

Beispiel:

Segment 1 (Befehle Prozess 1, schreibgeschützt)
Segment 2 (Stack Prozess 2)
frei
Segment 4 (Befehle Prozess 2)
Segment 3 (Daten Prozess 1)
Segment 5 (*Stack* Prozess 1)
Segment 6 (Daten Prozess 2)

Segmentadressierung (2)

Speicher	virtuell	real
Segment 1 (Befehle Prozess 1, schreibgeschützt)	0	0100
Segment 2 (Stack Prozess 2)	0	0230
frei	0	0304
Segment 4 (Befehle Prozess 2)	0	0390
Segment 3 (Daten Prozess 1)	0	0520
Segment 5 (*Stack* Prozess 1)	0	0720

Prozess 1		
Befehle	0100	
Daten	0520	
Stack	0720	

- Prozesse adressieren in jedem Segement ab Adresse 0
- Zu jedem Prozess gehört eine Tabelle der für ihre Segmente zu addierenden Adressen (**verdeckte Basen**)

Segment 6 (Daten Prozess 2)	0	0830

Segmentadressierung (2)

Speicher	virtuell	real
Segment 1 (Befehle Prozess 1, schreibgeschützt)	0	0100
Segment 2 (Stack Prozess 2)	0	0230
frei	0	0304
Segment 4 (Befehle Prozess 2)	0	0390
Segment 3 (Daten Prozess 1)	0	0520
Segment 5 (*Stack* Prozess 1)	0	0720

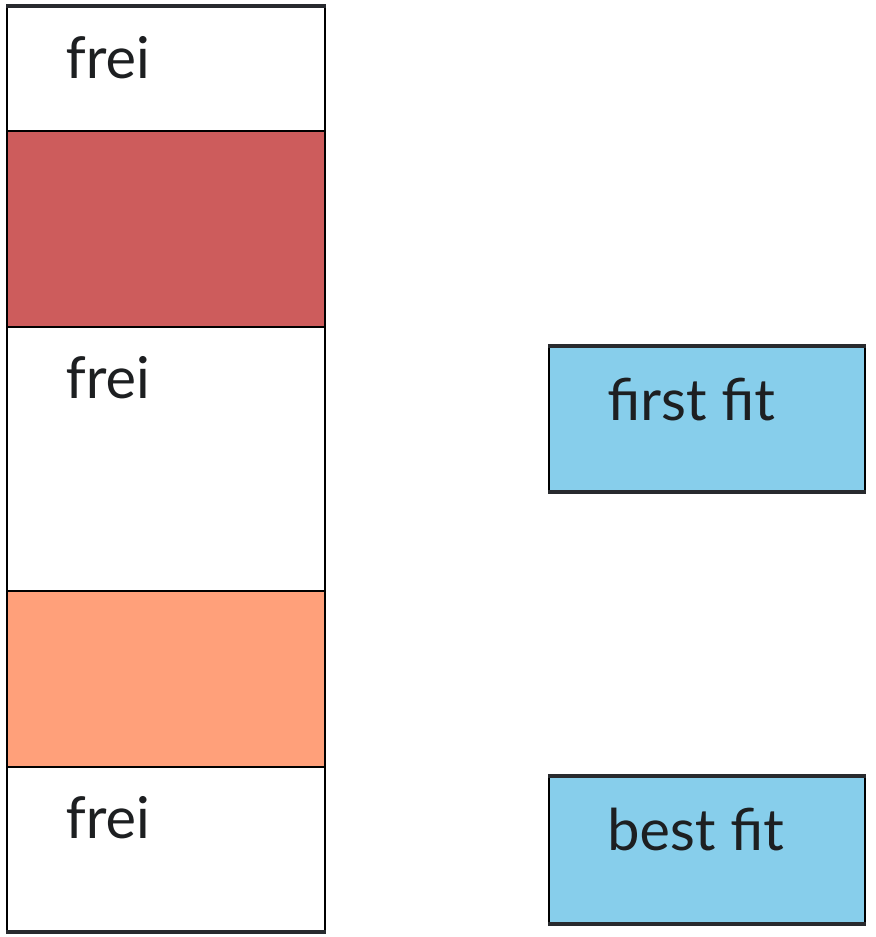
Prozess 2		
Befehle	0390	
Daten	0830	
Stack	0230	

- Prozesse adressieren in jedem Segement ab Adresse 0
- Zu jedem Prozess gehört eine Tabelle der für ihre Segmente zu addierenden Adressen (**verdeckte Basen**)
- Bei jedem Speicherzugriff wird die verdeckte Basis des jeweiligen Segments zur virtuellen Adresse addiert
- Erfordert Befehlssatz mit separaten Befehlen für *stack*-Zugriffe

Segment 6 (Daten Prozess 2)	0	0830

Eigenschaften der Segmentadressierung (1)

- Speicherschutz: Abfrage der Segmentlänge, R/W/E-Modi
- Beim Laden müssen keine Adressen angepasst werden, dies passiert stets zur Laufzeit.
- Verschieben von Segmenten ist möglich, da alle Adressen mittels Änderung der verdeckten Basis automatisch angepasst werden.
- Auslagern ganzer Segmente und Rückschreiben an beliebige, ausreichend große Lücke ist möglich.
- Strategien zur Sucher einer ausreichend großen, freien Lücke: first fit, best fit



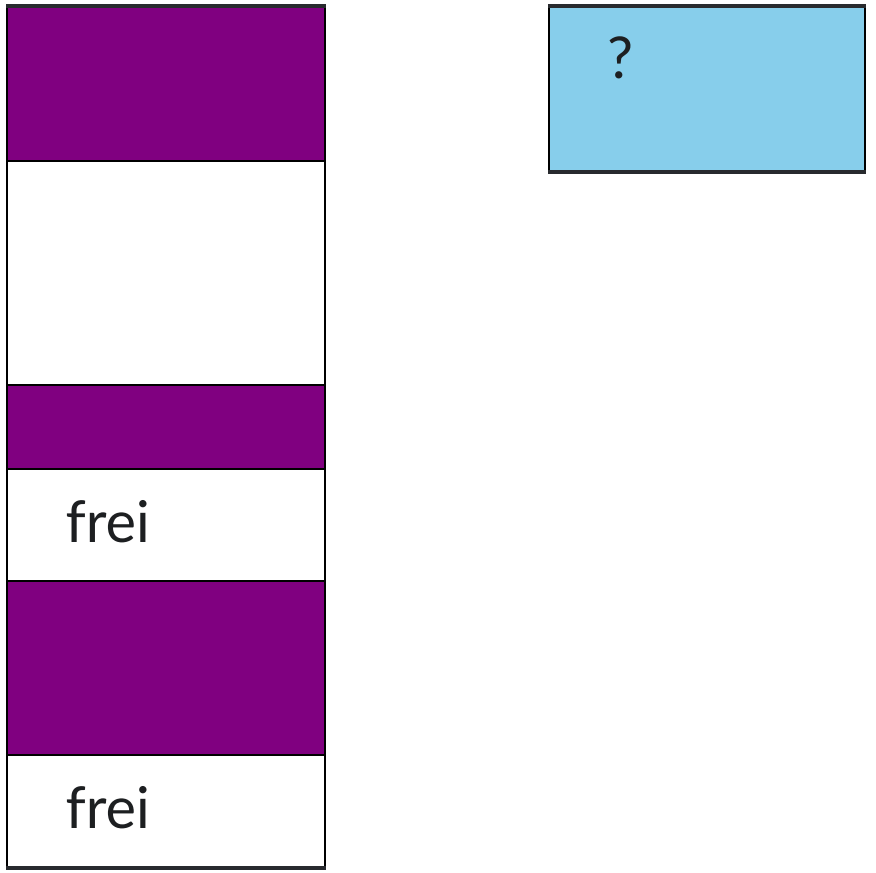
Eigenschaften der Segmentadressierung (2)

- Segmente können sich ausdehnen. Ist der anschließende Speicher belegt, so müssen sie umkopiert werden.
- Ein Teil des Speichers außerhalb der Segmente bleibt unbenutzt (externe Fragmentierung).
- Ggf. müssen Segmente verschoben werden, um ausreichend große freie Bereiche herzustellen.



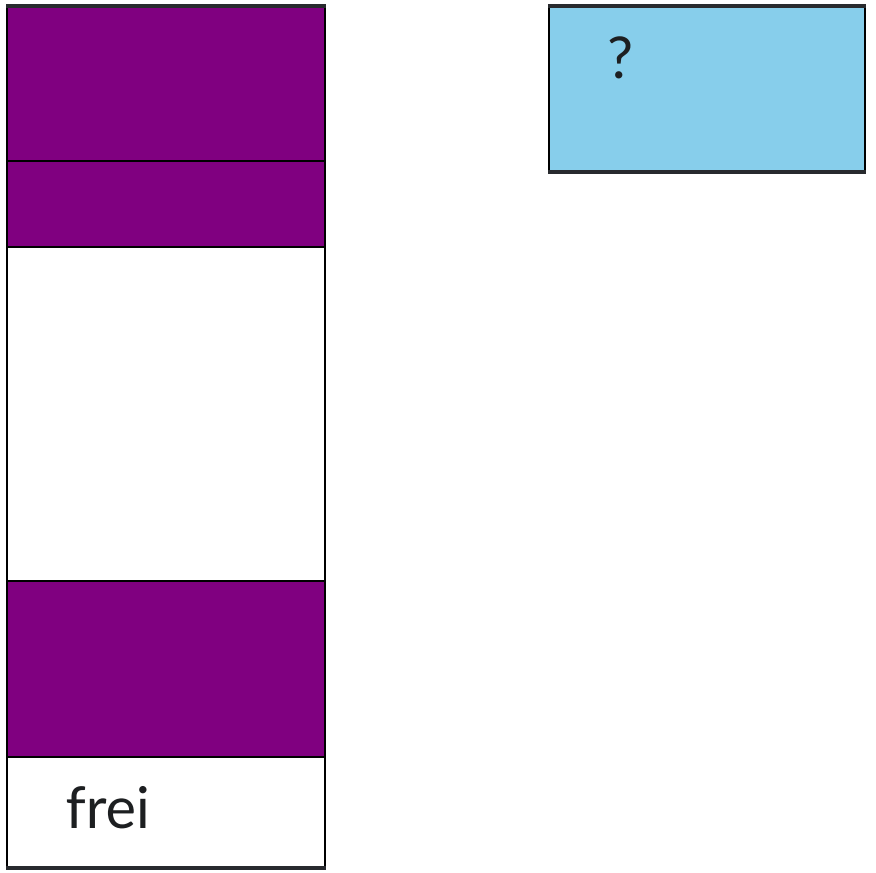
Eigenschaften der Segmentadressierung (2)

- Segmente können sich ausdehnen. Ist der anschließende Speicher belegt, so müssen sie umkopiert werden.
- Ein Teil des Speichers außerhalb der Segmente bleibt unbenutzt (externe Fragmentierung).
- Ggf. müssen Segmente verschoben werden, um ausreichend große freie Bereiche herzustellen.



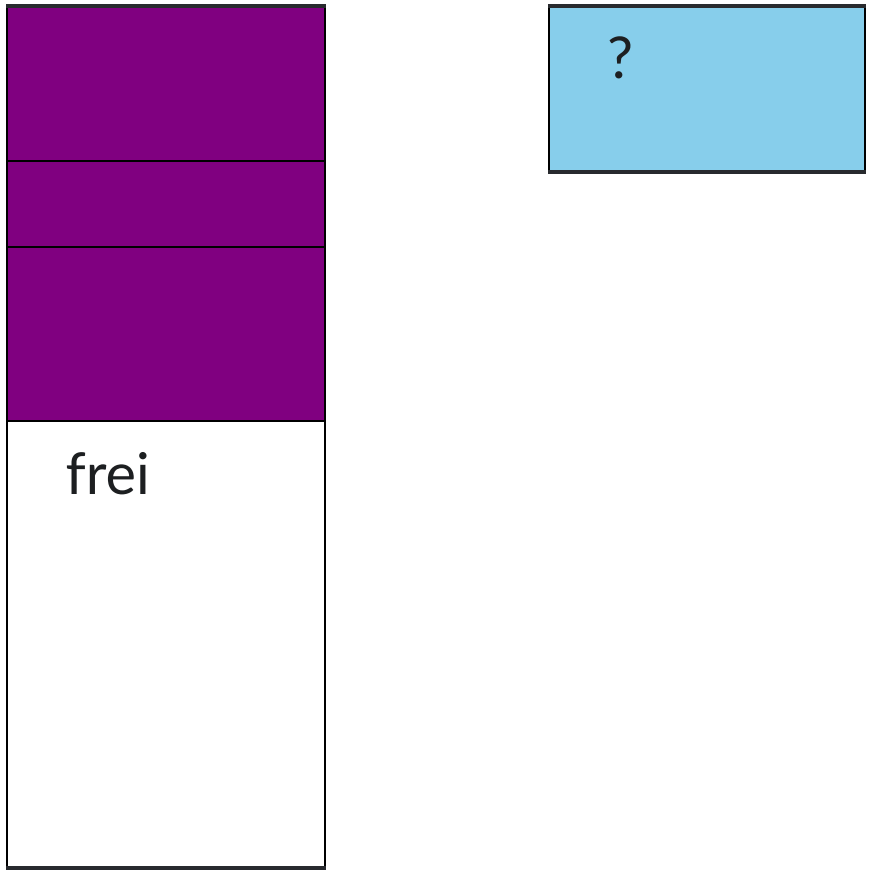
Eigenschaften der Segmentadressierung (2)

- Segmente können sich ausdehnen. Ist der anschließende Speicher belegt, so müssen sie umkopiert werden.
- Ein Teil des Speichers außerhalb der Segmente bleibt unbenutzt (externe Fragmentierung).
- Ggf. müssen Segmente verschoben werden, um ausreichend große freie Bereiche herzustellen.



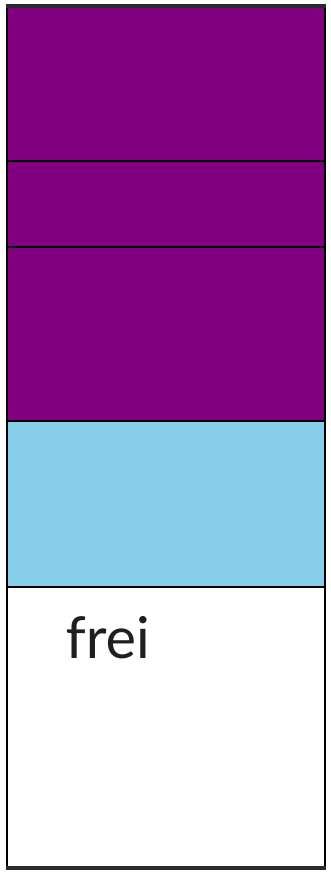
Eigenschaften der Segmentadressierung (2)

- Segmente können sich ausdehnen. Ist der anschließende Speicher belegt, so müssen sie umkopiert werden.
- Ein Teil des Speichers außerhalb der Segmente bleibt unbenutzt (externe Fragmentierung).
- Ggf. müssen Segmente verschoben werden, um ausreichend große freie Bereiche herzustellen.

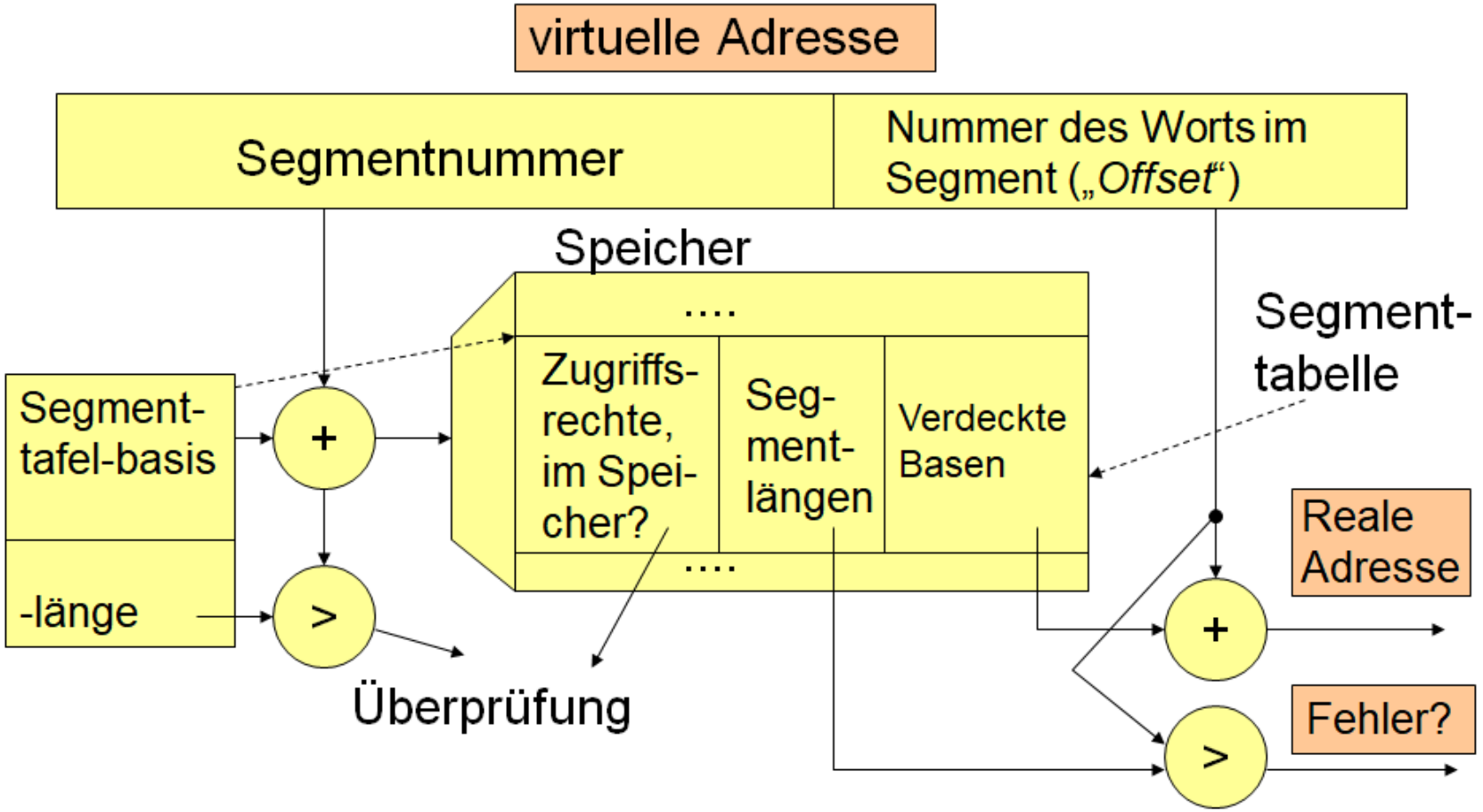


Eigenschaften der Segmentadressierung (2)

- Segmente können sich ausdehnen. Ist der anschließende Speicher belegt, so müssen sie umkopiert werden.
- Ein Teil des Speichers außerhalb der Segmente bleibt unbenutzt (externe Fragmentierung).
- Ggf. müssen Segmente verschoben werden, um ausreichend große freie Bereiche herzustellen.



Adressabbildung bei Segmentadressierung



Bei zusammenhängenden Segmentnummern effizient.

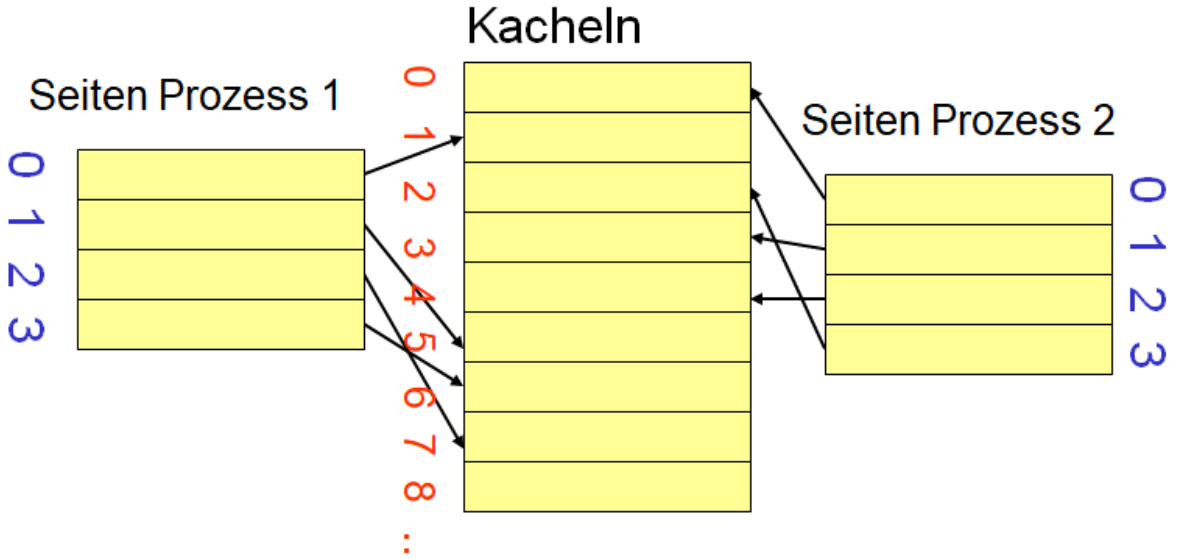
Formen der Segementadressierung & Perspektiven

- Segmentnummern in separaten Registern
  Beispiel: Intel Pentium
- Segmentnummern sind Teil der Adressen im Speicher
  Beispiel: PowerPC, IBM-Großrechner

Segmentadressierung wird von Betriebssystemen nur selten genutzt.

Seiten-Adressierung (paging)

- Einteilung der realen Adressen in Bereiche gleicher Länge (**Seitenrahmen, Kacheln, page frames**)
- Einteilung der virtuellen Adresse in Bereiche konstanter Länge (**Seiten, pages**)
- Jede **Seite** kann auf jede **Kachel** gespeichert werden



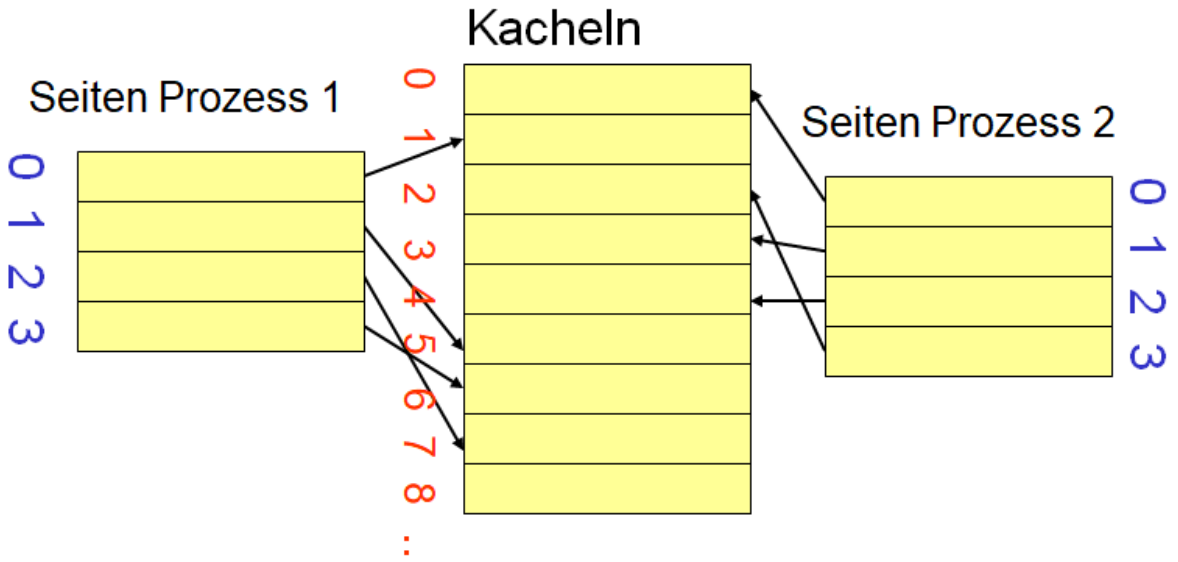
Seiten-Adressierung (paging)

Virtuelle Adresse

Seitennr.	Offset

Zu jedem Prozess existiert eine Tabelle, die **Seitentabelle**

P1:Seite	Kachel
0	1
1	5
2	7
3	6



reale Adresse = Anfangsadresse der Kachel + Offset

Vorteile

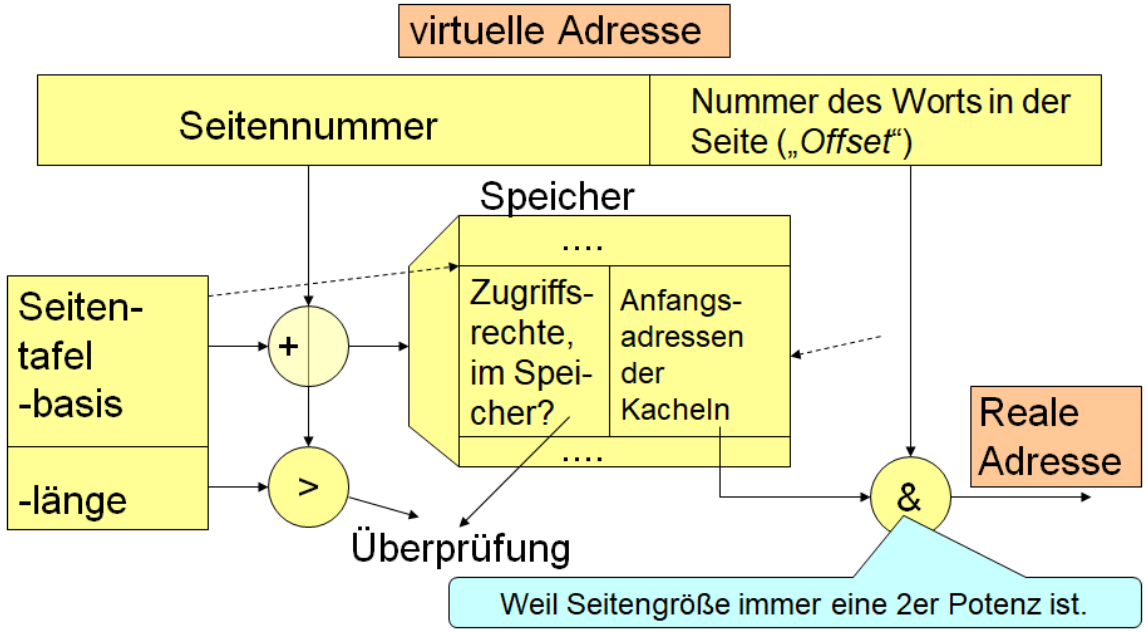
- Man muss sich nicht vorab überlegen, welchem Speicherbereich (text, stack, heap) man wie viel Speicher zuweist.
- Jede Kachel des Speichers kann genutzt werden (keine „**externe Fragmentierung**“).
- Virtuelle Adressräume aller Prozesse können bei 0 beginnen, nur bei getrennt übersetzten Prozeduren sind Adressanpassungen erforderlich.

Nachteil:

- „**interne Fragmentierung**“ (Seiten nicht unbedingt gefüllt)

Organisation der Adressabbildung

- **Schnelle** Umrechnung von virtuellen auf reale Adressen!
- Bei zusammenhängenden Adressen und Seitenadressierung:



- & steht für Konkatenation (Aneinanderreihung) in der Register-Transfer-Notation (siehe ISA, Folie 14)
- Bsp: 8 Bit Anfangsadresse der Kachel **“0101 0101”** (0x55) und 8 Bit Offset **“0000 0011”** (0x03)

→ Reale Adresse: **“01010101”** & **“00000011”** = **“0101 0101 0000 0011”** (0x5503)

- Bsp2: 10 Bit Anfangsadresse der Kachel **“00 0101 0101”** (0x055) und 6 Bit Offset **“00 0011”** (0x03)

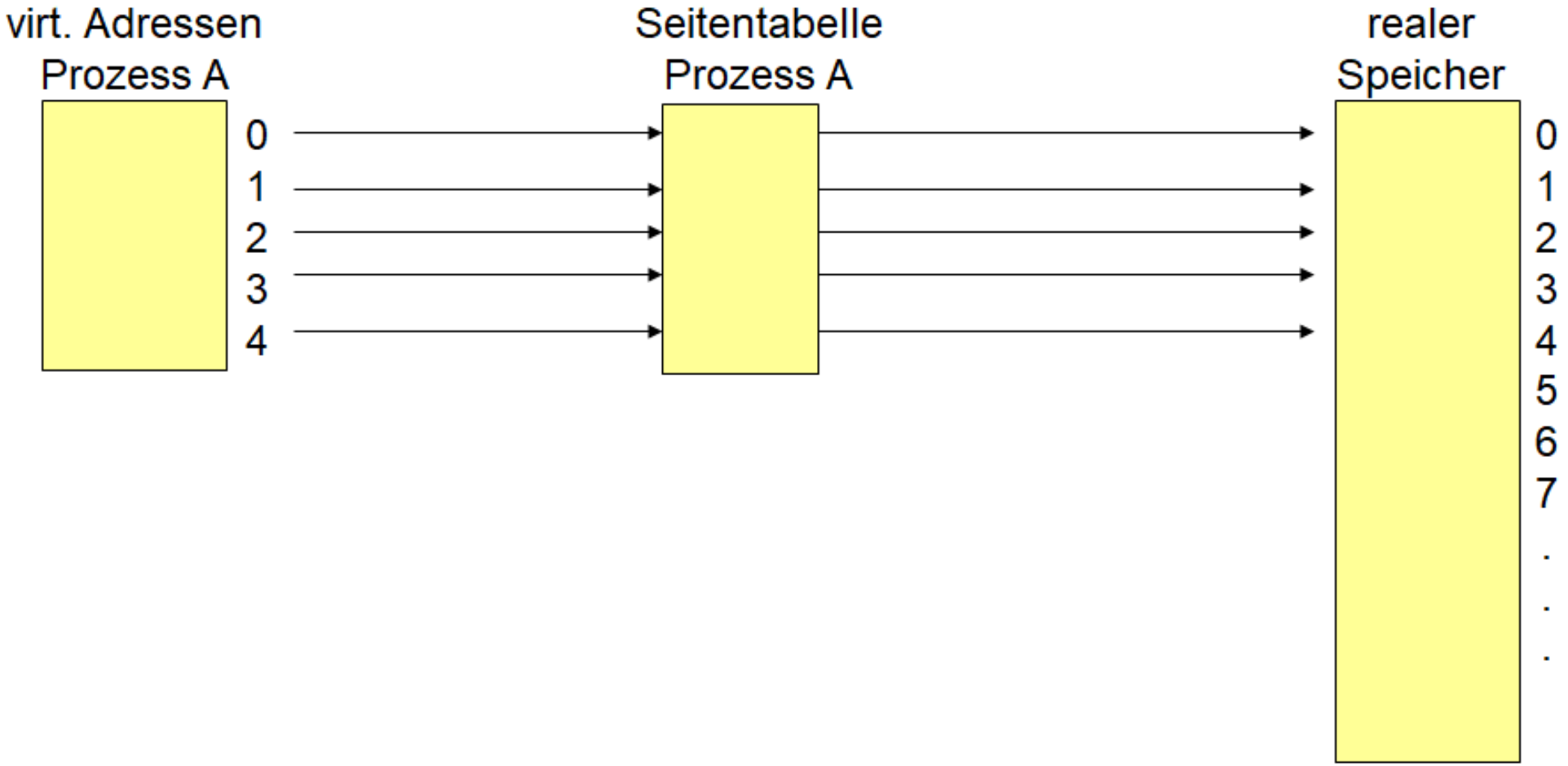
→ Reale Adresse: **“0001010101”** & **“000011”** = **“0001 0101 0100 0011”** (0x1543)

Zusammenfassung

- Reale und virtuelle Adressen, *memory management*
- Form der Speicherverwaltung
  - Identität
  - Seitenadressierung
    - Sharing
  - Segmentadressierung
  - Segmentadressierung mit Seitenadressierung

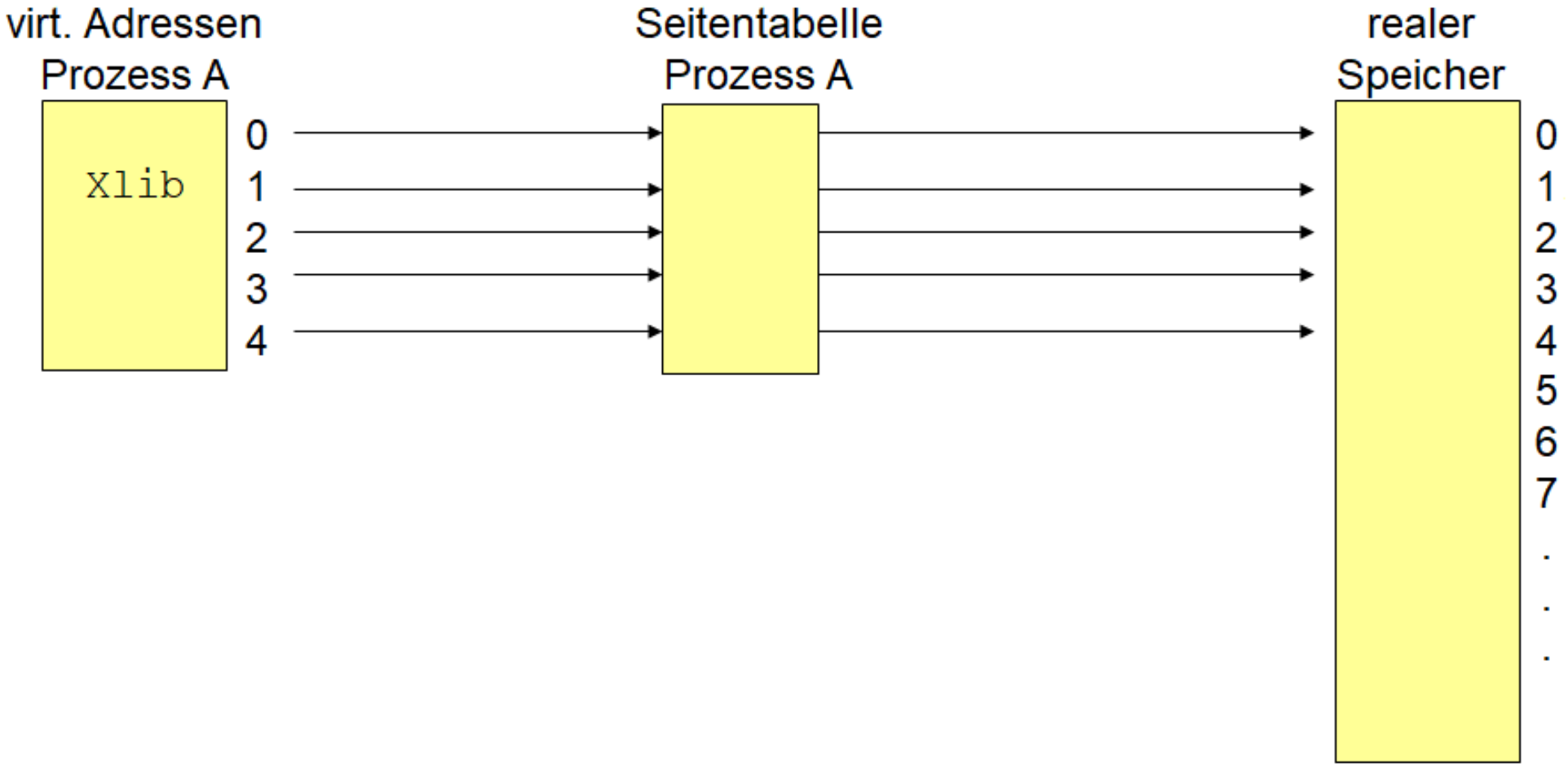
Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



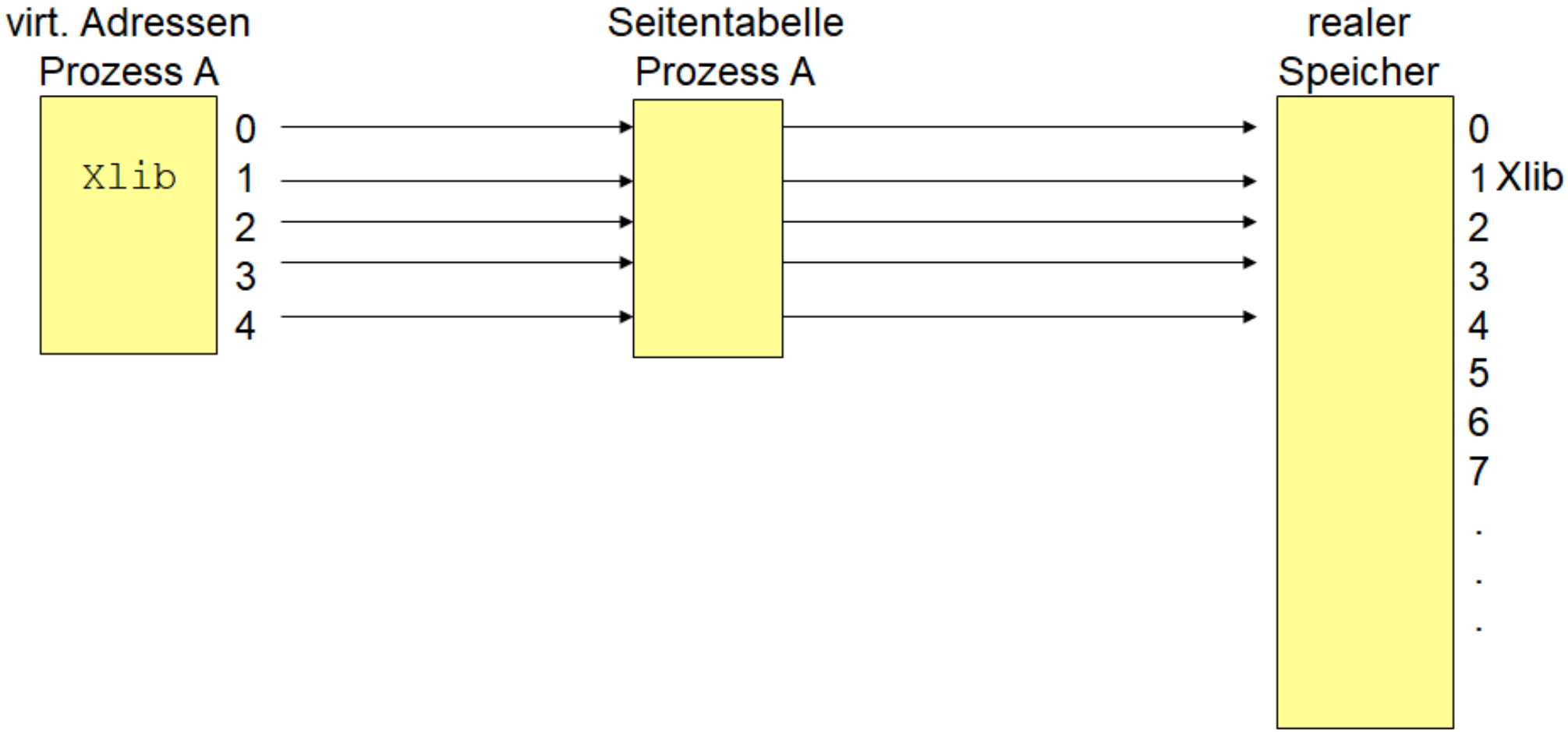
Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



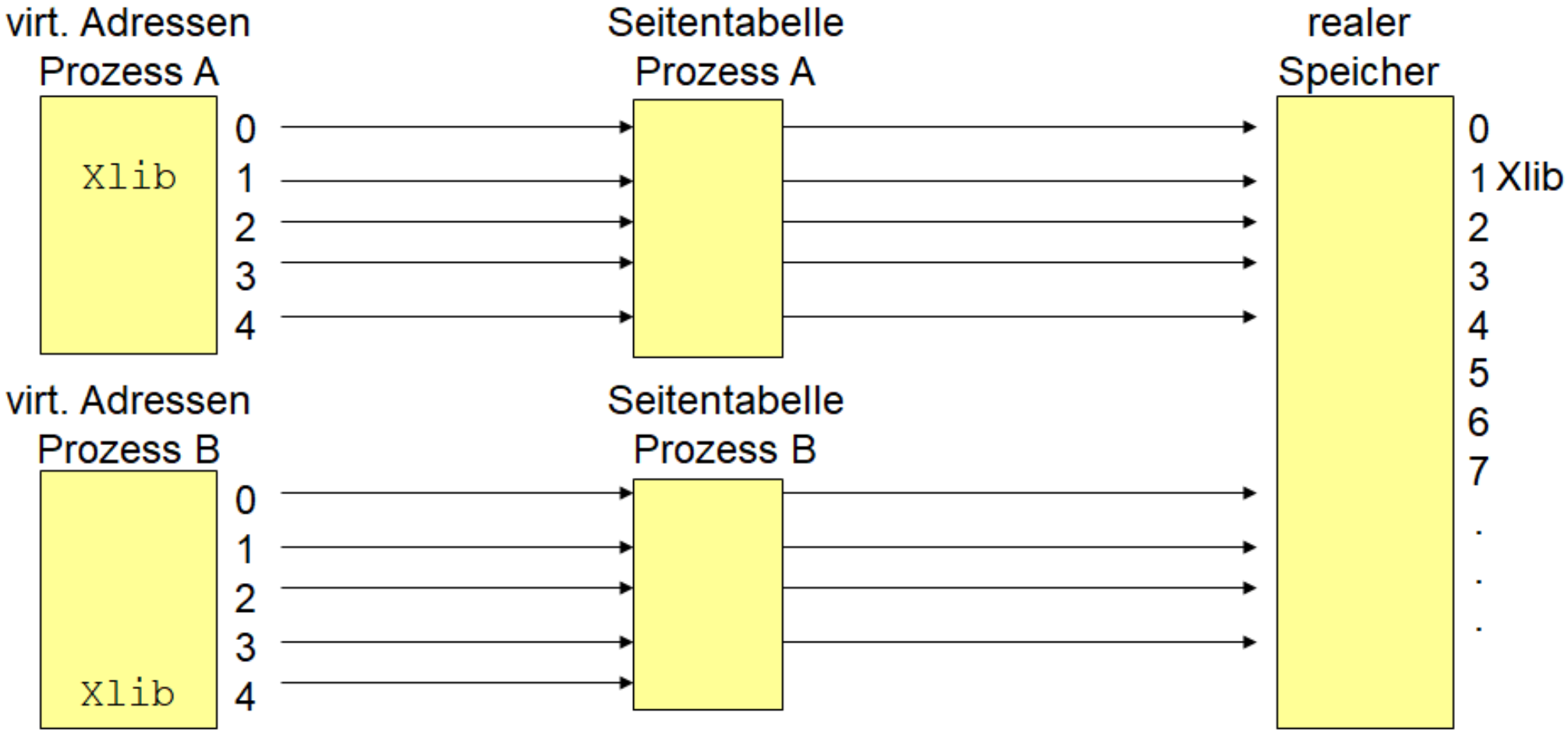
Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



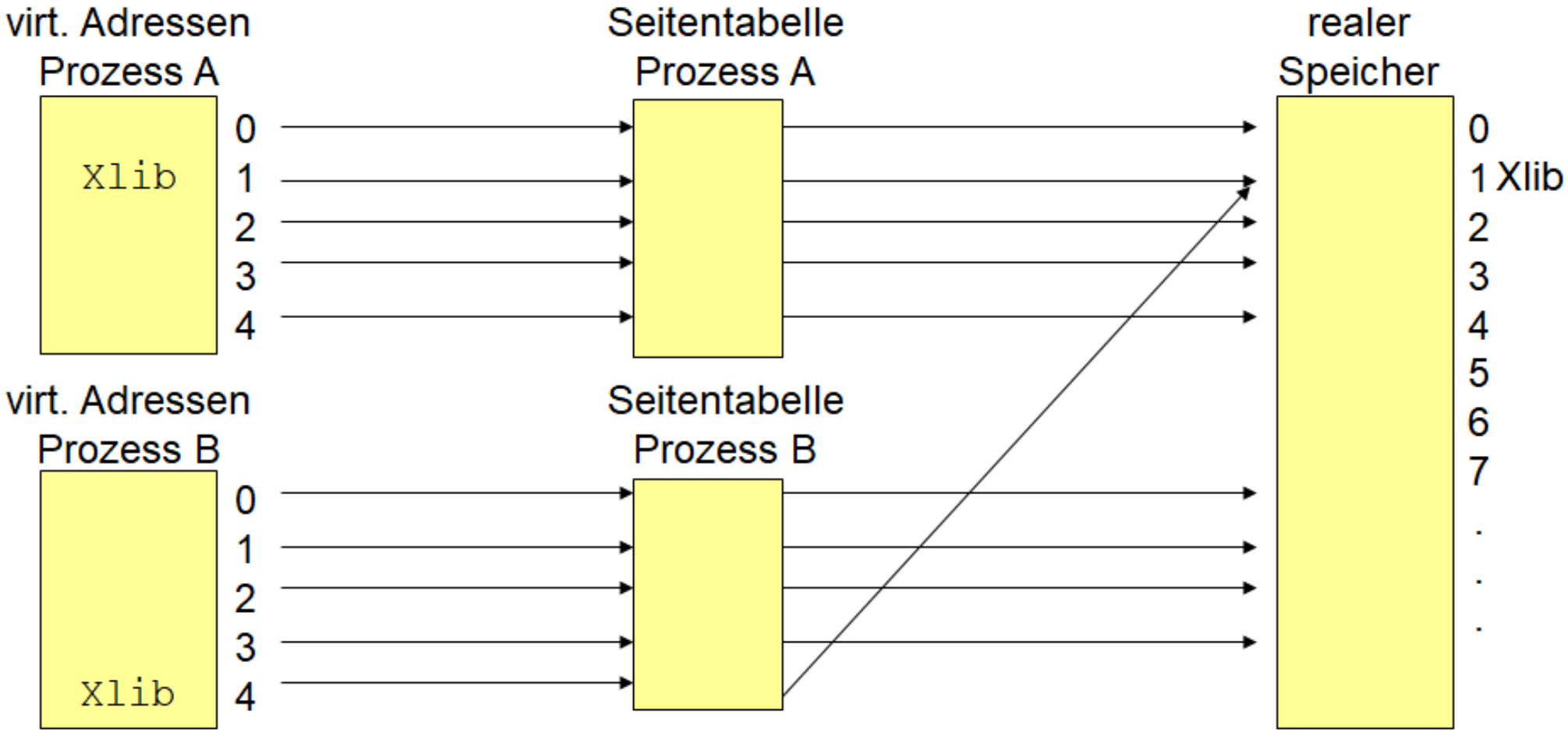
Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



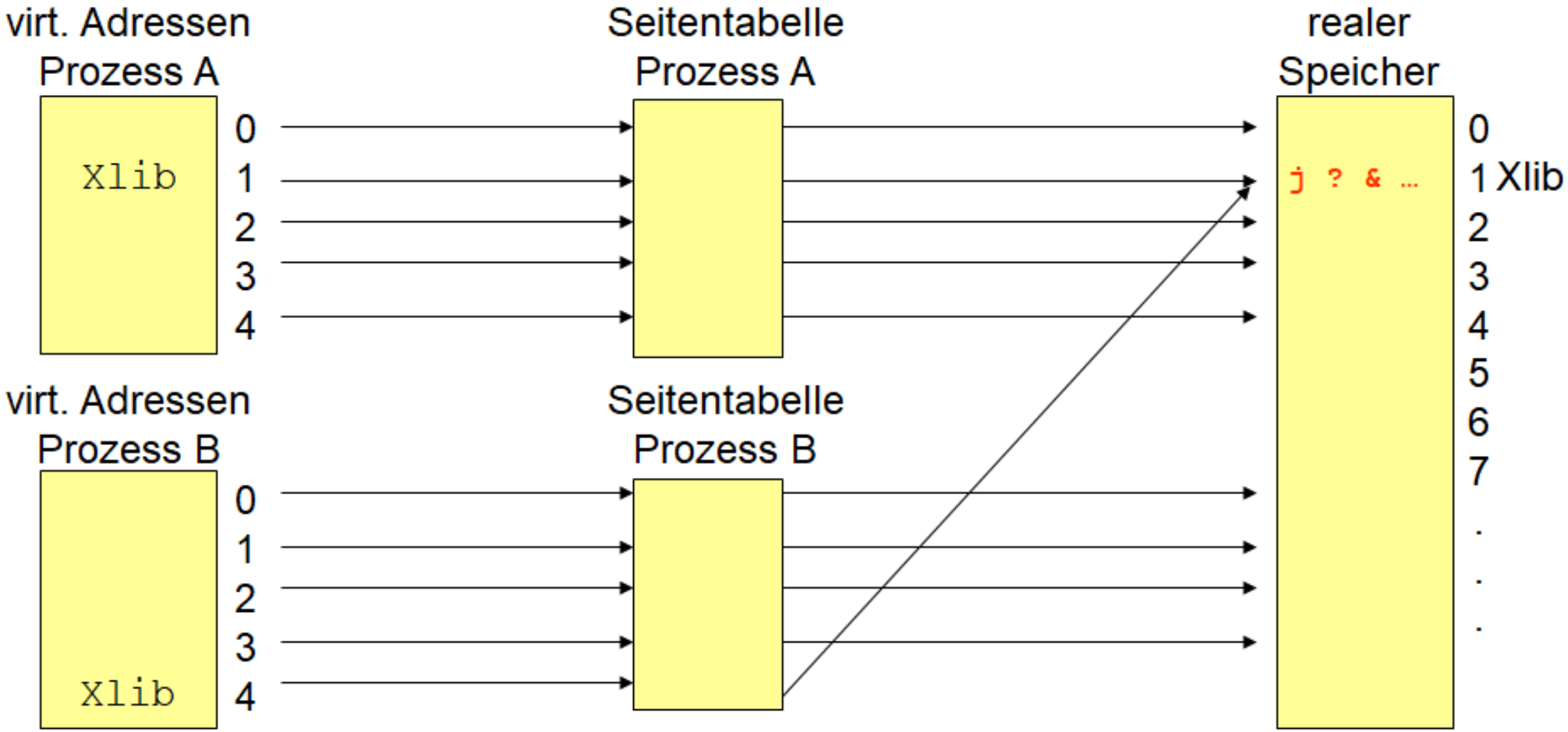
Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



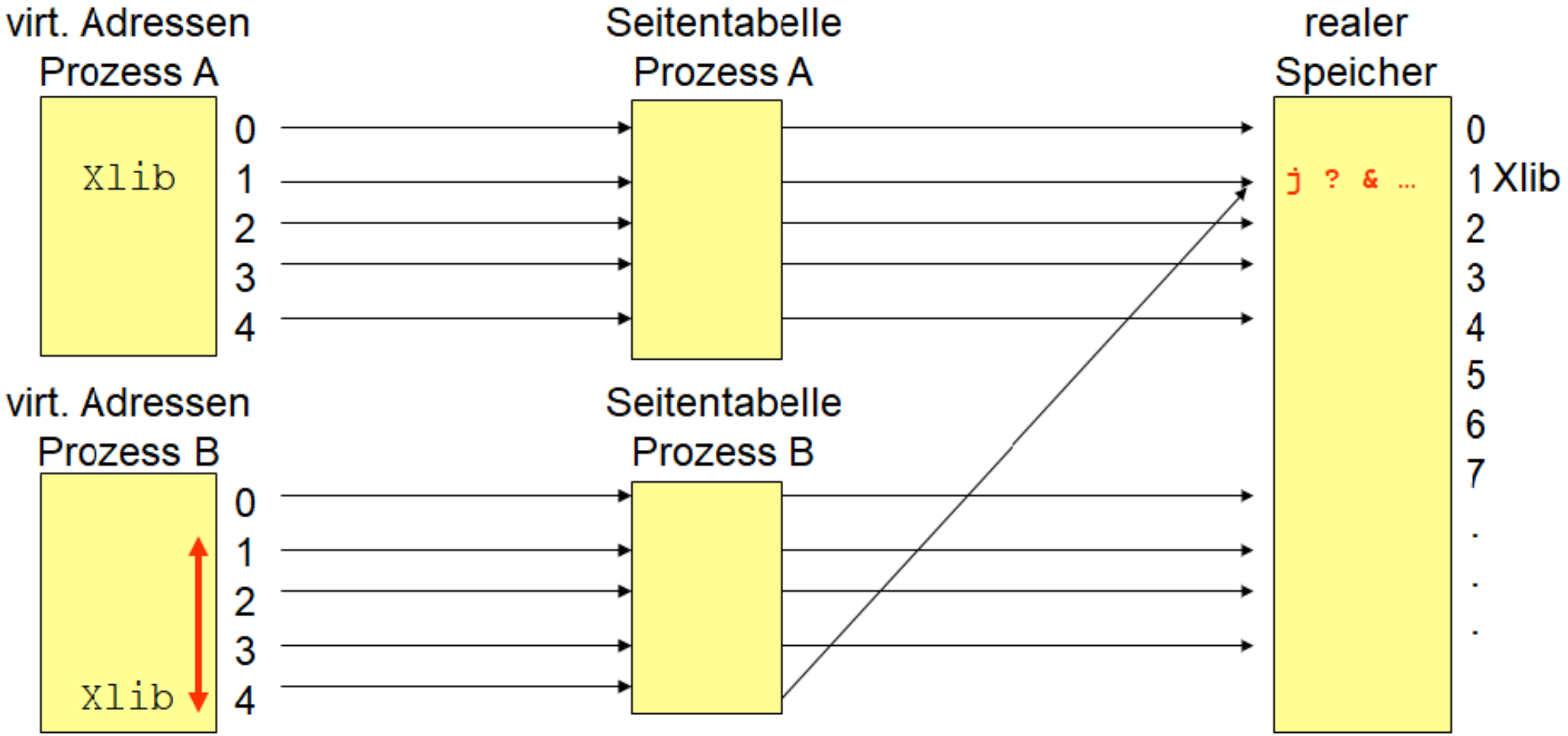
Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



Sharing (shared libraries)

- **Einfache Idee**: Seiten-Abbildung mehrerer Prozesse führen zur selben Kachel



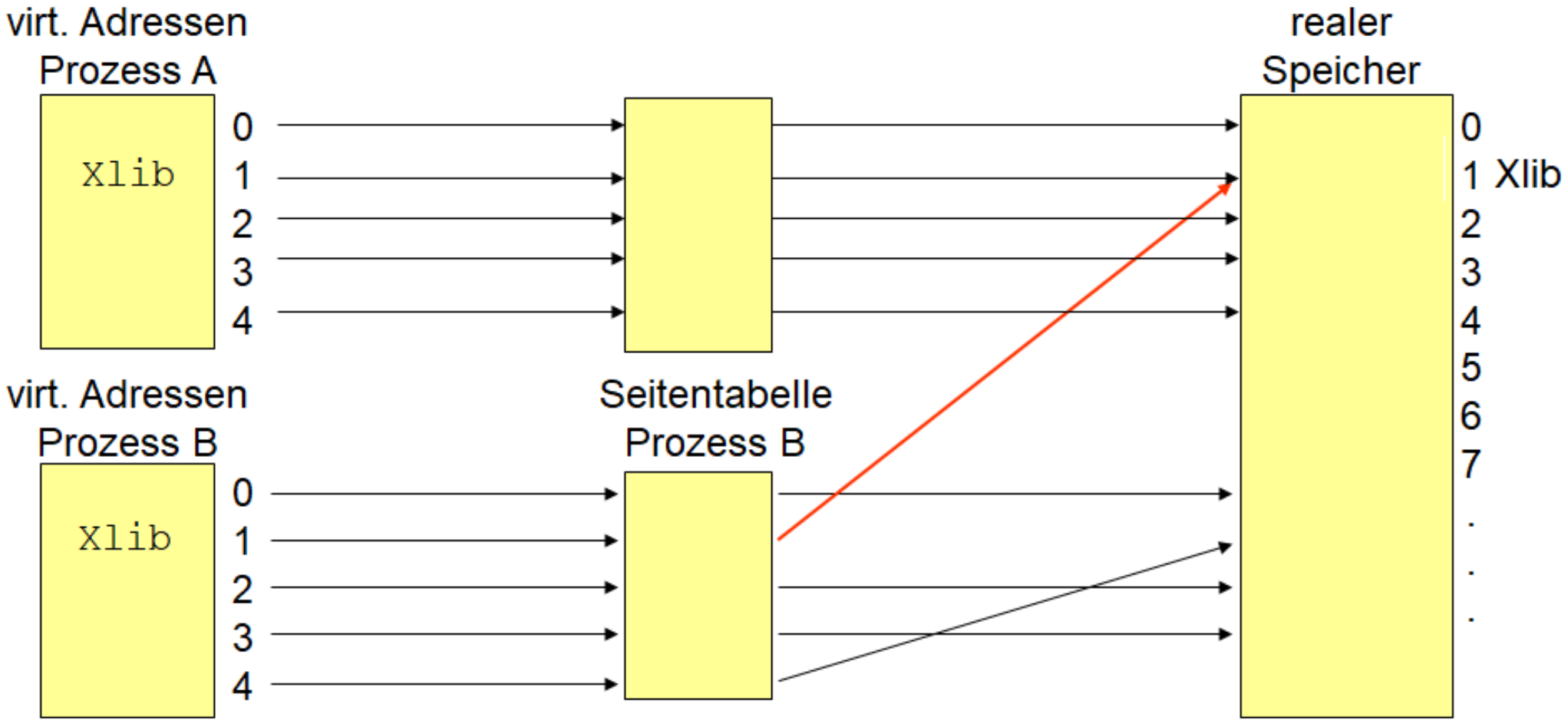
Idee ist so nicht realisierbar

- Im gemeinsam genutzten Code kommen ggf. nicht-relative (virtuelle) Befehlsadressen vor, die sich z.B. auf Adressen in diesem Code beziehen
- Adressen müssen Seitennummern verwenden, die nach Abbildung auf die realen Adressen über die jeweiligen Seitentafeln auf gemeinsam genutzten Bereich zeigen
- Wenn diese Seitennummern ≠ sind:
  → nicht im Code eintragbar (in 1 Speicherzelle nur 1 Seitennummer).
  (Bei Segmentadressierung mit separaten Segmentregistern ginge es!)

→ Nutzen Prozesse gemeinsamen Code, so muss dieser bei der Seitenadressierung bei allen beteiligten Prozessen auf dieselben virtuellen Adressen abgebildet werden.

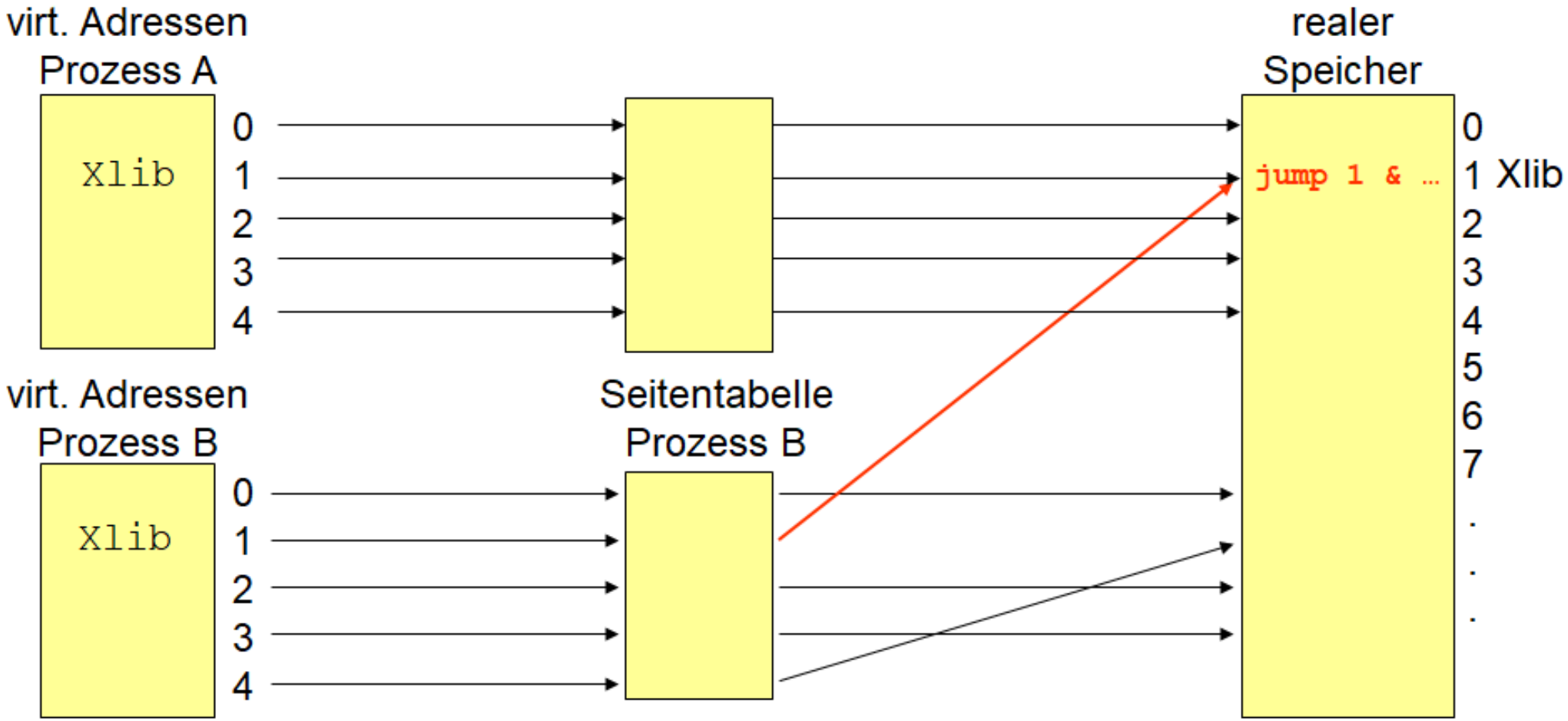
Realisierbares Sharing

- *Shared libraries* befinden sich an derselben Stelle des virtuellen Adressraums



Realisierbares Sharing

- *Shared libraries* befinden sich an derselben Stelle des virtuellen Adressraums



Kombination von Segment- und Seitenadressierung

Segmentadressierung besitzt:

- Vorteile beim *Sharing*, wenn Segmentnummern in Registern gespeichert werden.
- Nachteile durch *externe* Fragmentierung

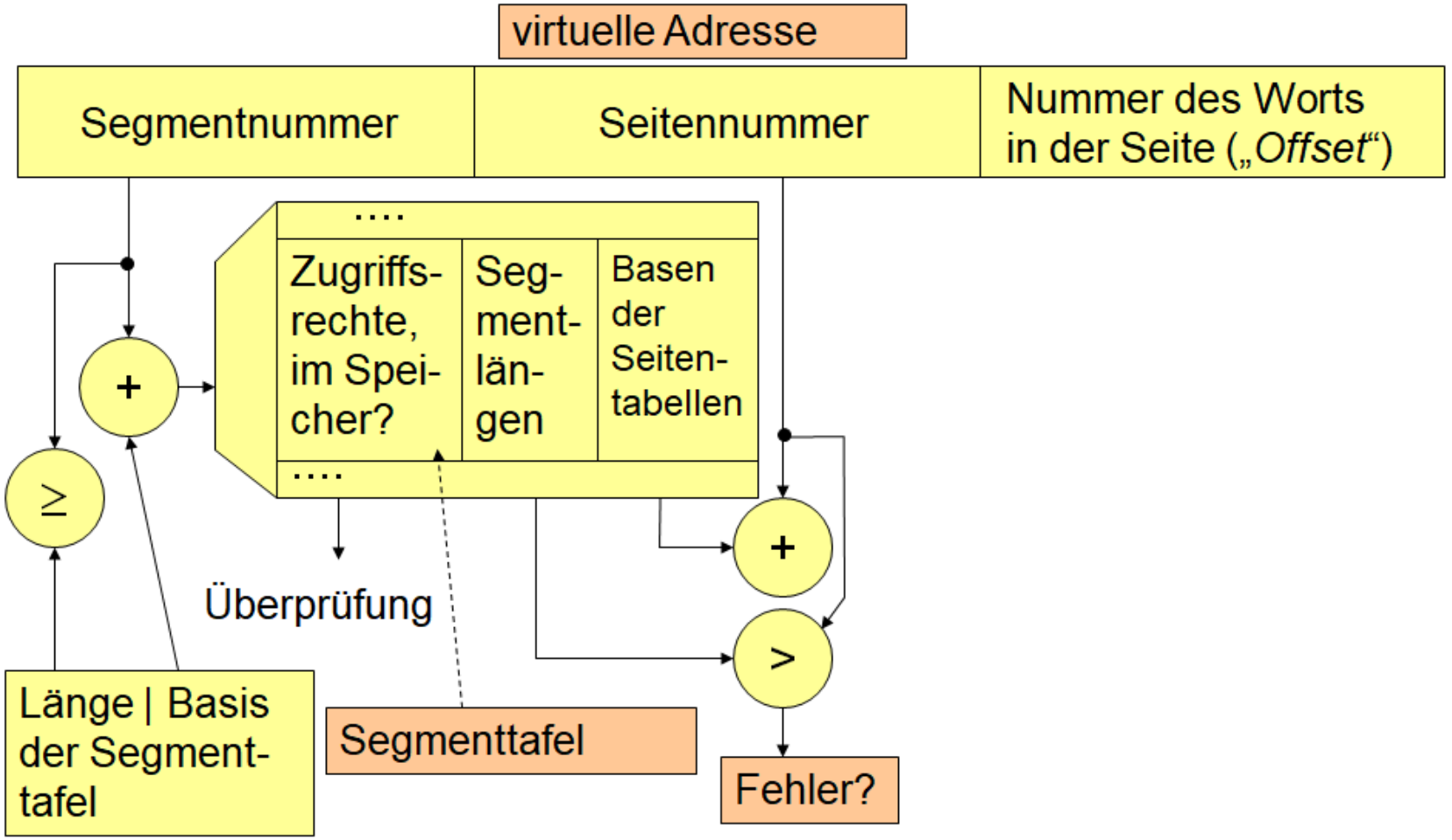
Kombination von Segment- und Seitenadressierung:

- Jedes Segment enthält Seiten, diese wiederum Worte
- Fragmentierung reduziert sich auf interne Fragmentierung
- Jede Kachel kann jede Seite aufnehmen (kein Kopieren)
- Vorteile bei *shared libraries*

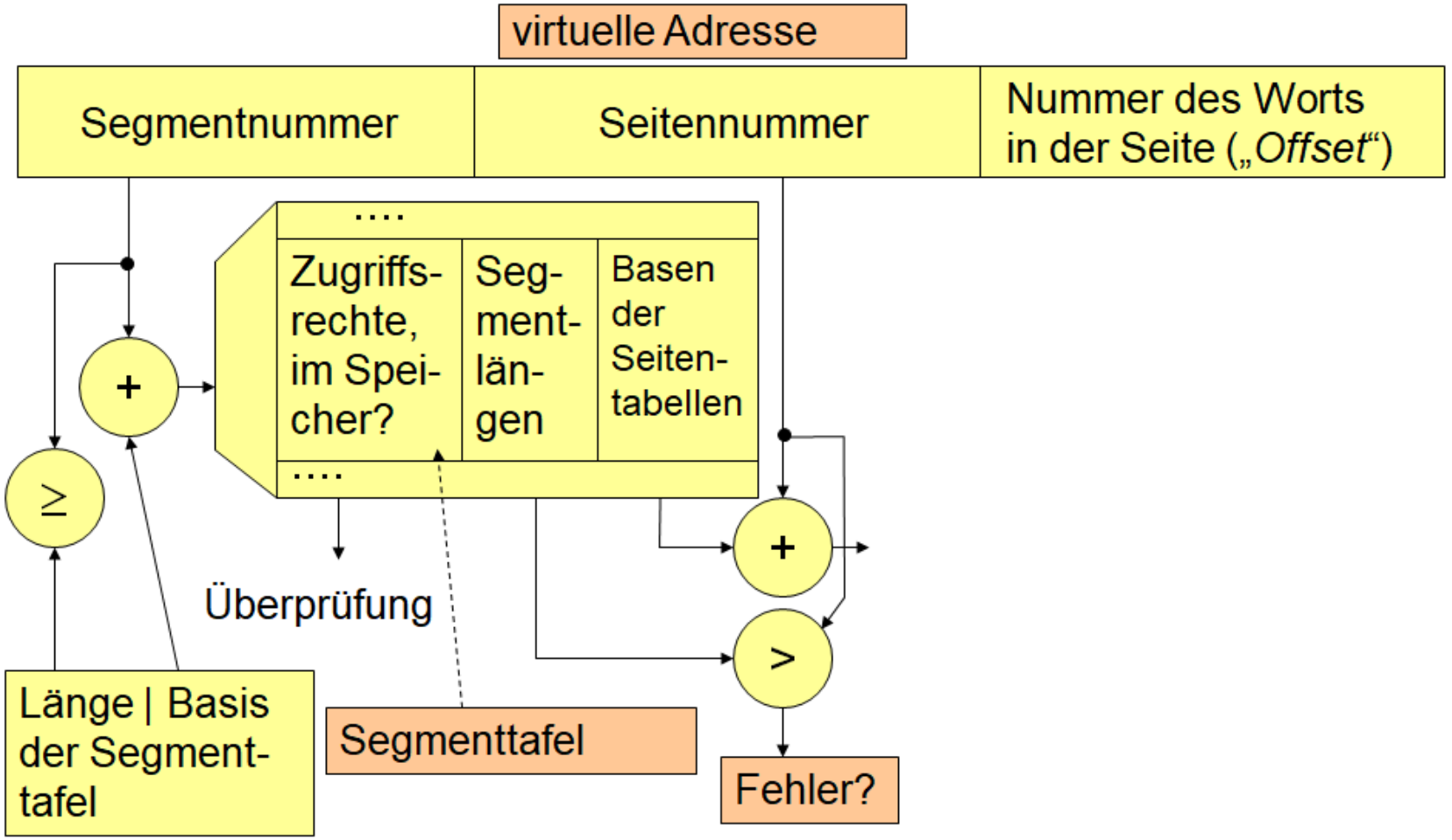
Virtuelle Adressen:

Segmentnr.	Seitennr.	Offset

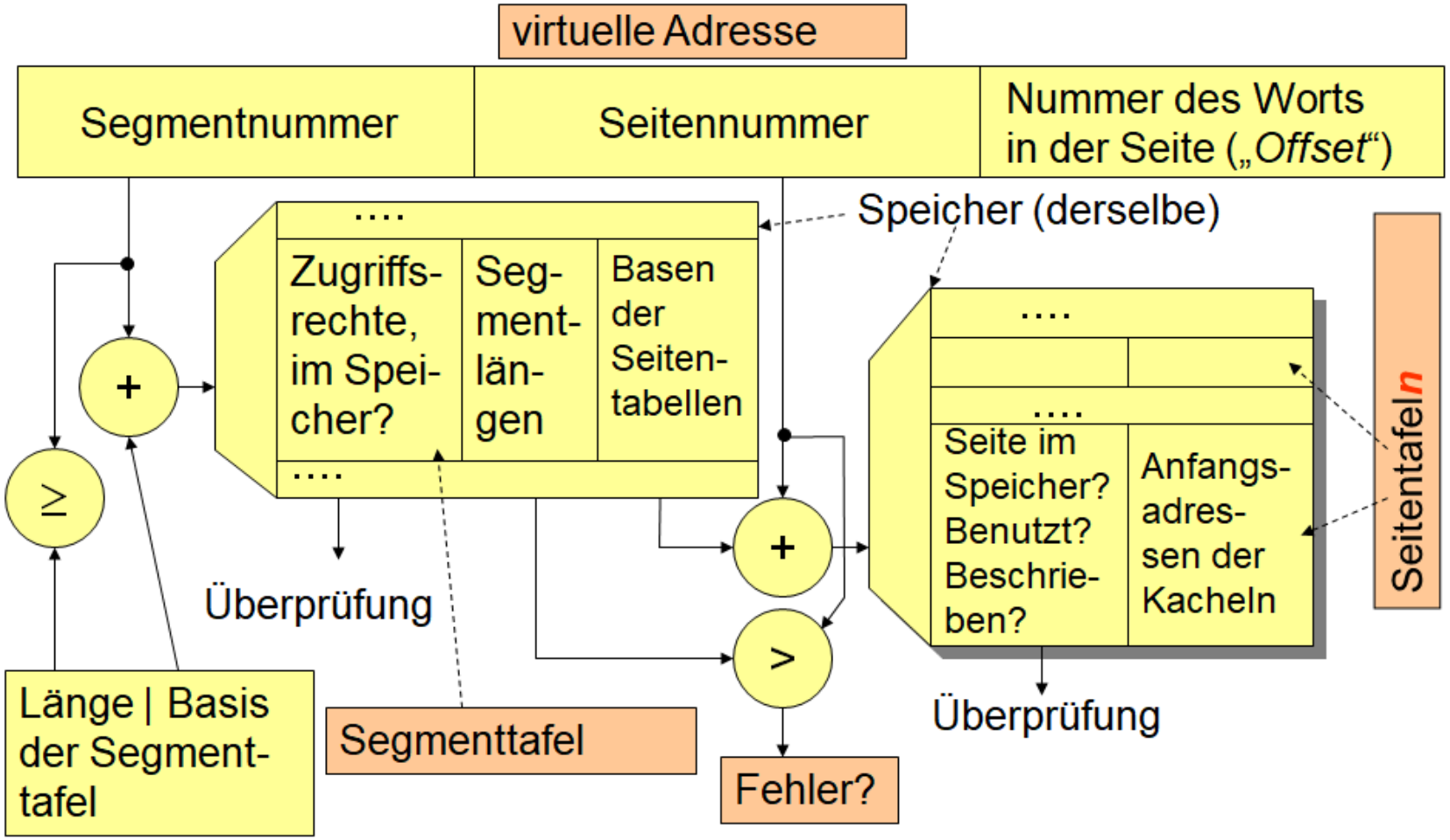
Segmentadressierung mit Seitenadressierung



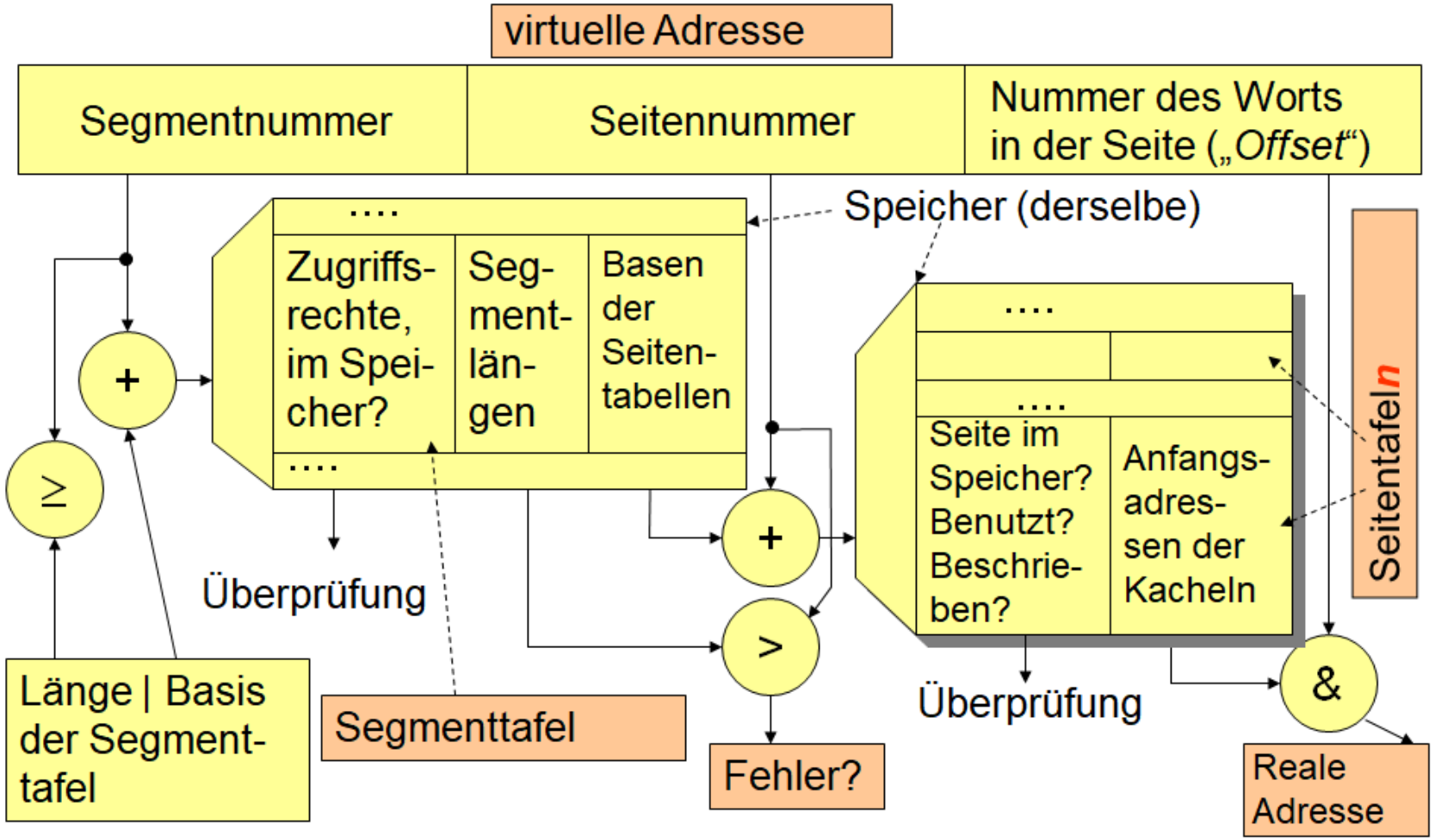
Segmentadressierung mit Seitenadressierung



Segmentadressierung mit Seitenadressierung



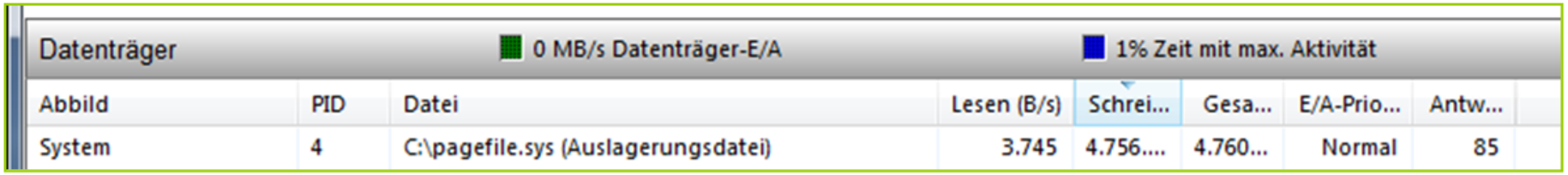
Segmentadressierung mit Seitenadressierung



Demand paging
(Realisierung einer Speicherhierarchie)

- Wenn größerer und langsamerer Speicher (HD) verfügbar:
  - Einteilung des Speichers in Bereiche konstanter Länge
  - Automatisches Aus- und Einlagern
    - Selten benutzte Seiten auslagern
    - Einlagern, wenn auf ausgelagerte Seiten zugegriffen wird.
- Auslagern einzelner Seiten auf die Platte und Rückschreiben auf beliebige Kachel möglich.

Beispiel: Windows Resource Monitor:



Zusammenfassung

- Reale und virtuelle Adressen, *memory management*
- Formen der Speicherverwaltung
  - Identität
  - Segmentadressierung
  - Seitenadressierung
    - Sharing
  - Segmentadressierung mit Seitenadressierung

Mögliche Stufen der Speicherhierarchie und derzeit eingesetzte Technologien

Tertiärspeicher (Optisch, Bänder, Cloud) (persistent)
Sekundärspeicher (Platten, Flash) (persistent)
Plattencaches (DRAM, Flash)
Haupt- bzw. Primärspeicher (DRAM)
Caches (SRAM) & TLBs
Register (SRAM)

2.4.3 Translation Look-Aside Buffer (TLBs)

Seitentabellen und Segmenttabellen im Hauptspeicher:

- Jeder Speicherzugriff erfordert zusätzliche Zugriffe zur Umrechnung von virtuellen in reale Adressen.
- Unakzeptable Verlangsamung der Ausführung.
- Einführung kleiner, schneller Hardwarespeicher, welche häufig die benötigten Tabelleneinträge enthalten.

Diese heißen translation look aside buffer (TLB) oder auch address translation memory (ATM).

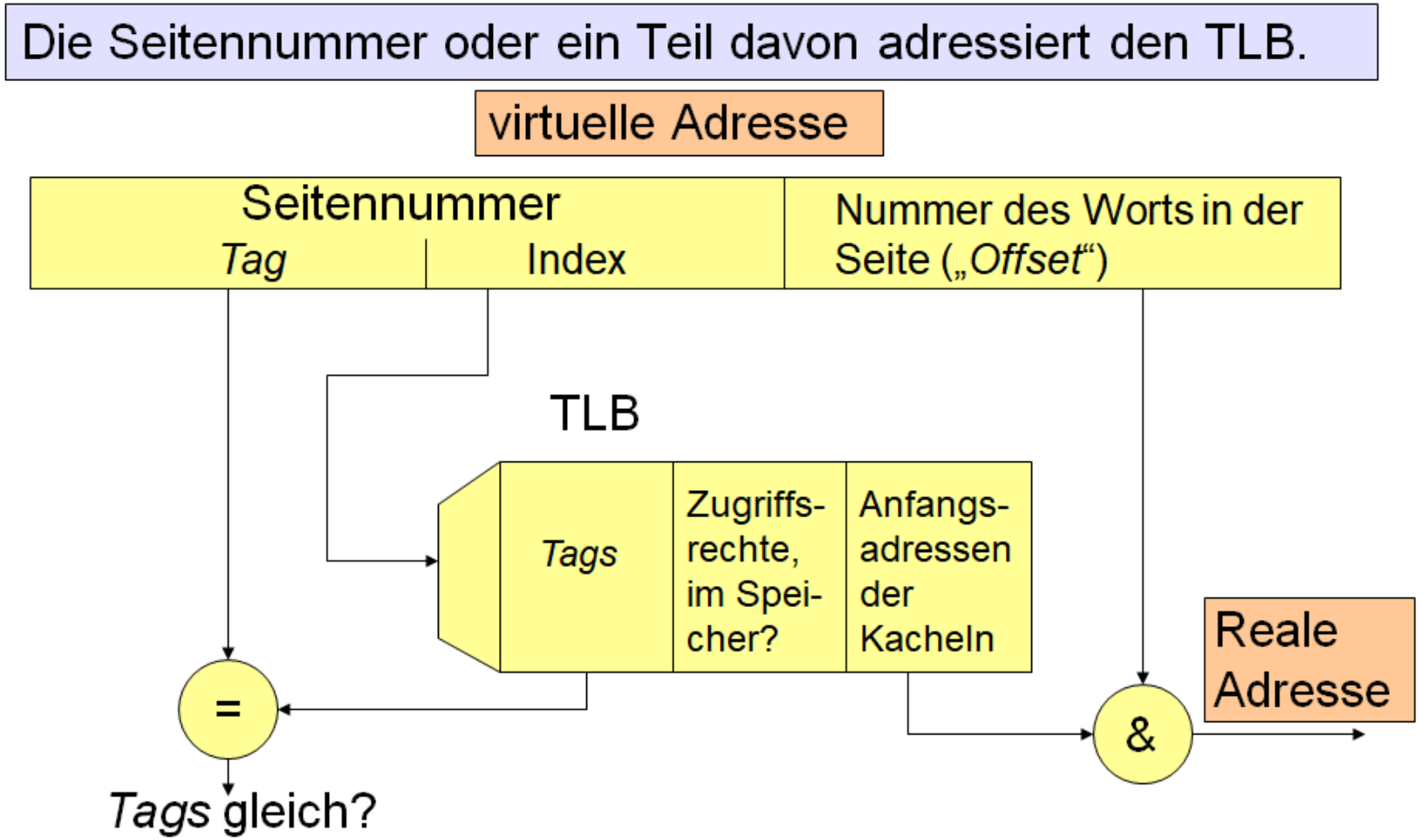
- TLBs enthalten Auszüge aus den vollständigen Tabellen
- Gesuchte Seiten- bzw. Segmentnummer nicht im TLB: Nachsehen in der vollständigen Tabelle, Eintrag aus der vollständigen Tabelle verdrängt einen Eintrag im TLB

Die 3 Organisationsformen von TLBs

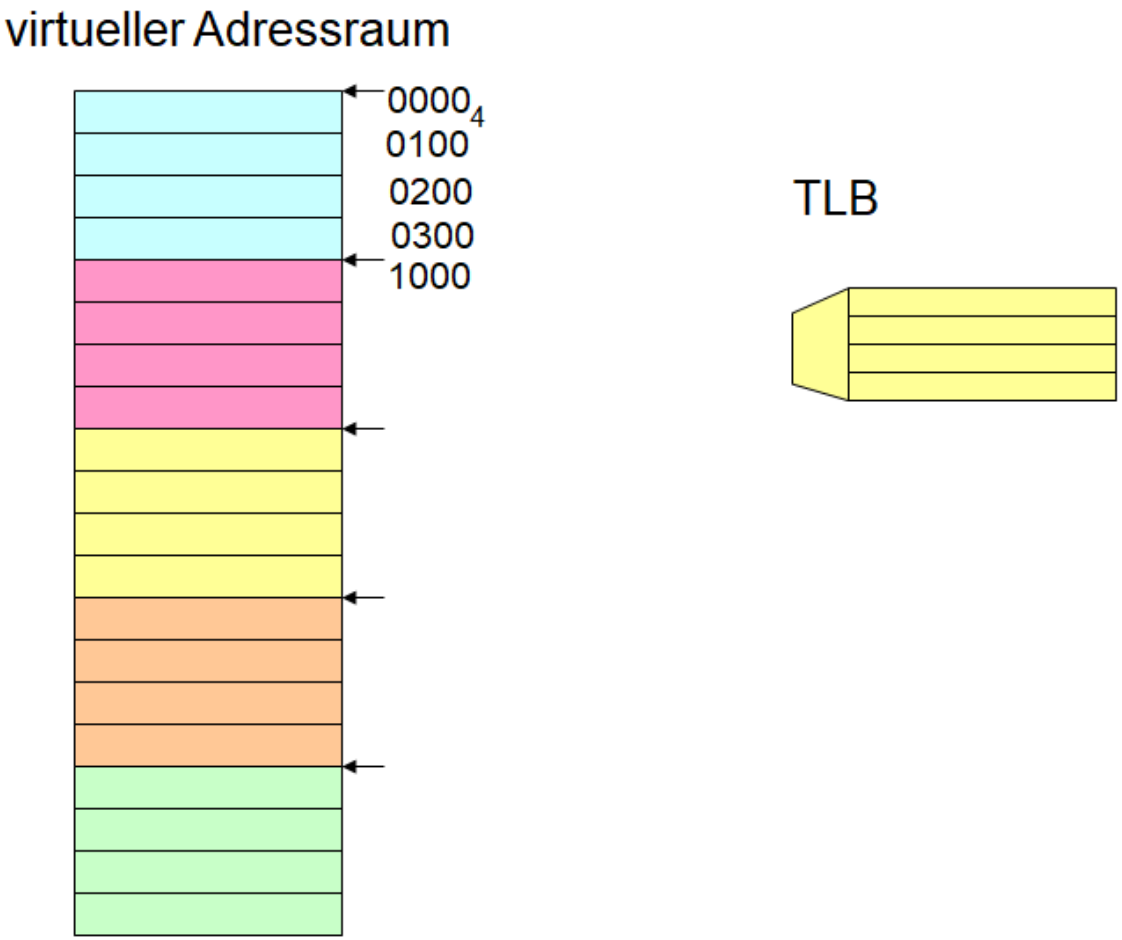
Drei Organisationsformen von TLBs:

1. Direct mapping,
2. Mengen-assoziative Abbildung
   (set associative mapping),
3. und Assoziativspeicher
   (associative mapping).

Direct Mapping

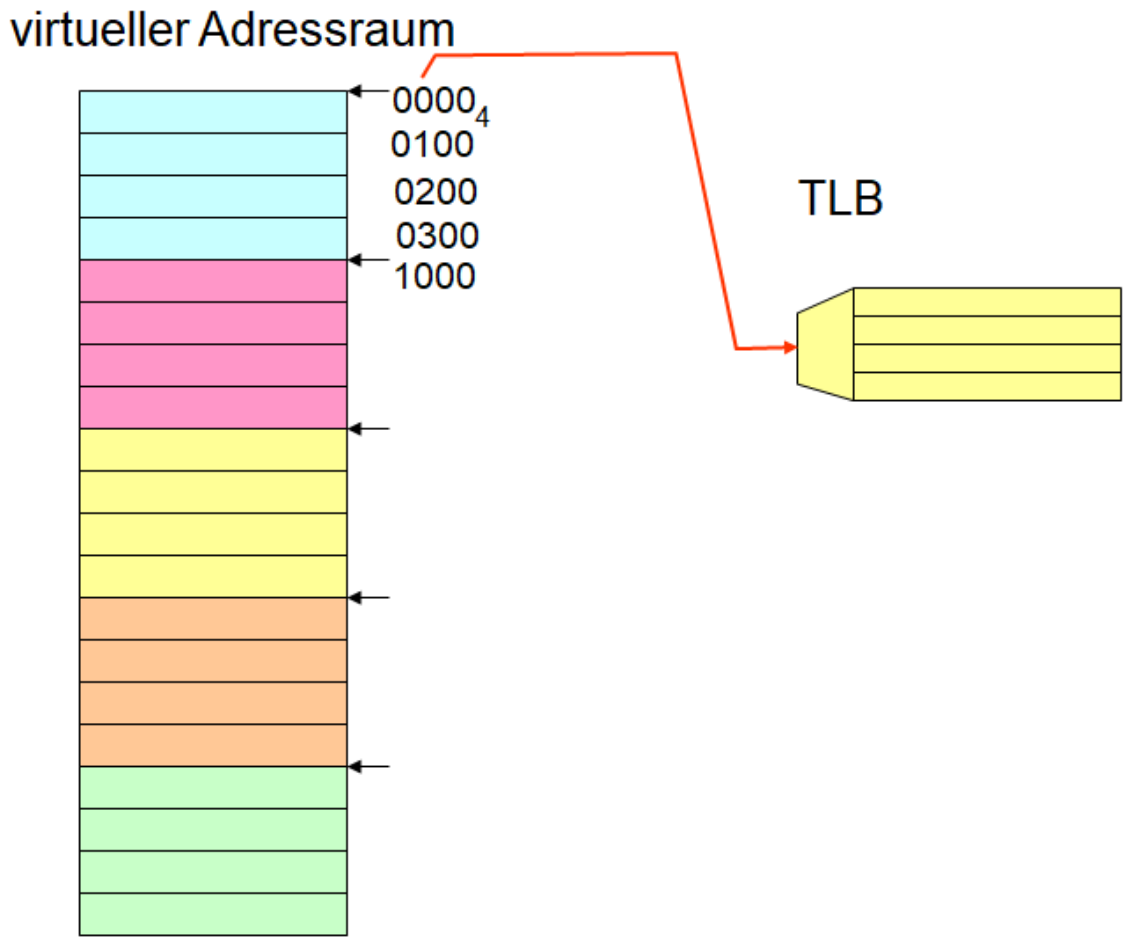


Aliasing bei direct mapping



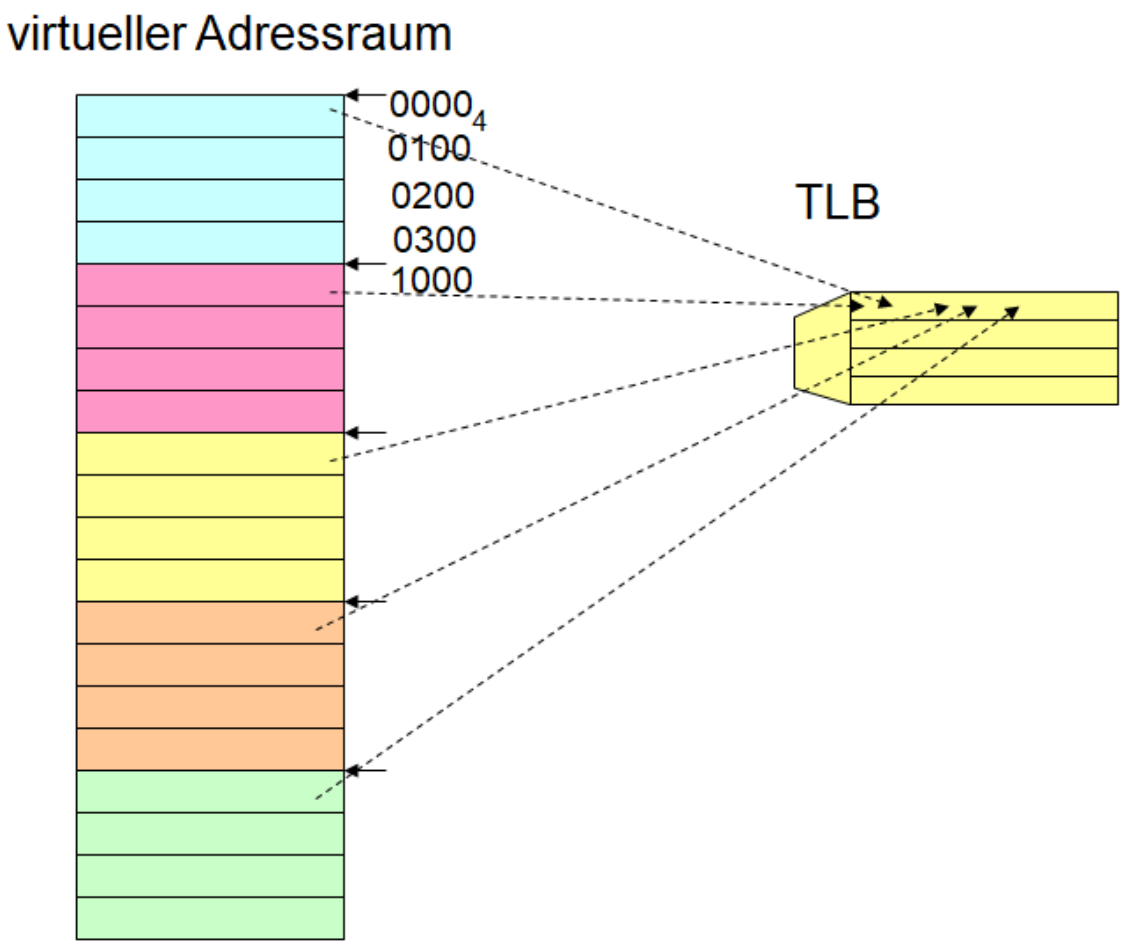
- Seitennummer (2 Bit Tag, 2 Bit Index) und Offset (4 Bit)

Aliasing bei direct mapping



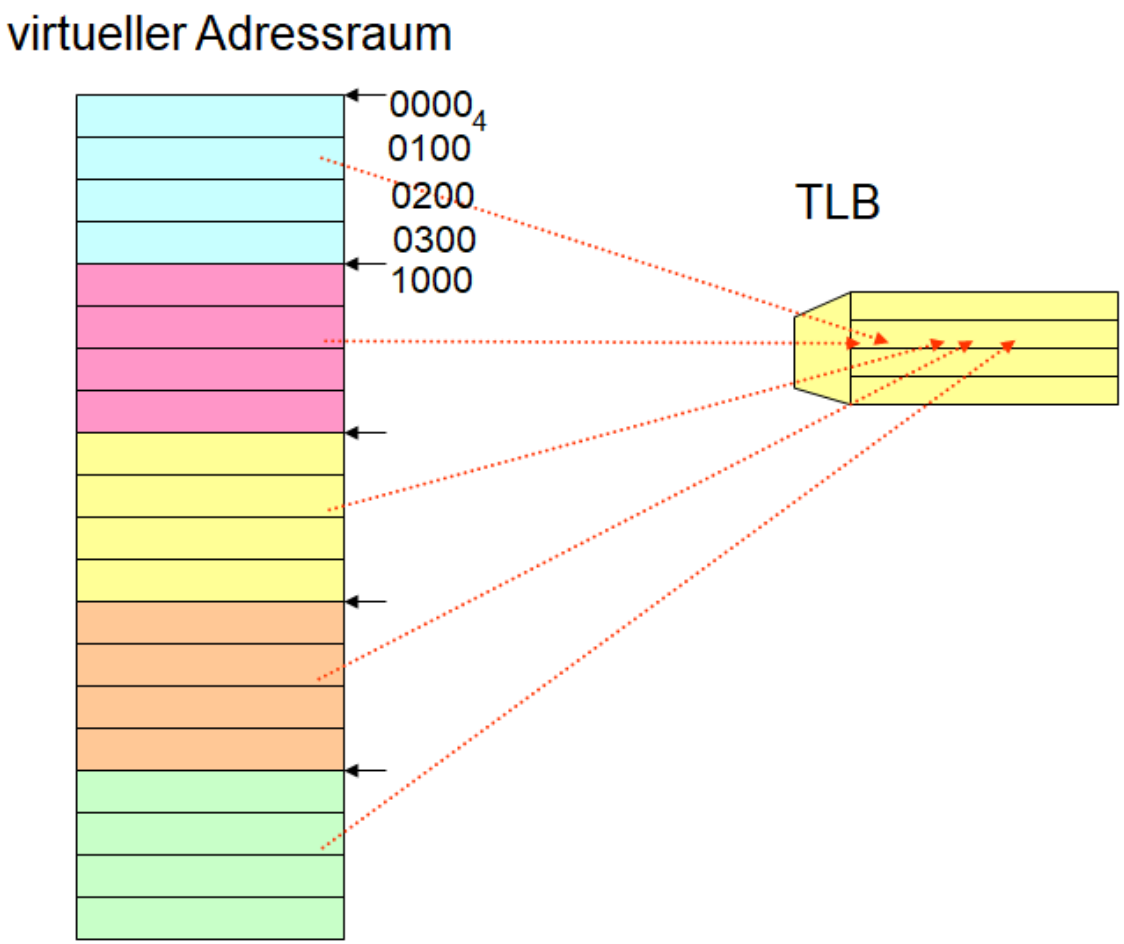
- Seitennummer (2 Bit Tag, 2 Bit Index) und Offset (8 Bit)

Aliasing bei direct mapping



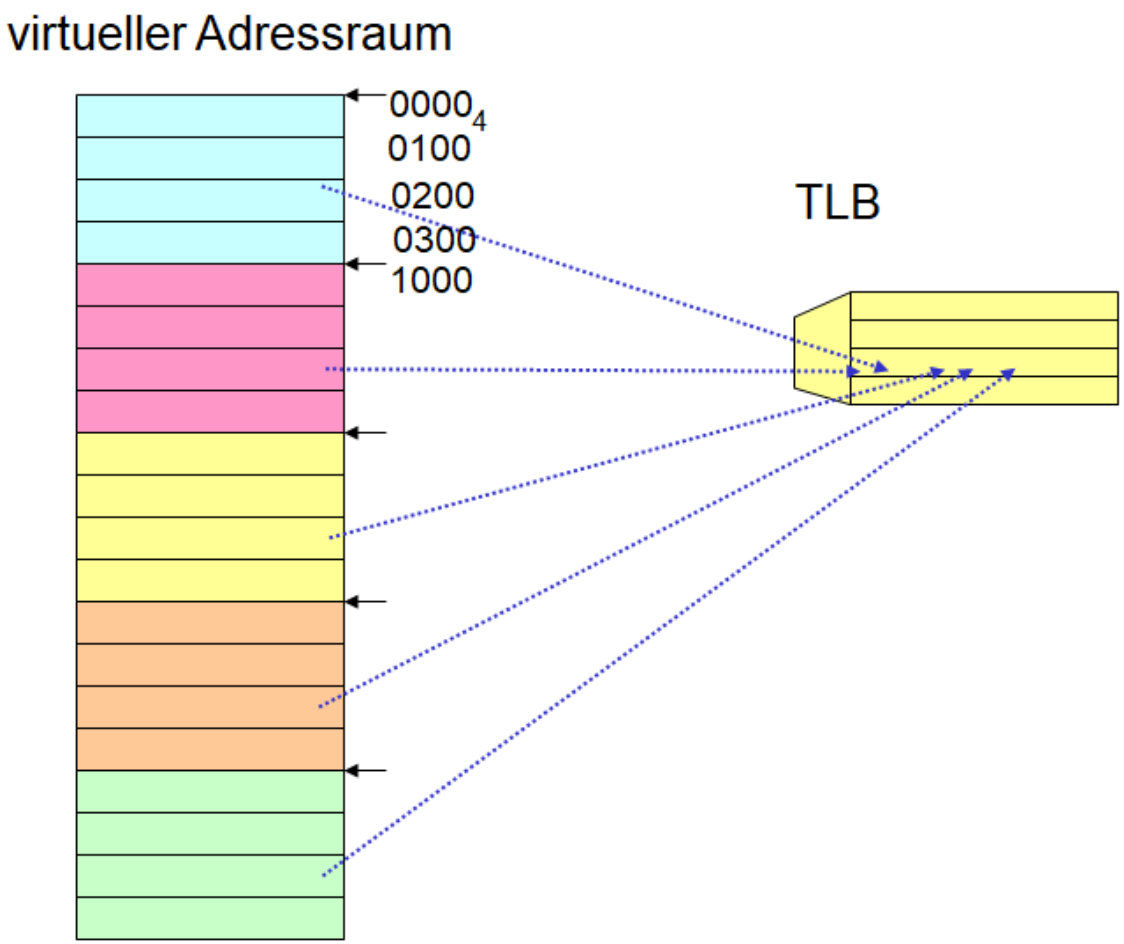
- Seitennummer (2 Bit Tag, 2 Bit Index) und Offset (8 Bit)

Aliasing bei direct mapping



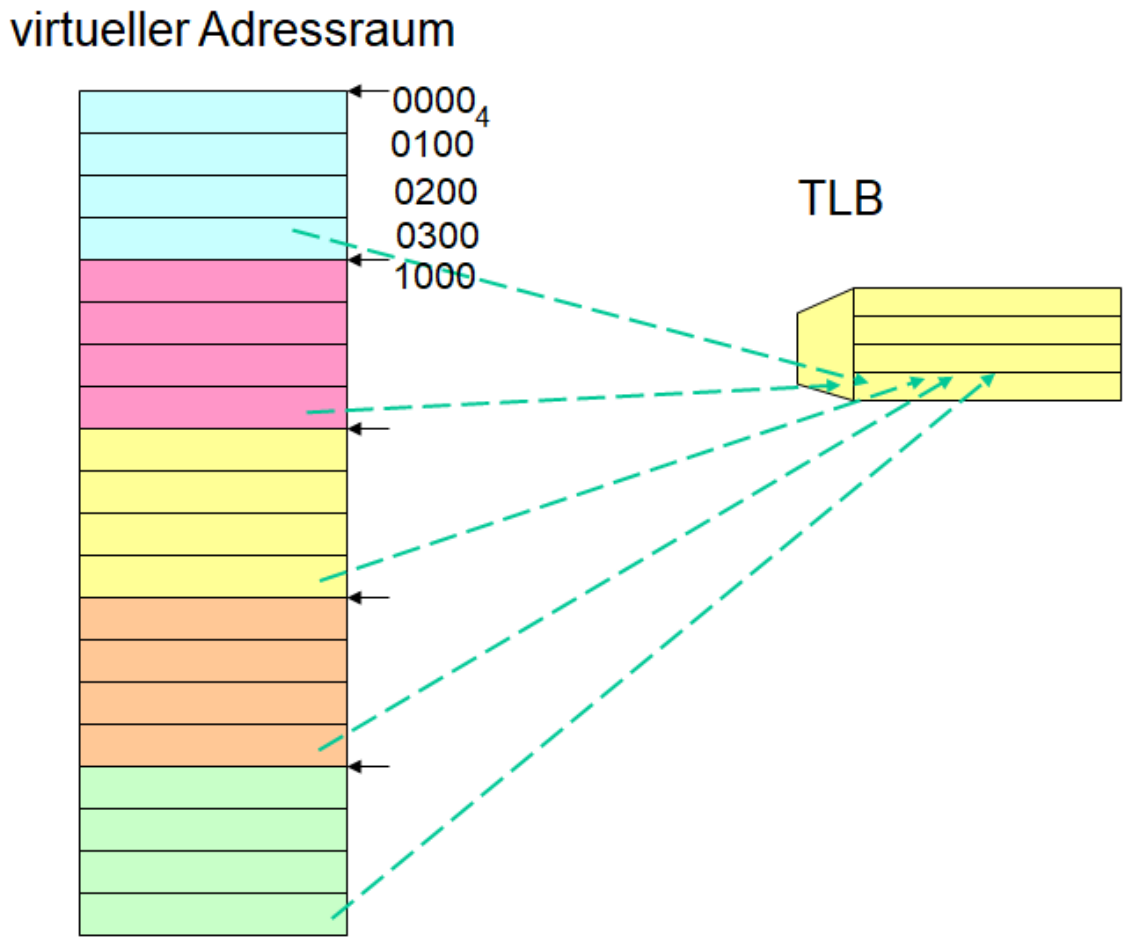
- Seitennummer (2 Bit Tag, 2 Bit Index) und Offset (8 Bit)

Aliasing bei direct mapping



- Seitennummer (2 Bit Tag, 2 Bit Index) und Offset (8 Bit)

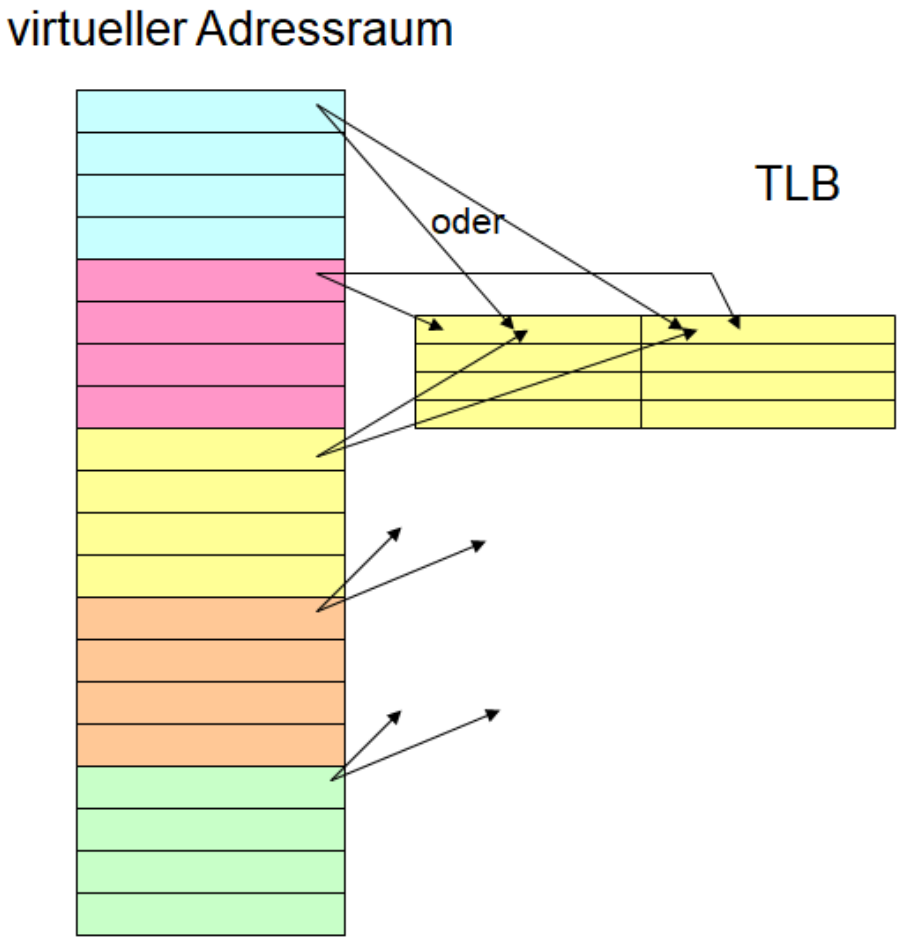
Aliasing bei direct mapping



- Alle Seiten, deren Adressen im Indexteil übereinstimmen, werden derselben TLB-Zelle (demselben Namen (Alias)) zugeordnet.
- Teilweise starkes Anwachsen der Laufzeit.

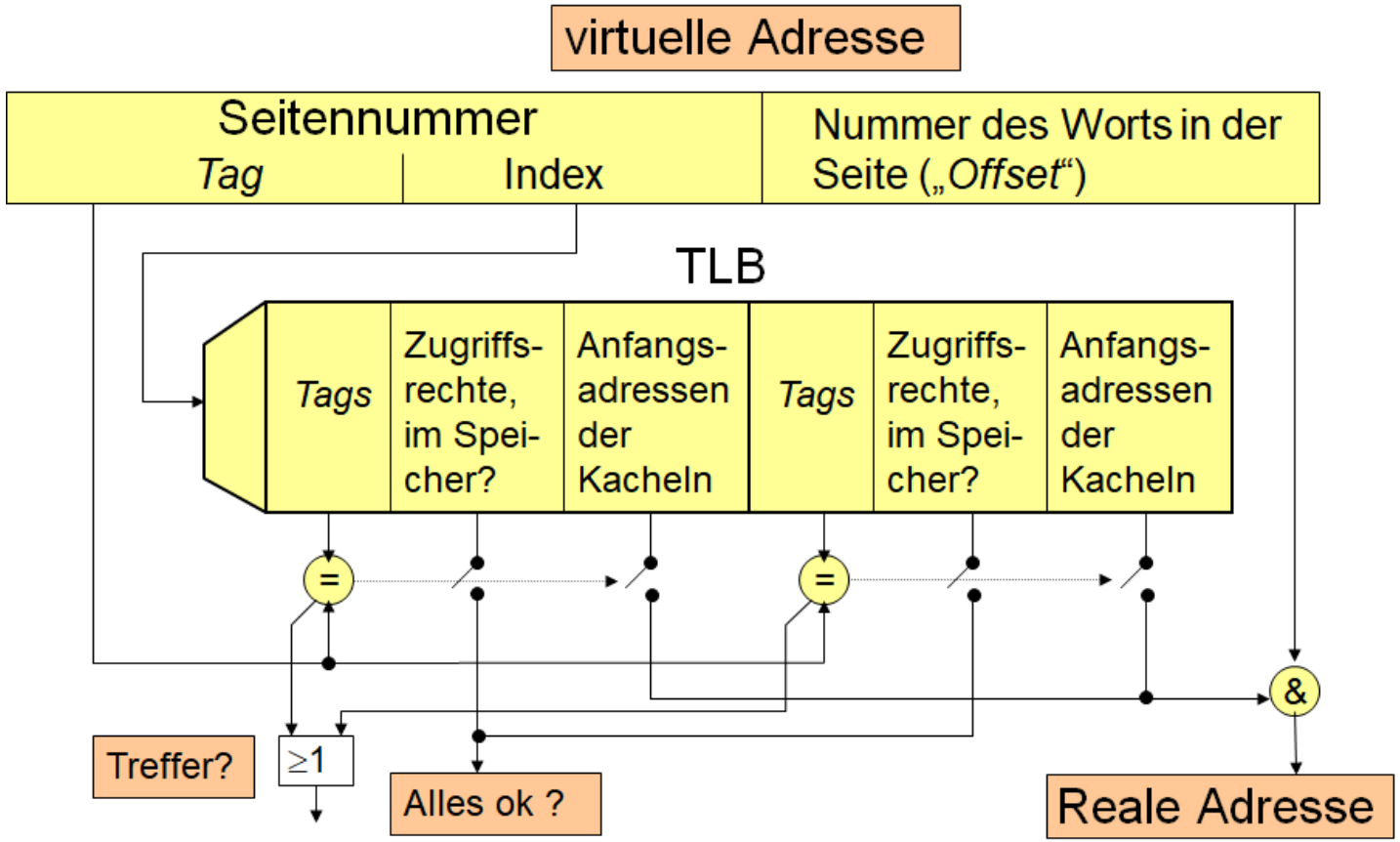
Mengen-assoziative Abbildung: Prinzip n-way set associative mapping

- Zu jedem Index-Wert gibt es n zugeordnete Plätze, mit n>1; n heißt **Assoziativität „n ways“** genannt
- Wenn über die vollständigen Tabellen im Hauptspeicher abgebildet wird, so wird ein Platz im TLB überschrieben, z.B. der am Längsten nicht benutzte (least recently used (LRU)-Strategie)



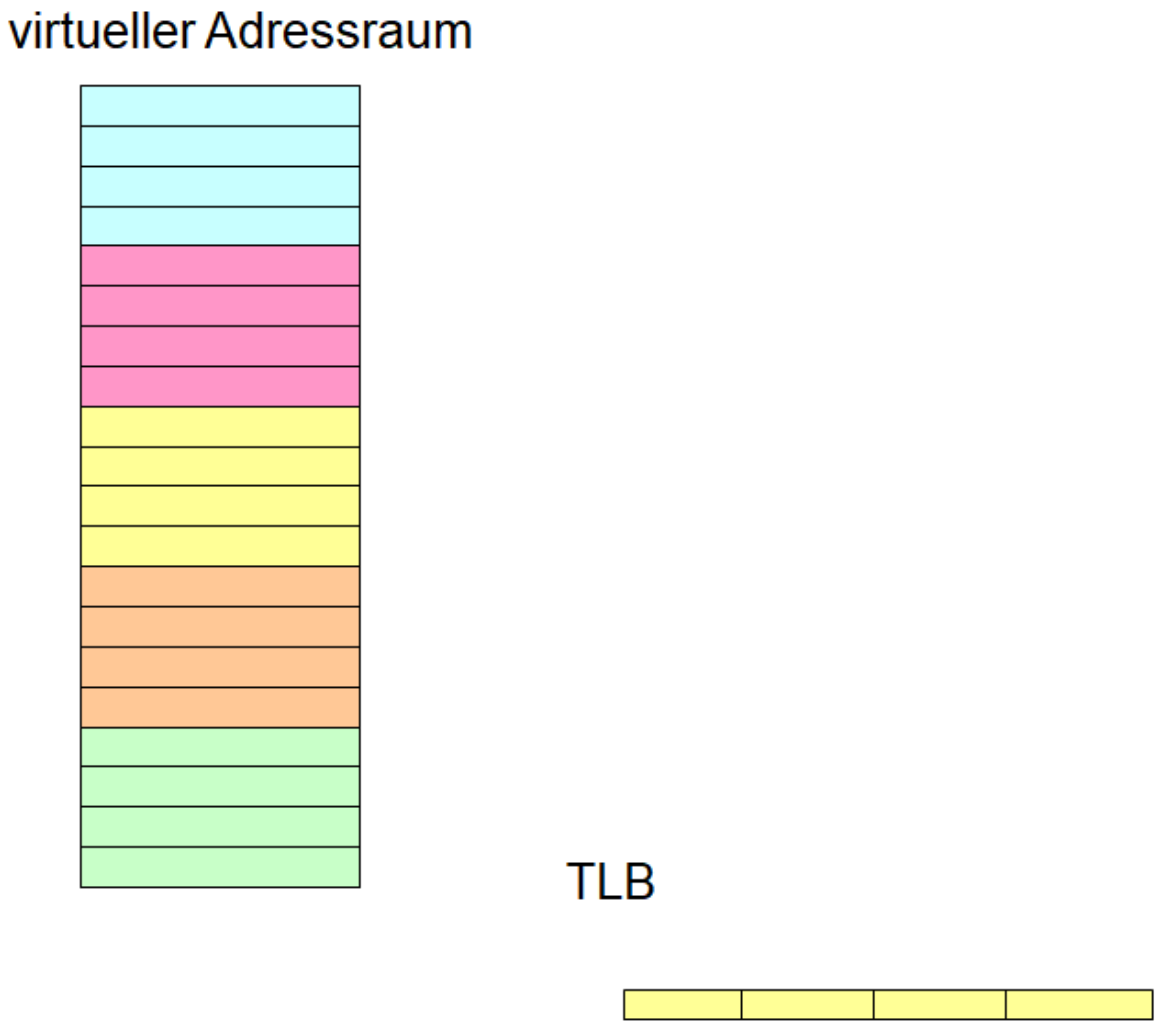
Mengen-assoziative Abbildung Realisierung

Beispiel: Setgröße n=2



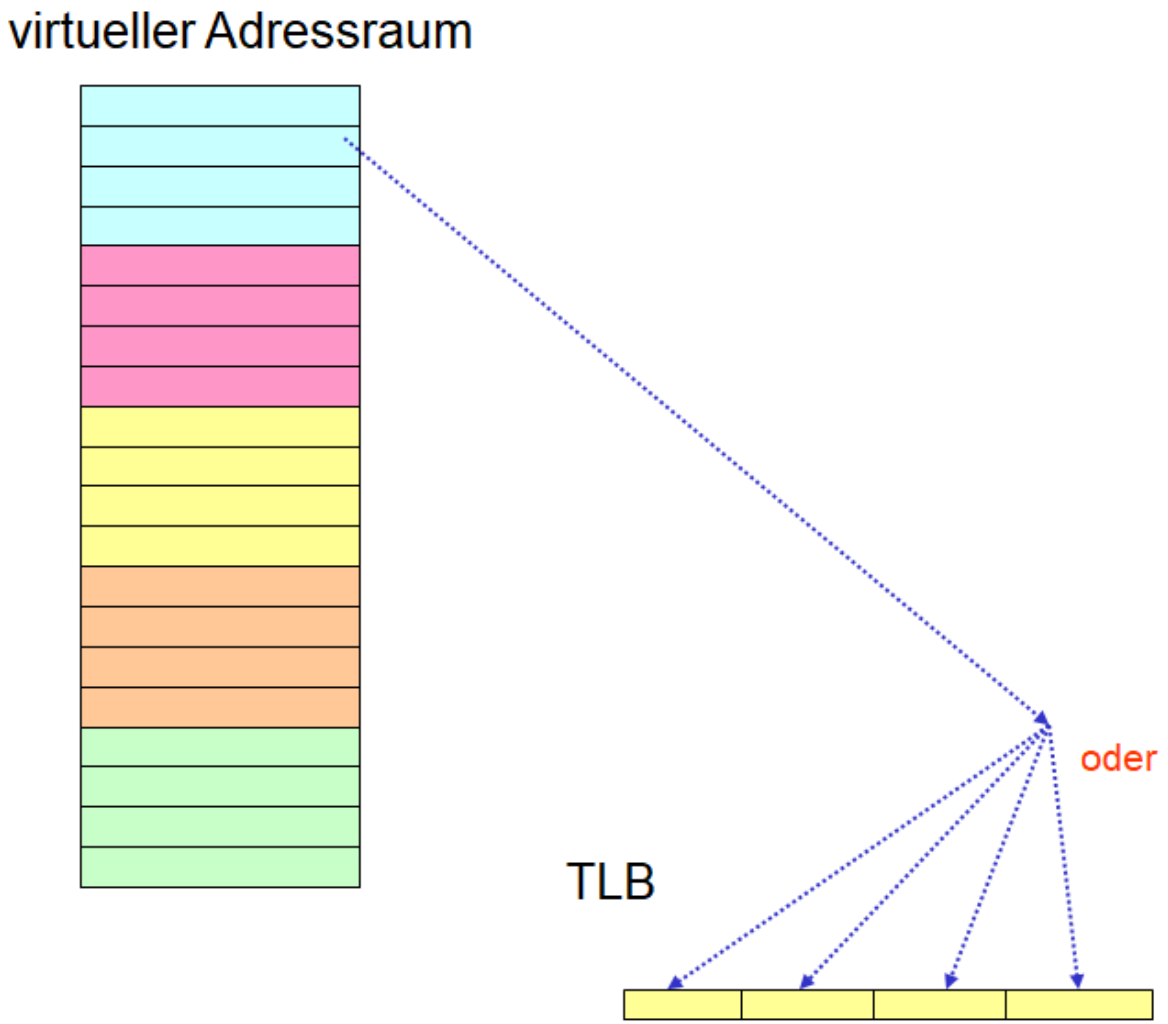
Assoziativspeicher, associative mappingPrinzip

- Der Fall **associative mapping** entsteht aus dem **set associative mapping** für „Anzahl der Bits für den Index-Teil → 0“.
- Jeder Platz des TLB kann die Verwaltungs-Information für jede beliebe Seite enthalten (kein **aliasing**).



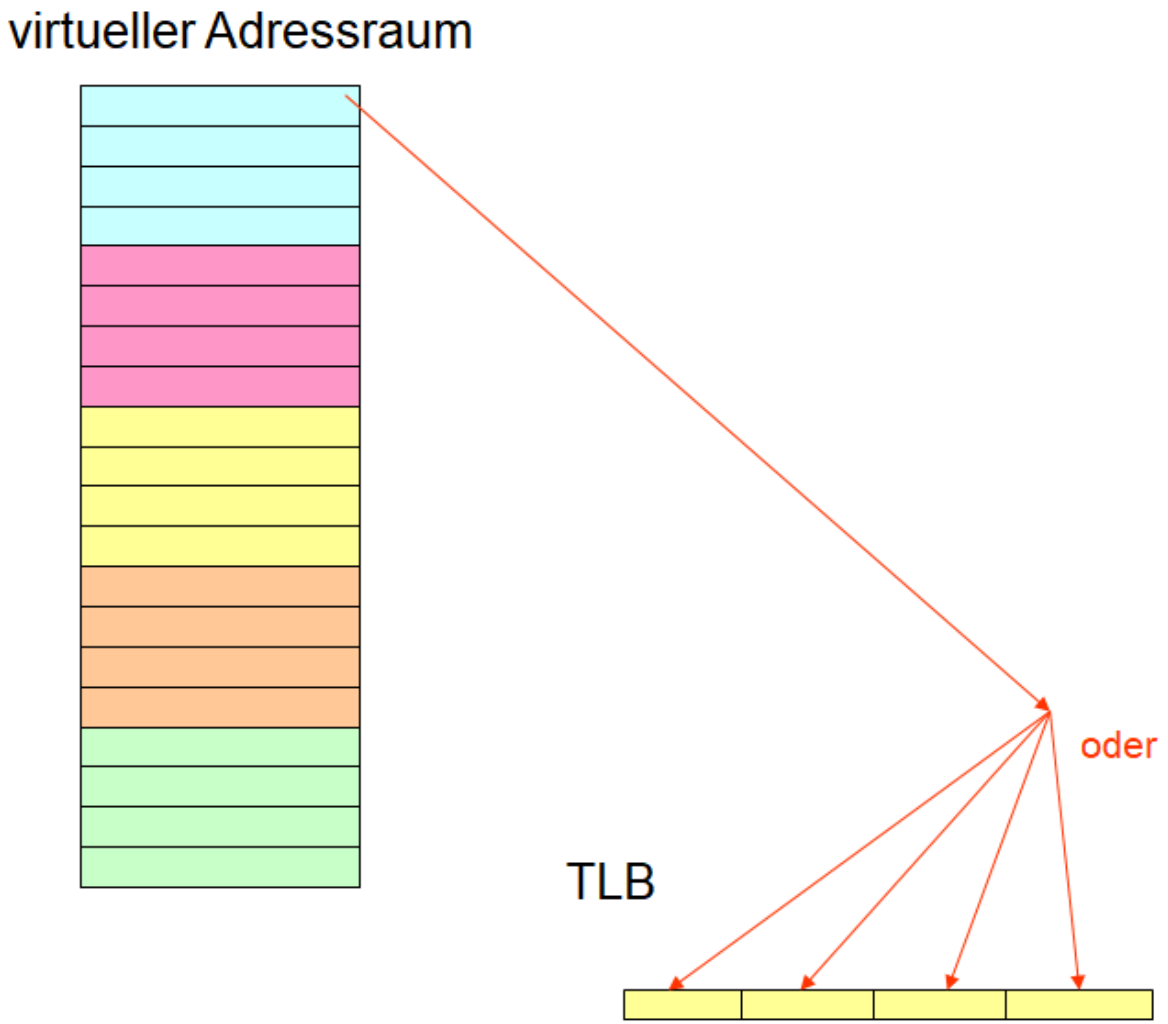
Assoziativspeicher, associative mappingPrinzip

- Der Fall **associative mapping** entsteht aus dem **set associative mapping** für „Anzahl der Bits für den Index-Teil → 0“.
- Jeder Platz des TLB kann die Verwaltungs-Information für jede beliebe Seite enthalten (kein **aliasing**).



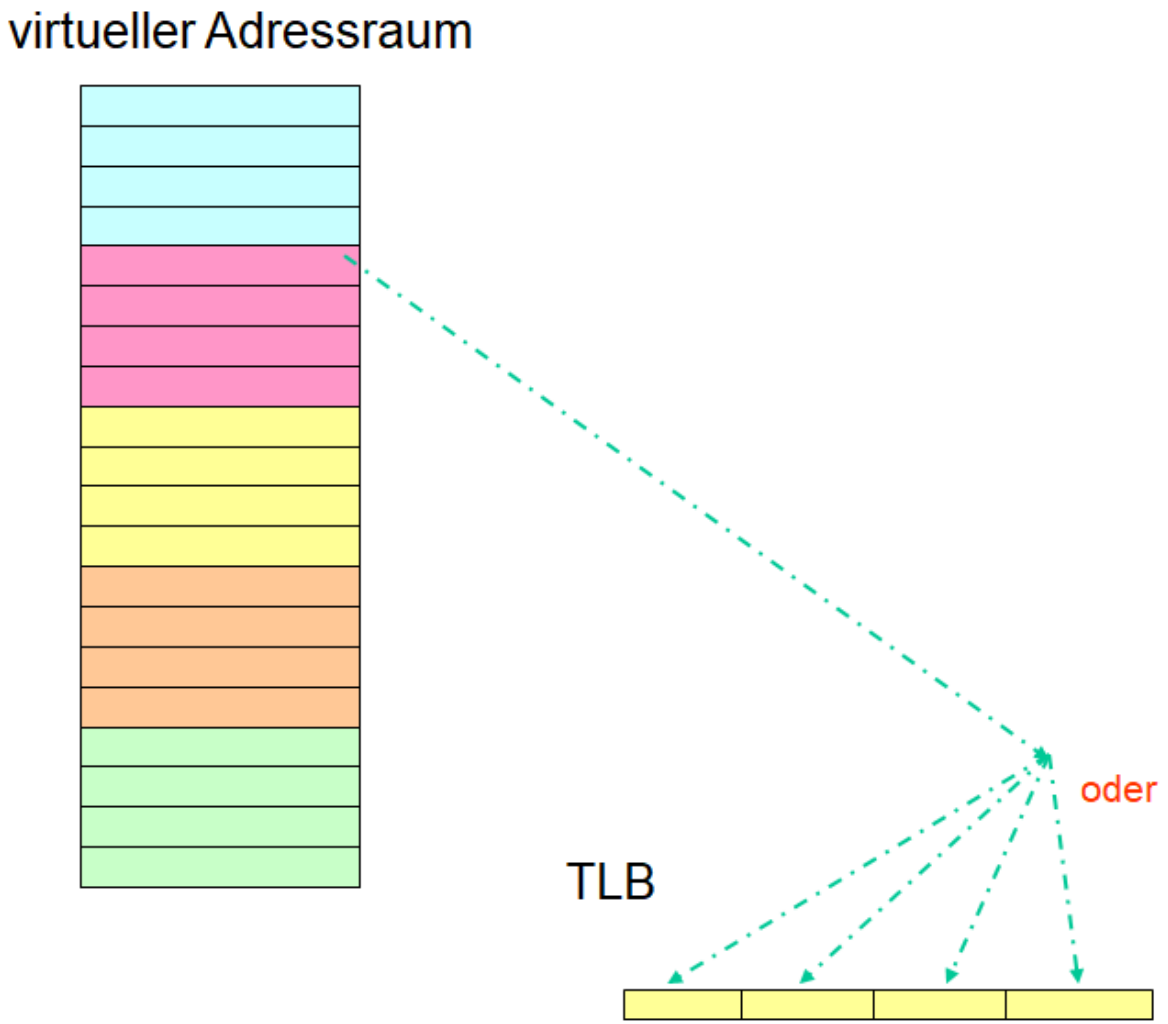
Assoziativspeicher, associative mappingPrinzip

- Der Fall **associative mapping** entsteht aus dem **set associative mapping** für „Anzahl der Bits für den Index-Teil → 0“.
- Jeder Platz des TLB kann die Verwaltungs-Information für jede beliebe Seite enthalten (kein **aliasing**).



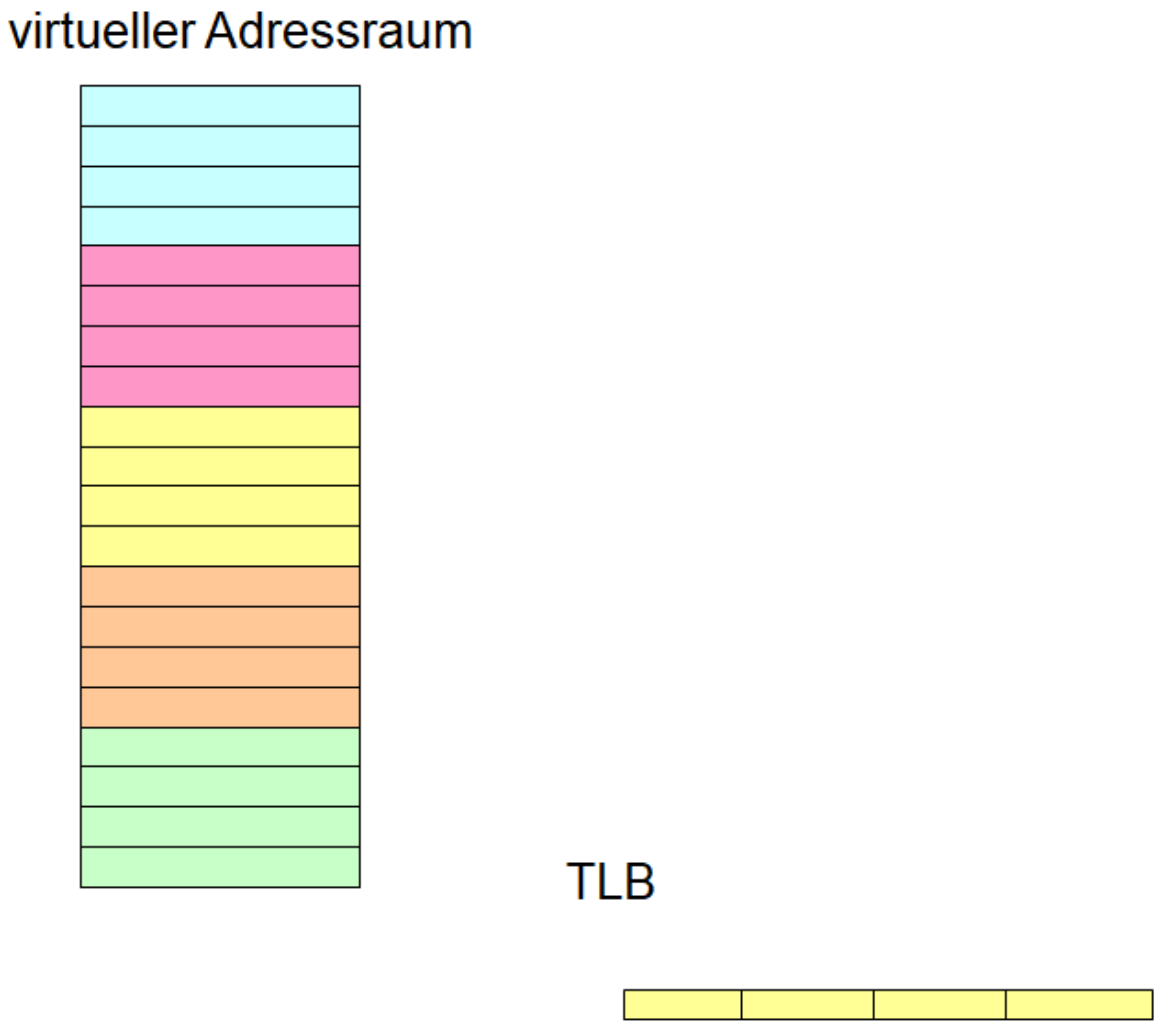
Assoziativspeicher, associative mappingPrinzip

- Der Fall **associative mapping** entsteht aus dem **set associative mapping** für „Anzahl der Bits für den Index-Teil → 0“.
- Jeder Platz des TLB kann die Verwaltungs-Information für jede beliebe Seite enthalten (kein **aliasing**).



Assoziativspeicher, associative mappingPrinzip

- Der Fall **associative mapping** entsteht aus dem **set associative mapping** für „Anzahl der Bits für den Index-Teil → 0“.
- Jeder Platz des TLB kann die Verwaltungs-Information für jede beliebe Seite enthalten (kein **aliasing**).



Zusammenfassung

Translation look aside buffer (TLBs) dienen dem raschen Zugriff auf häufig benutzte Adressumrechnungs-Informationen.

Fehlende Treffer → Umrechnung über den Hauptspeicher.

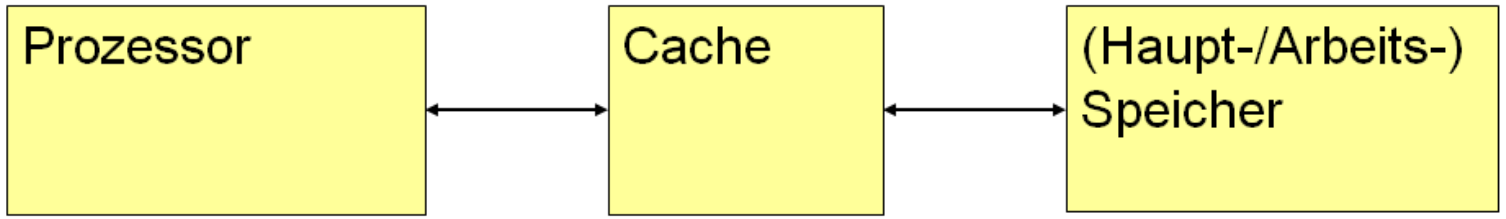
3 Organisationsformen:

1. **Direct mapping**
2. **Mengen-assoziative Abbildung**
   (set associative mapping), und
3. **Assoziativspeicher (associative mapping)**.

TLBs können Teil der Memory Management Unit (MMU) sein.

2.4.4 Caches

- **Cache** = Speicher, der vor einen größeren, langsamen Speicher geschaltet wird.
- Im weiteren Sinn: Puffer zur Aufnahme häufig benötigter Daten.
- Im engeren Sinn: Puffer zwischen Hauptspeicher und Prozessor.

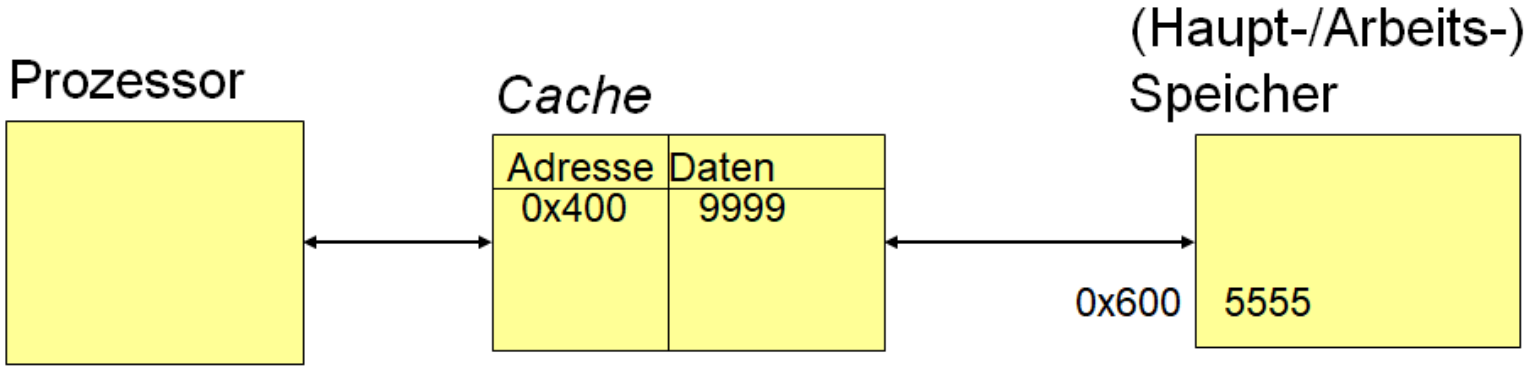


Ursprung: cacher (frz.): verstecken („versteckter Speicher“).

Ablauf

Organisation von Caches (im engeren Sinn):

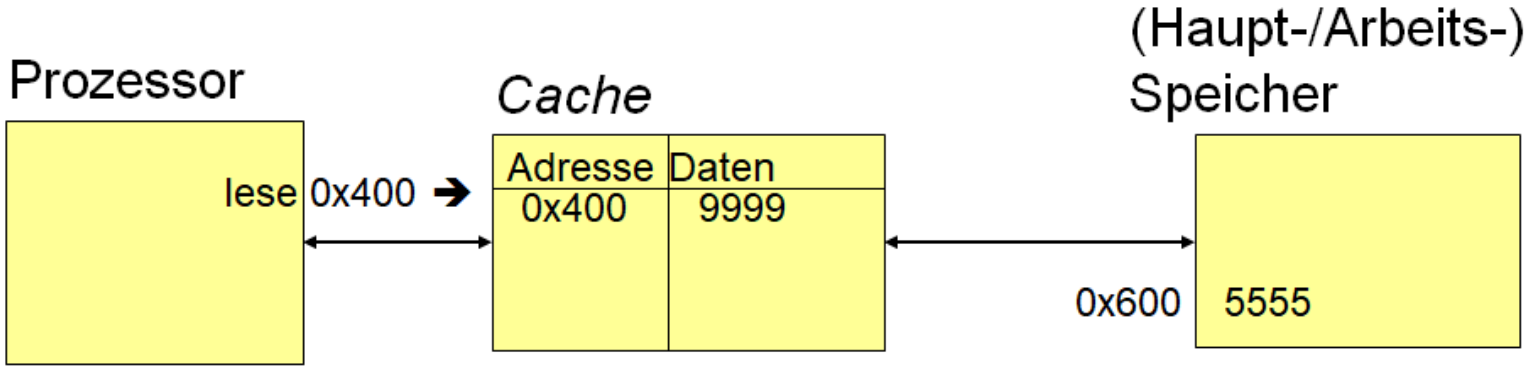
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



Ablauf

Organisation von Caches (im engeren Sinn):

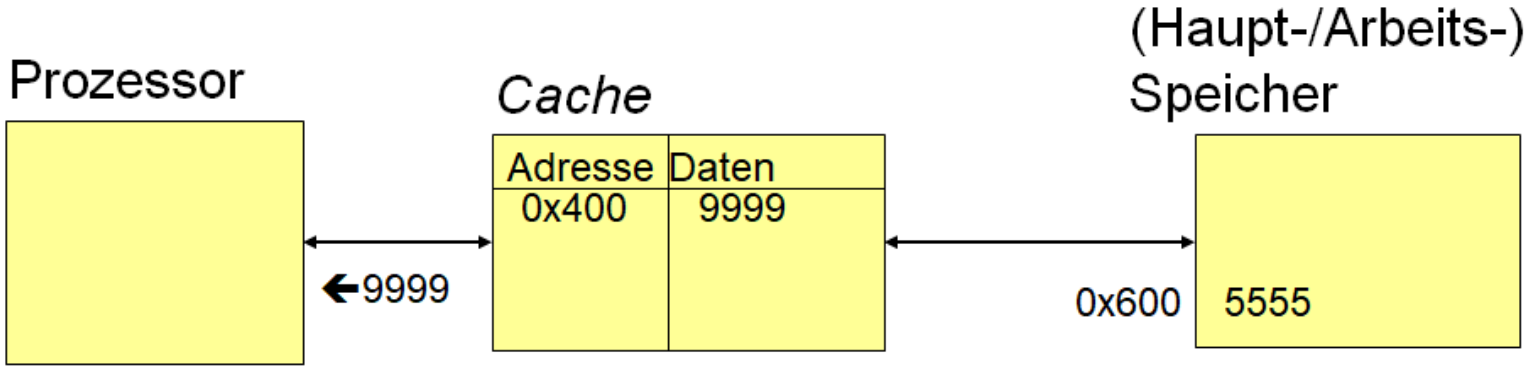
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



Ablauf

Organisation von Caches (im engeren Sinn):

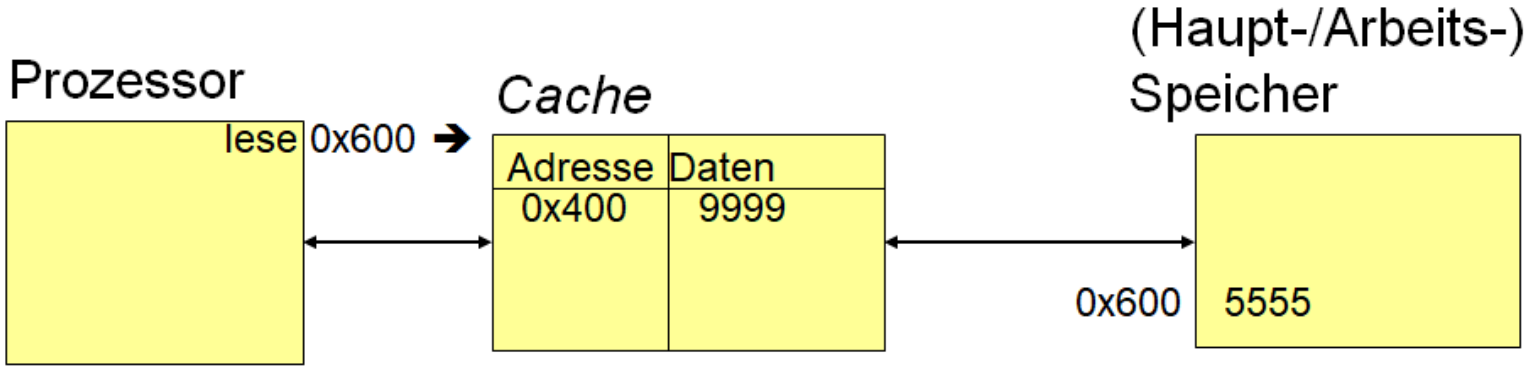
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



Ablauf

Organisation von Caches (im engeren Sinn):

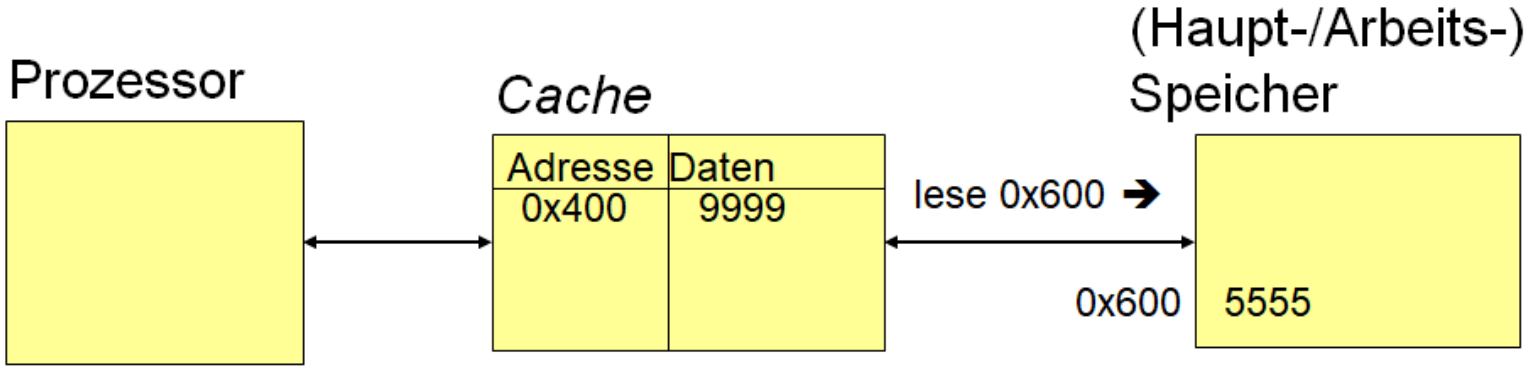
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



Ablauf

Organisation von Caches (im engeren Sinn):

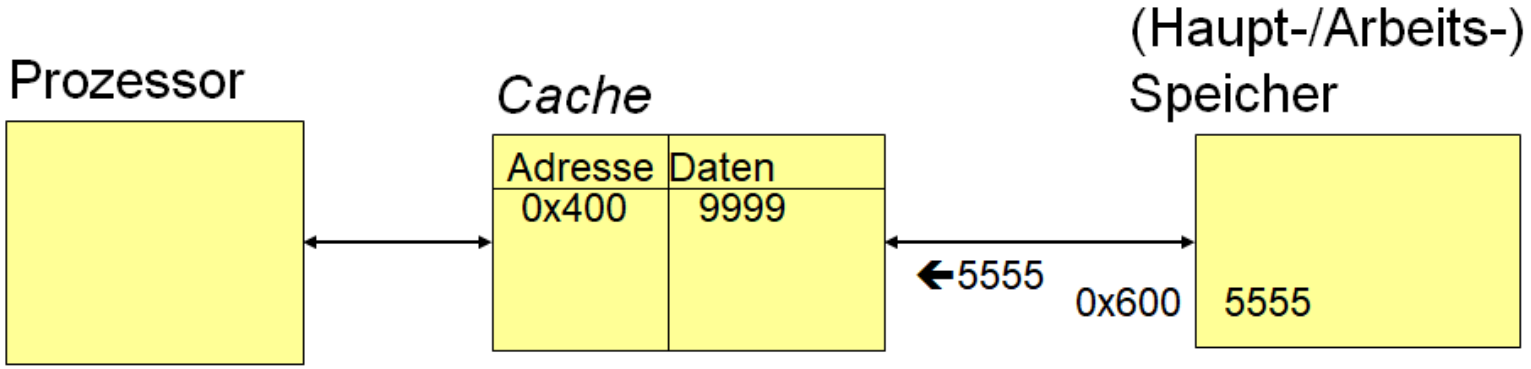
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



Ablauf

Organisation von Caches (im engeren Sinn):

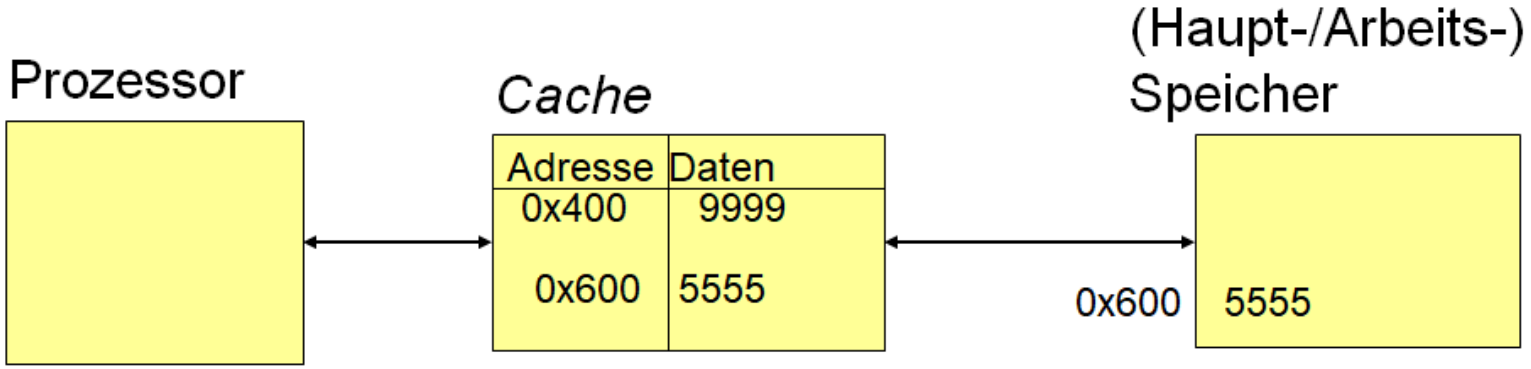
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



Ablauf

Organisation von Caches (im engeren Sinn):

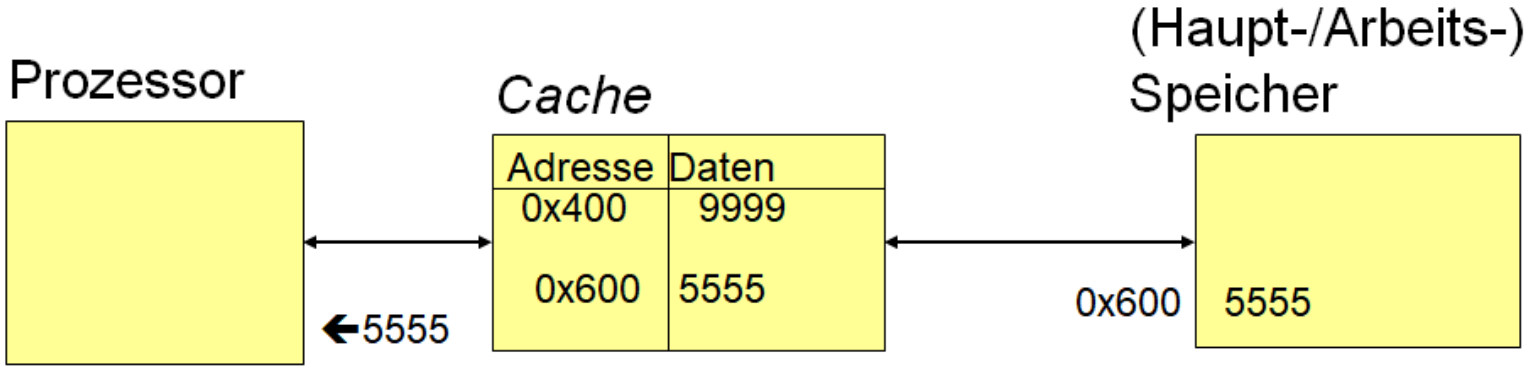
- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.



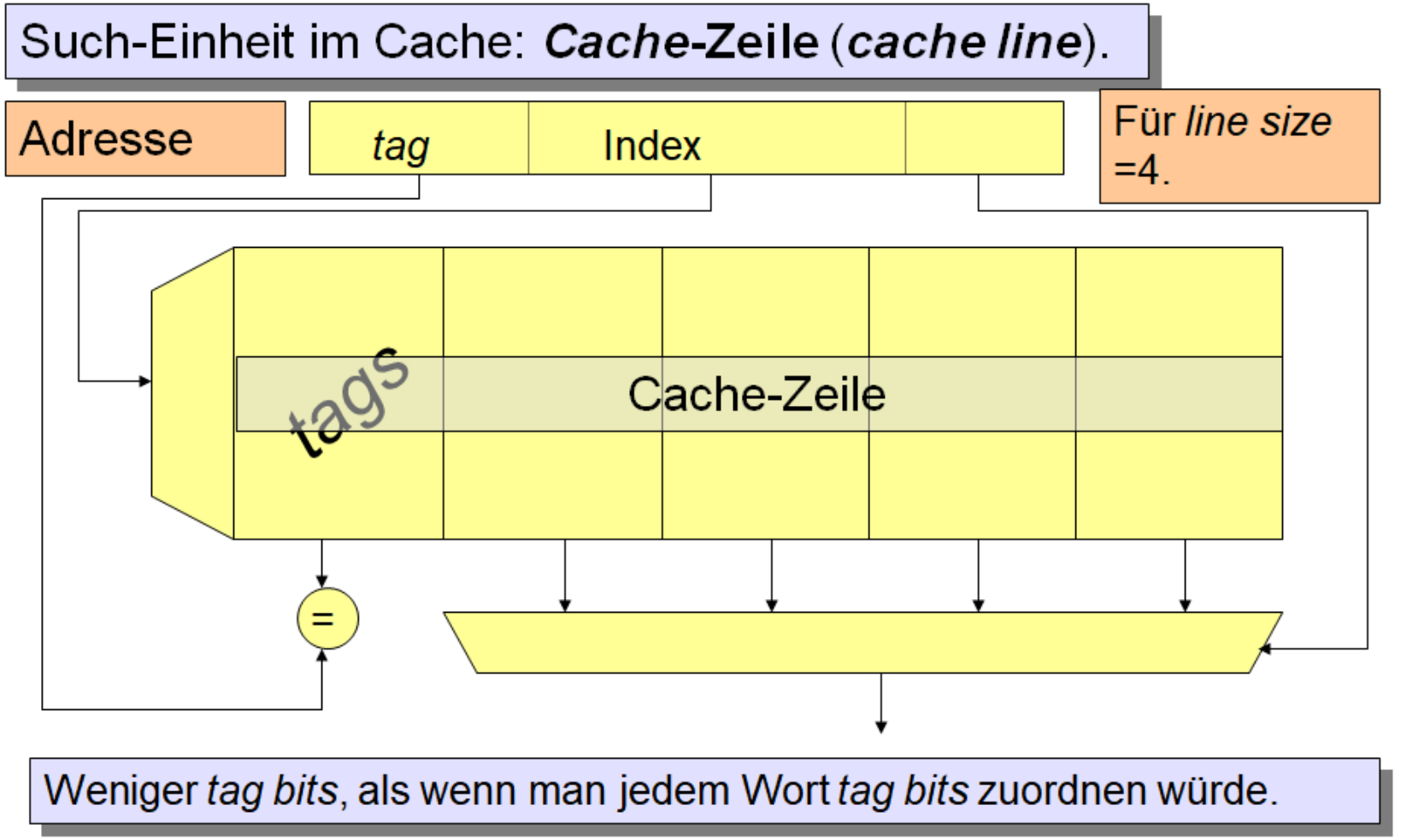
Ablauf

Organisation von Caches (im engeren Sinn):

- Prüfung anhand der (virtuellen oder realen) Adresse, ob benötigte Daten im Cache vorhanden sind („Treffer“; cache hit).
- Falls nicht (cache miss): Zugriff auf den (Haupt-) Speicher, Eintrag in den Cache.

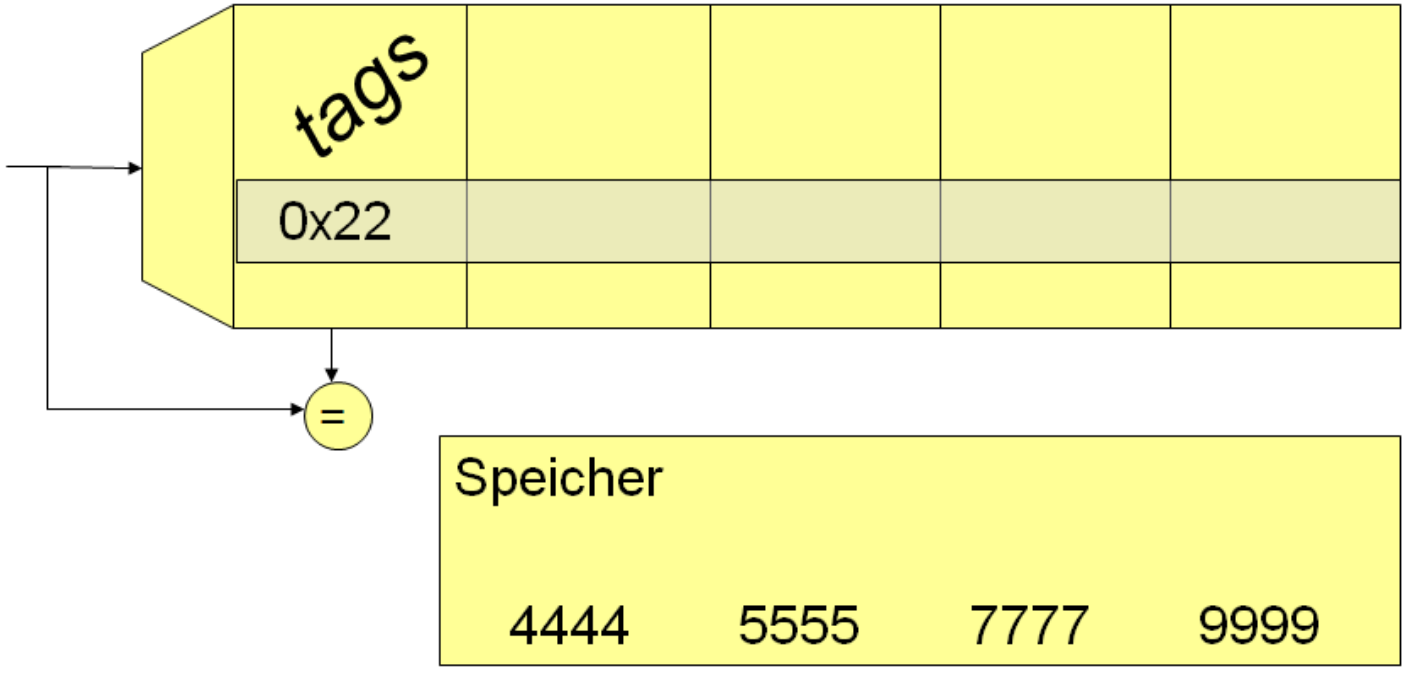


Prüfung auf cache hit: Cache-Zeilen (cache lines)



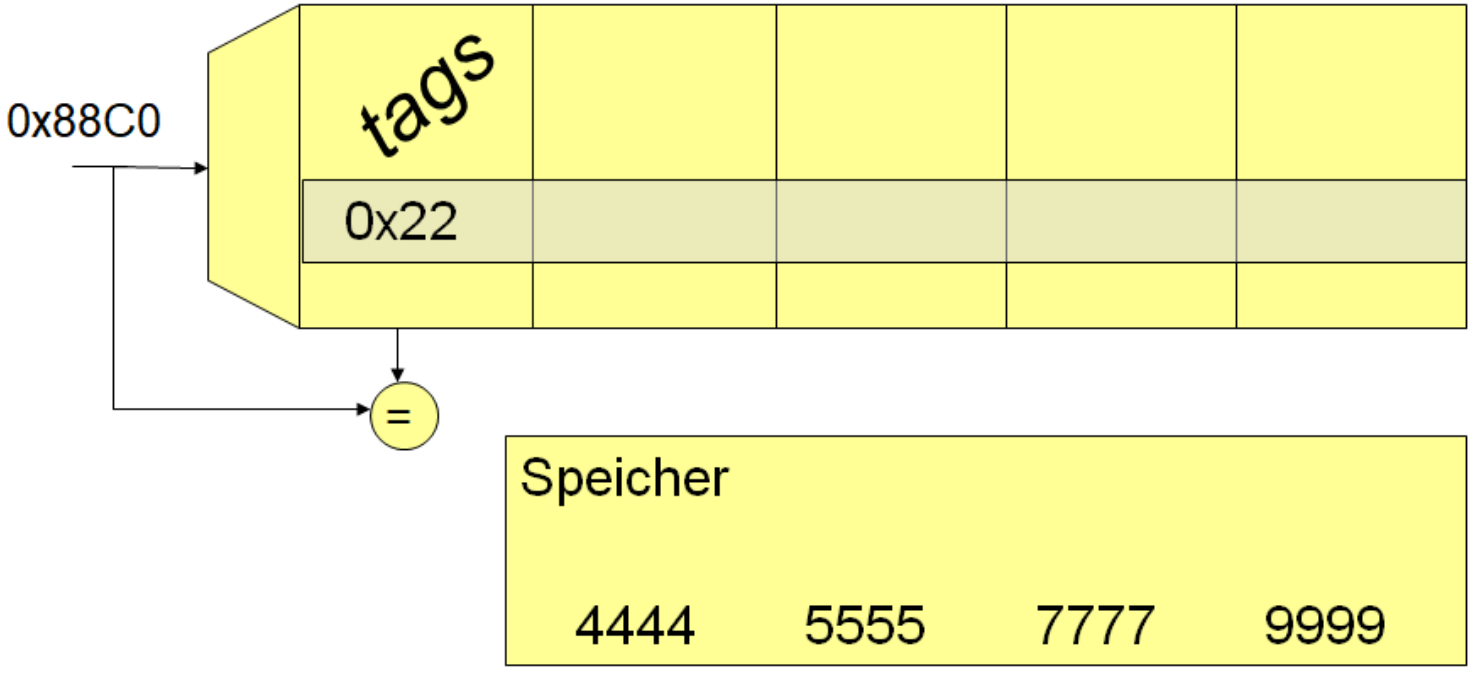
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



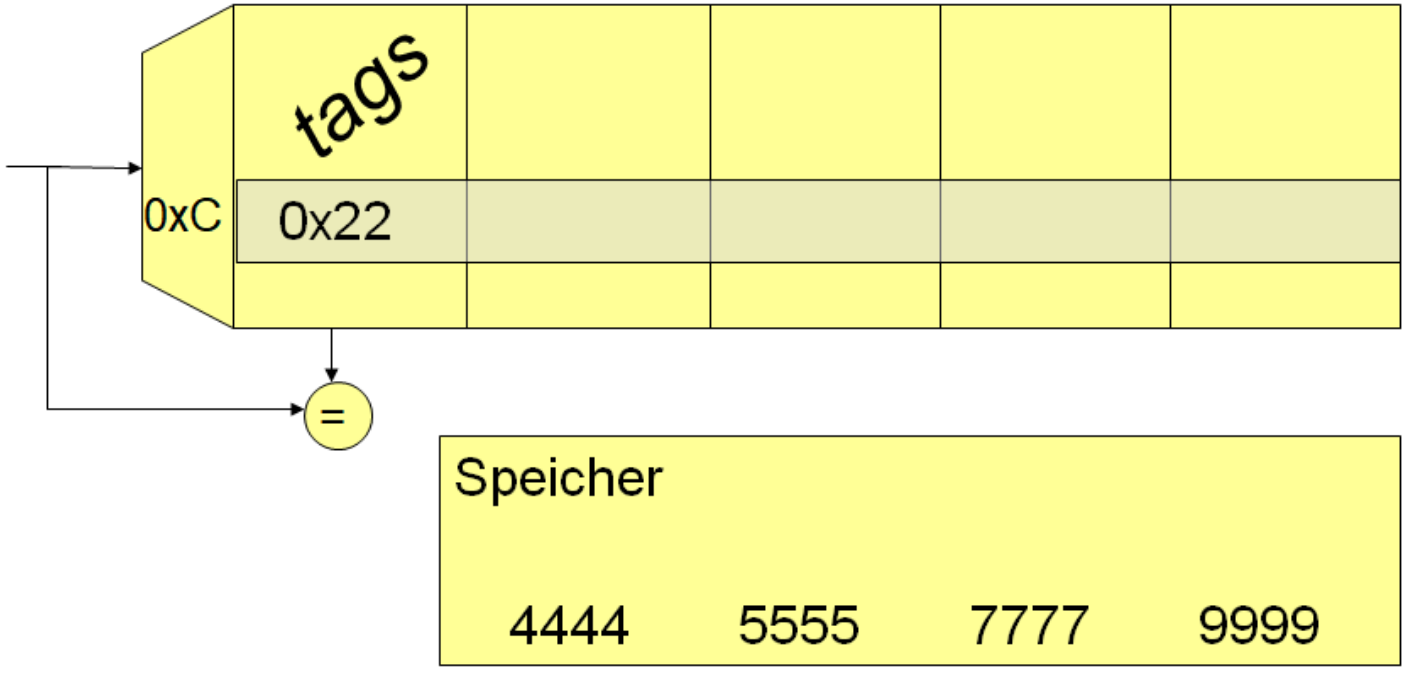
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



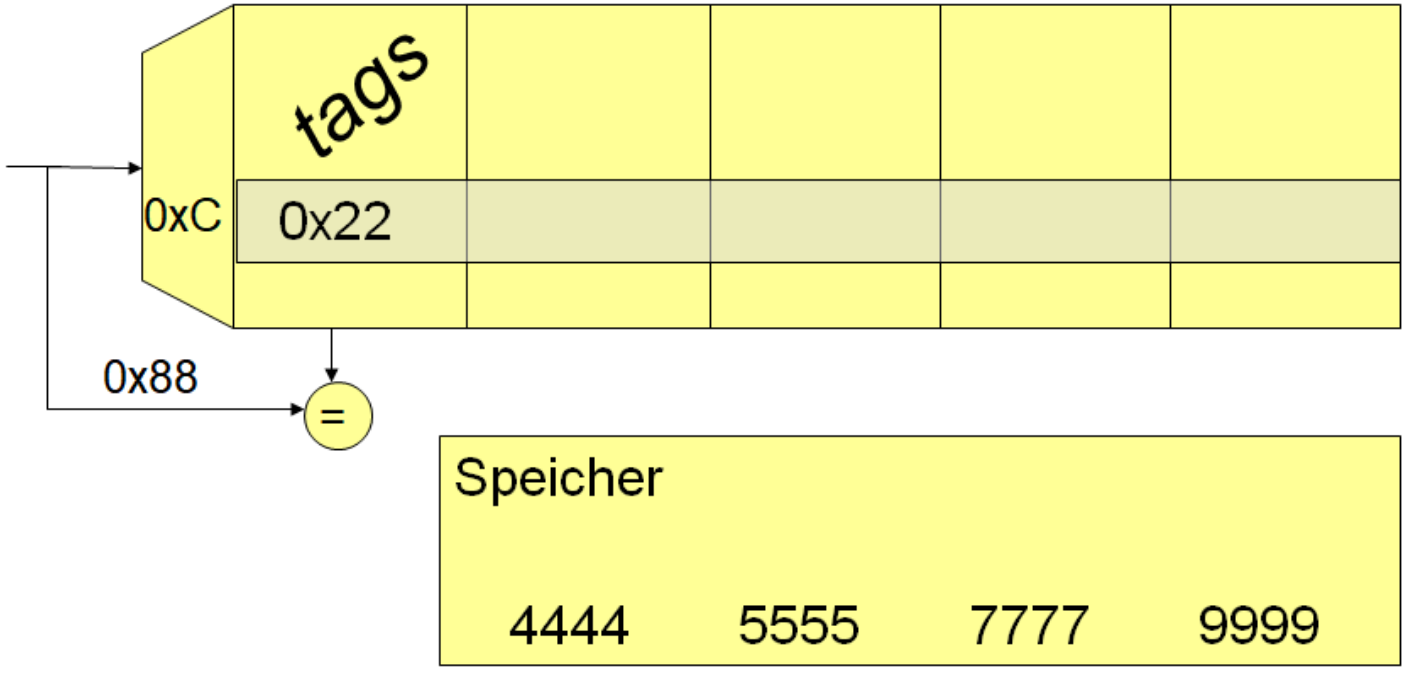
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



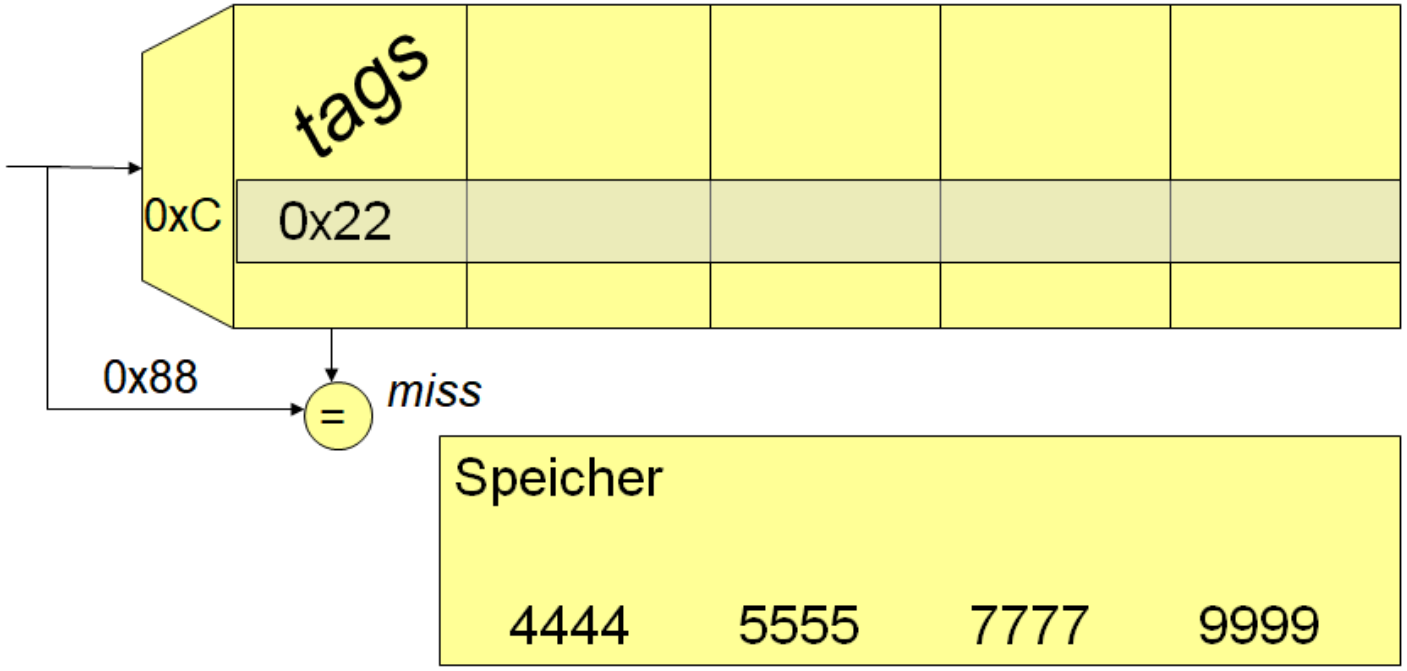
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



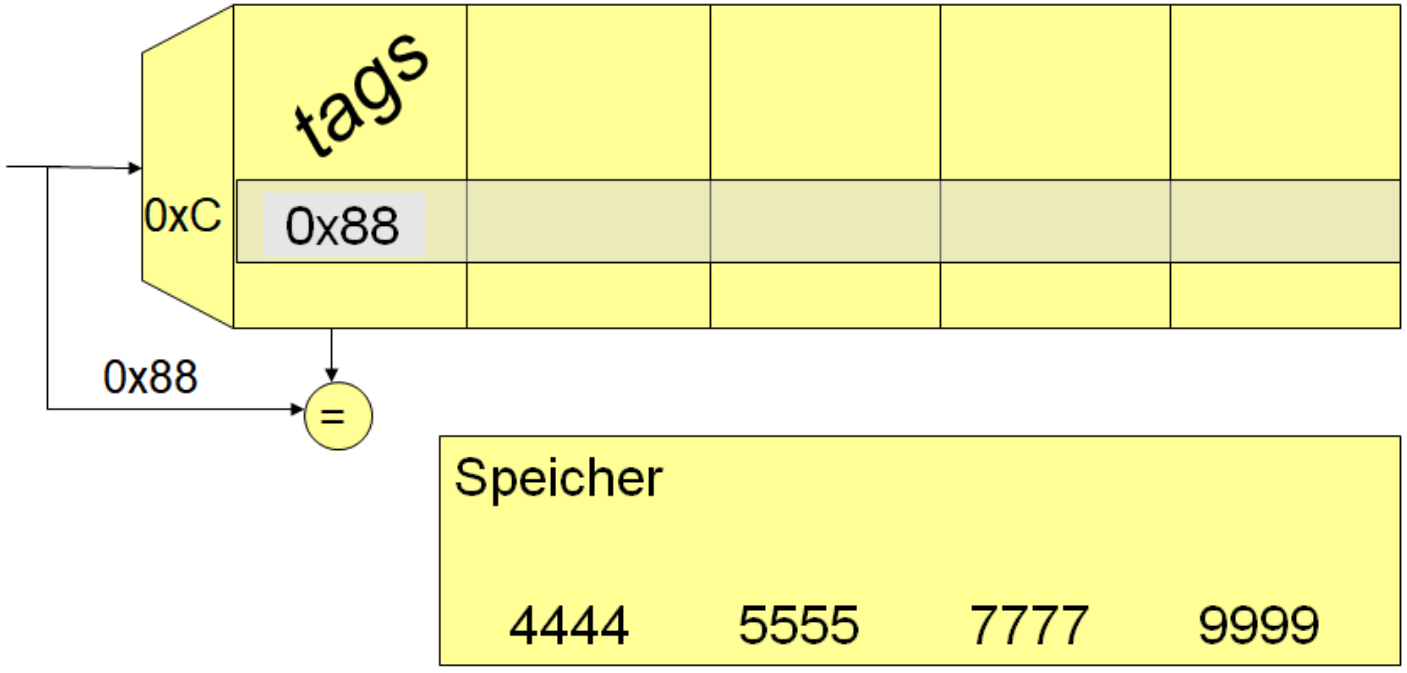
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



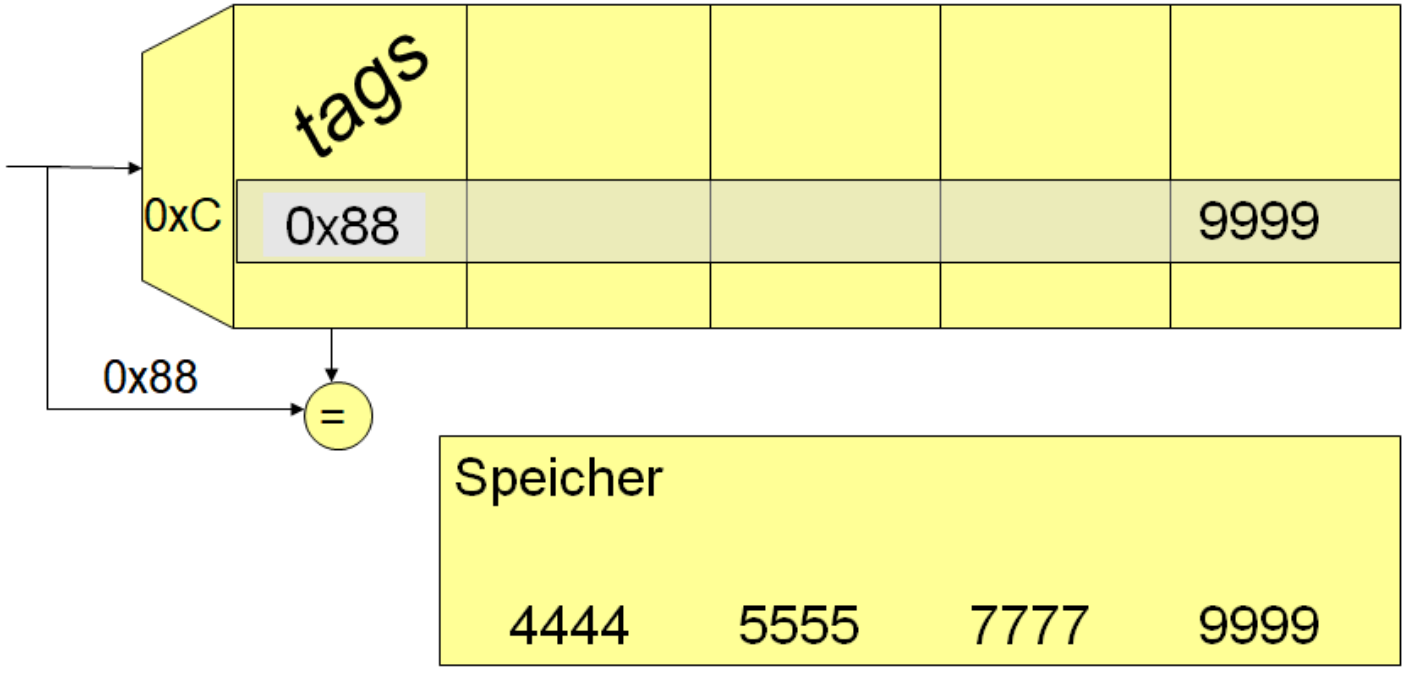
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



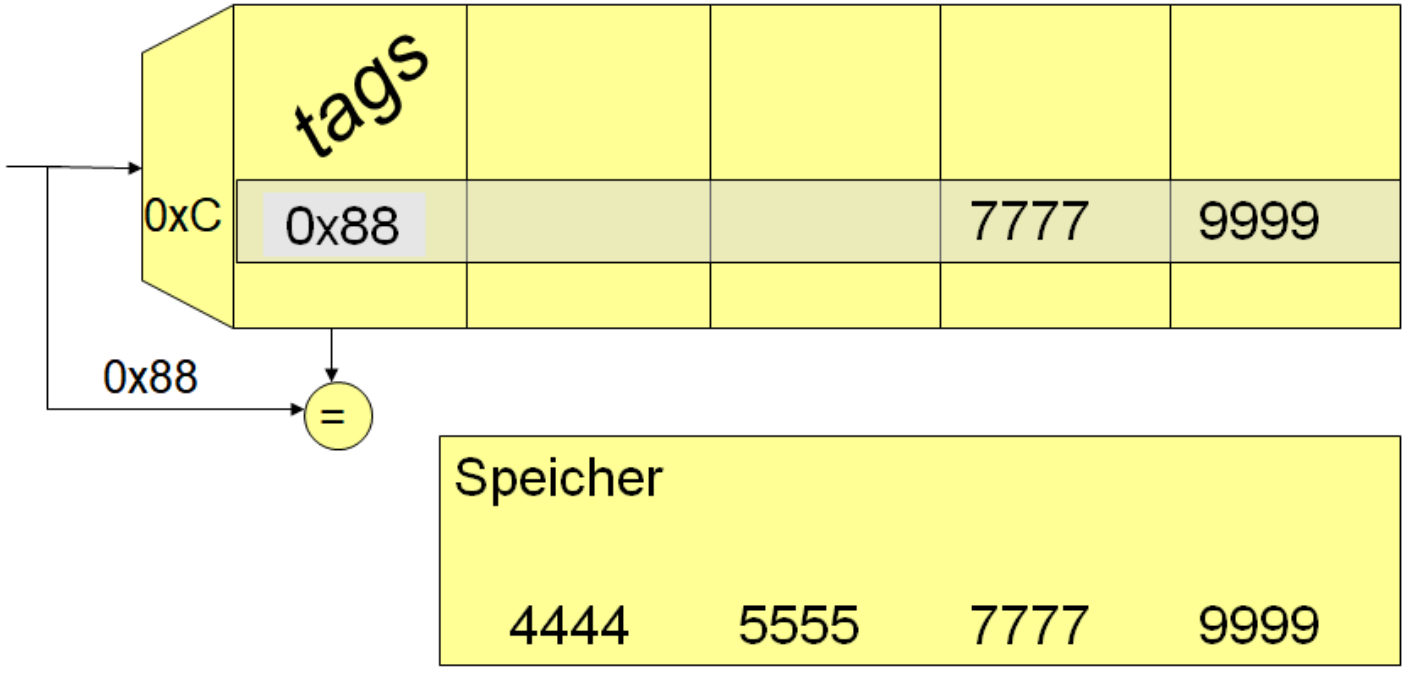
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



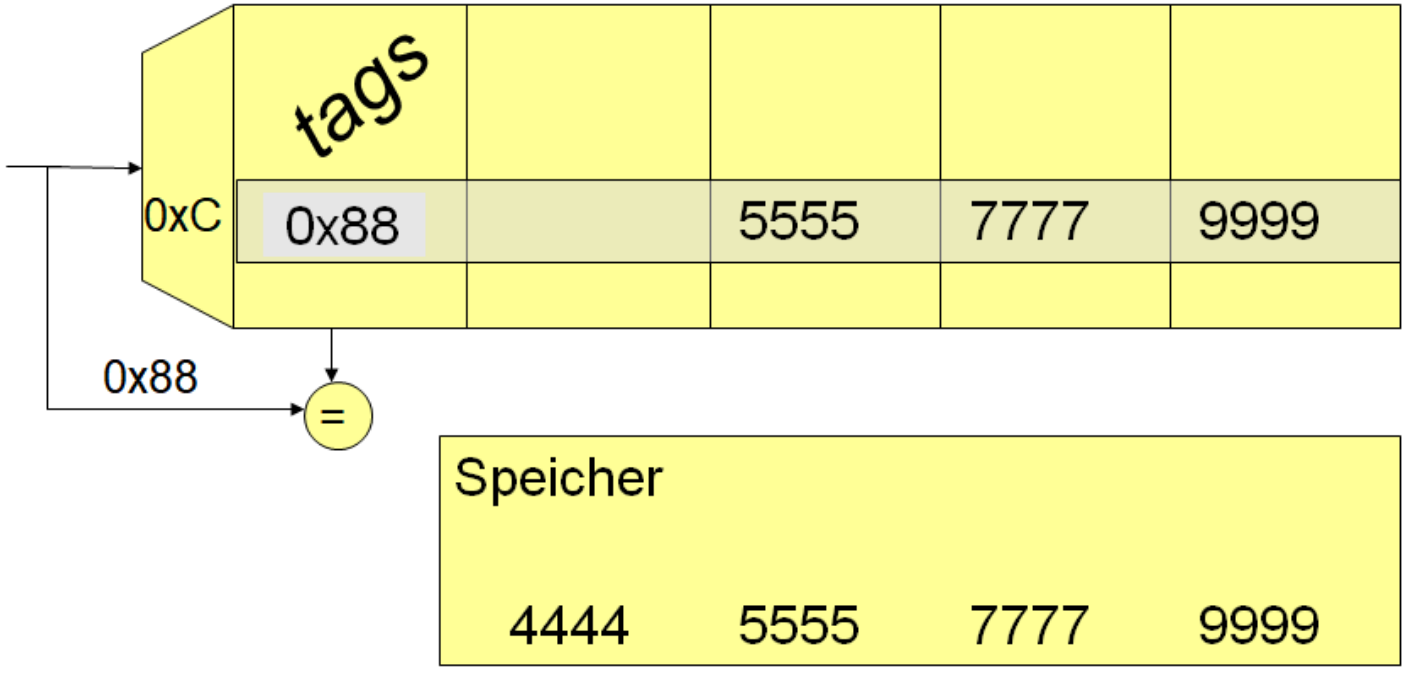
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



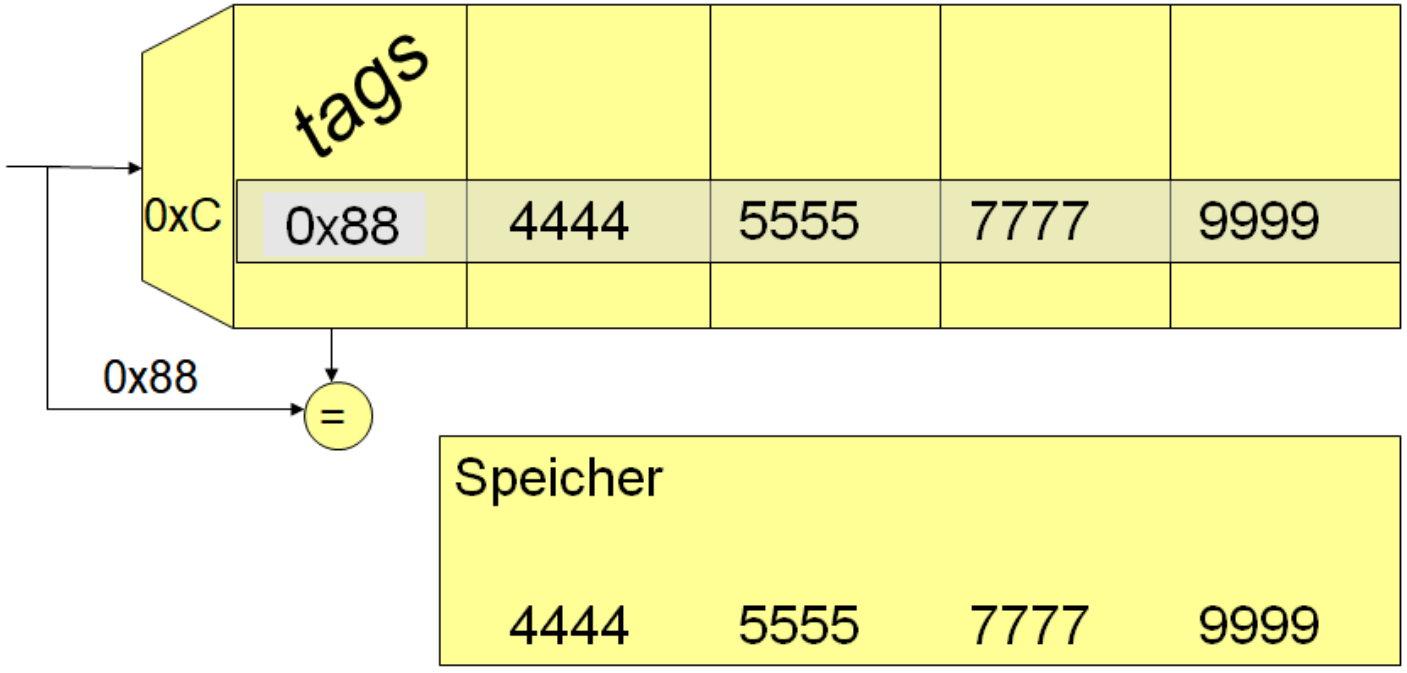
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



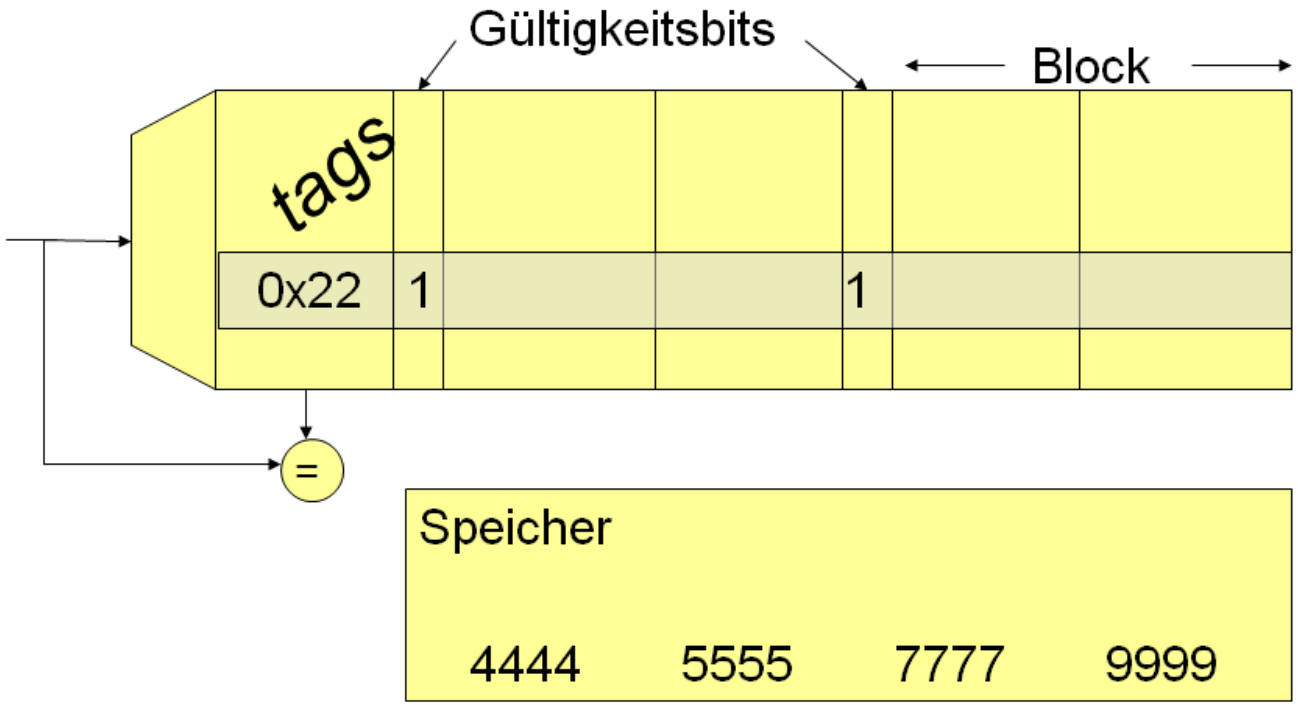
Cache-Blöcke (cache blocks) (1)

- Die **Blockgröße** ist die Anzahl der Worte, die im Fall eines *cache misses* aus dem Speicher nachgeladen werden.
- Beispiel: (Blockgröße=*line size*)
- Bezeichnung Blöcke & Zeilen in Literatur uneinheitlich!



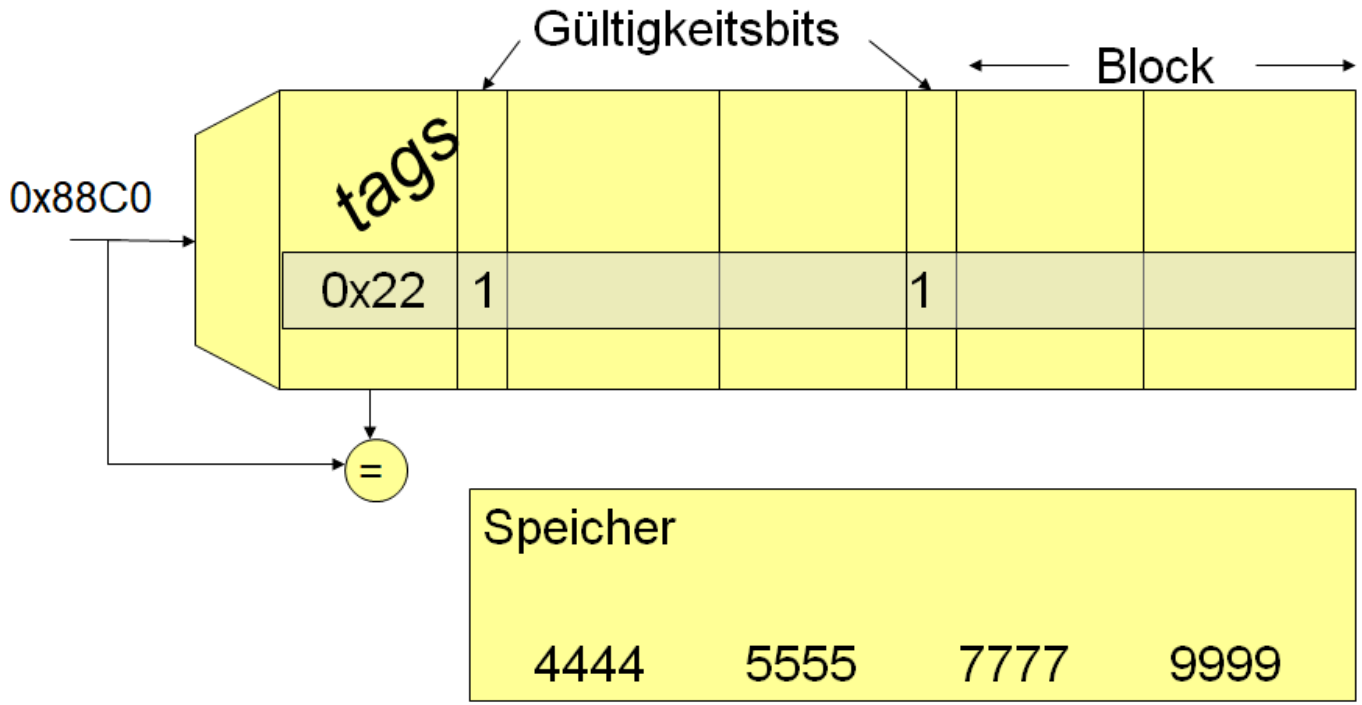
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



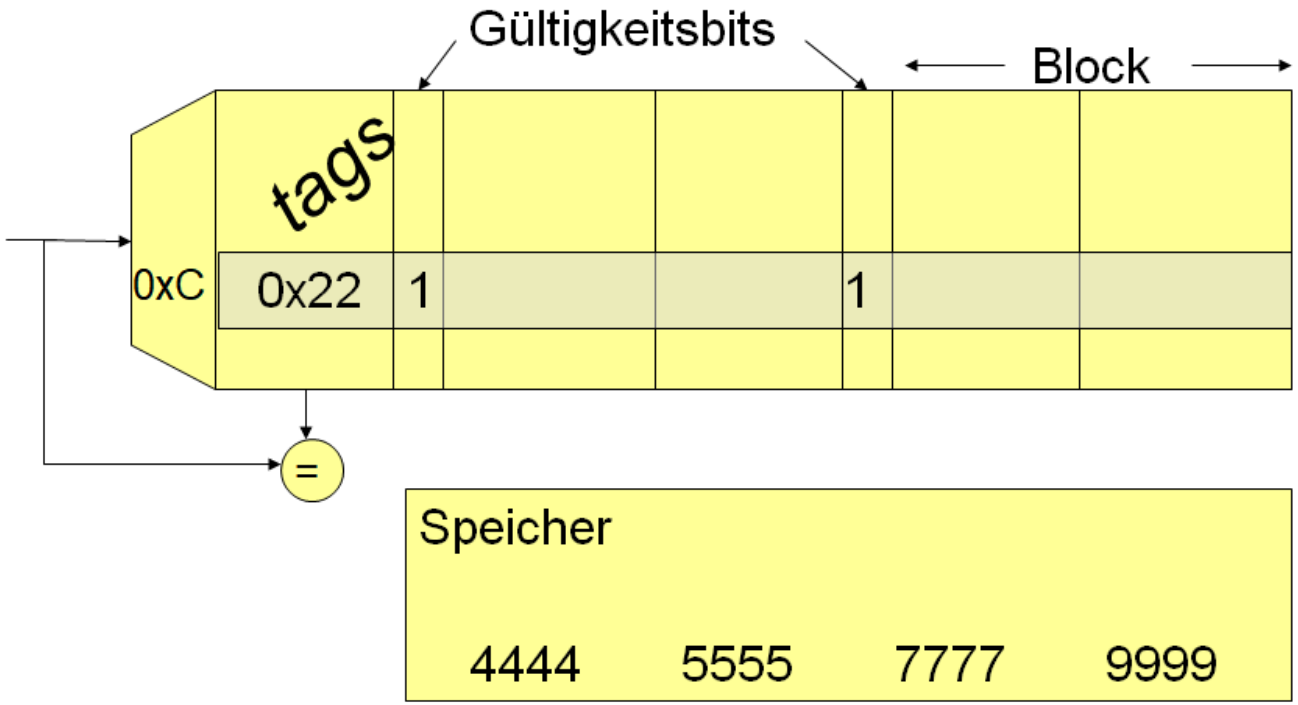
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



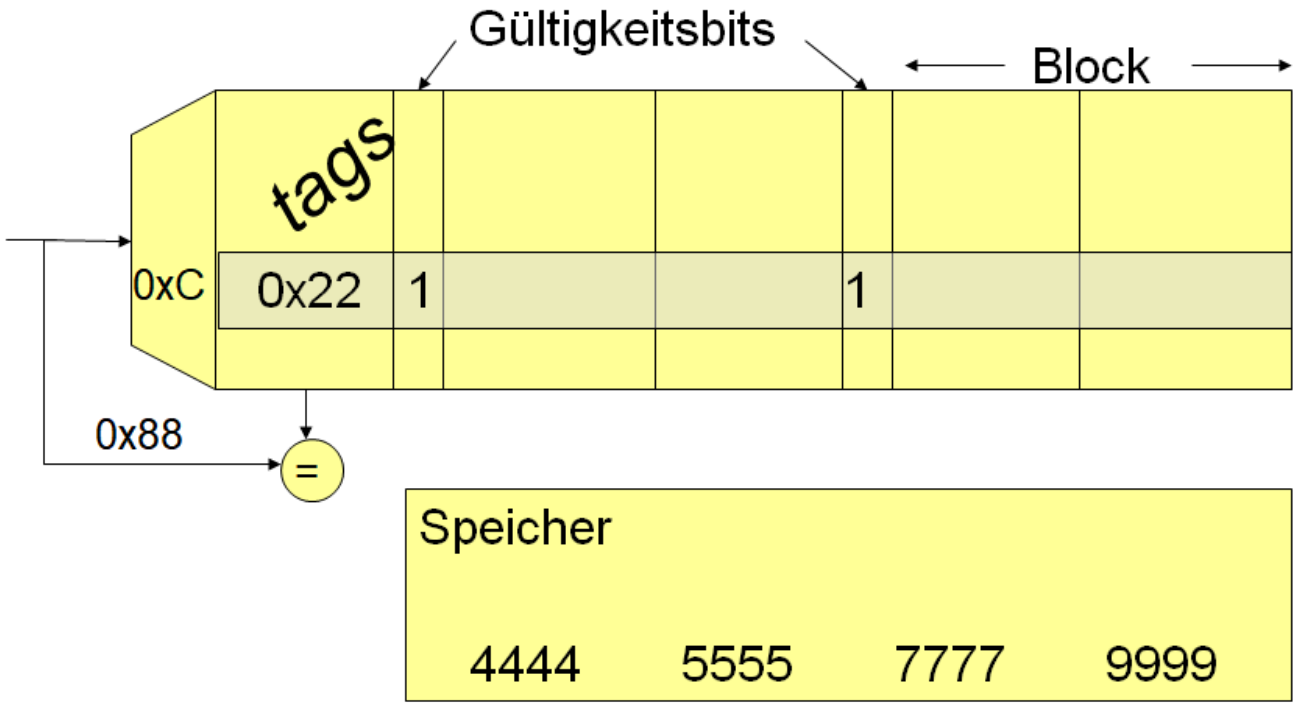
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



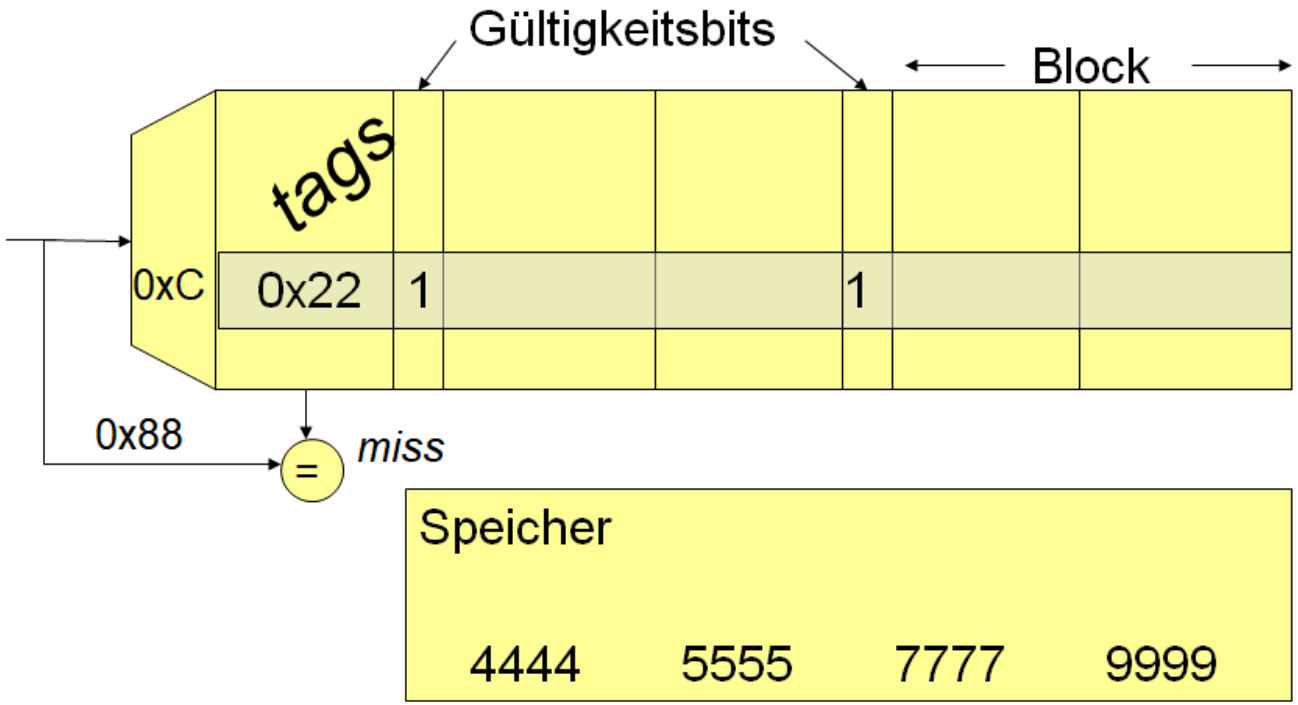
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



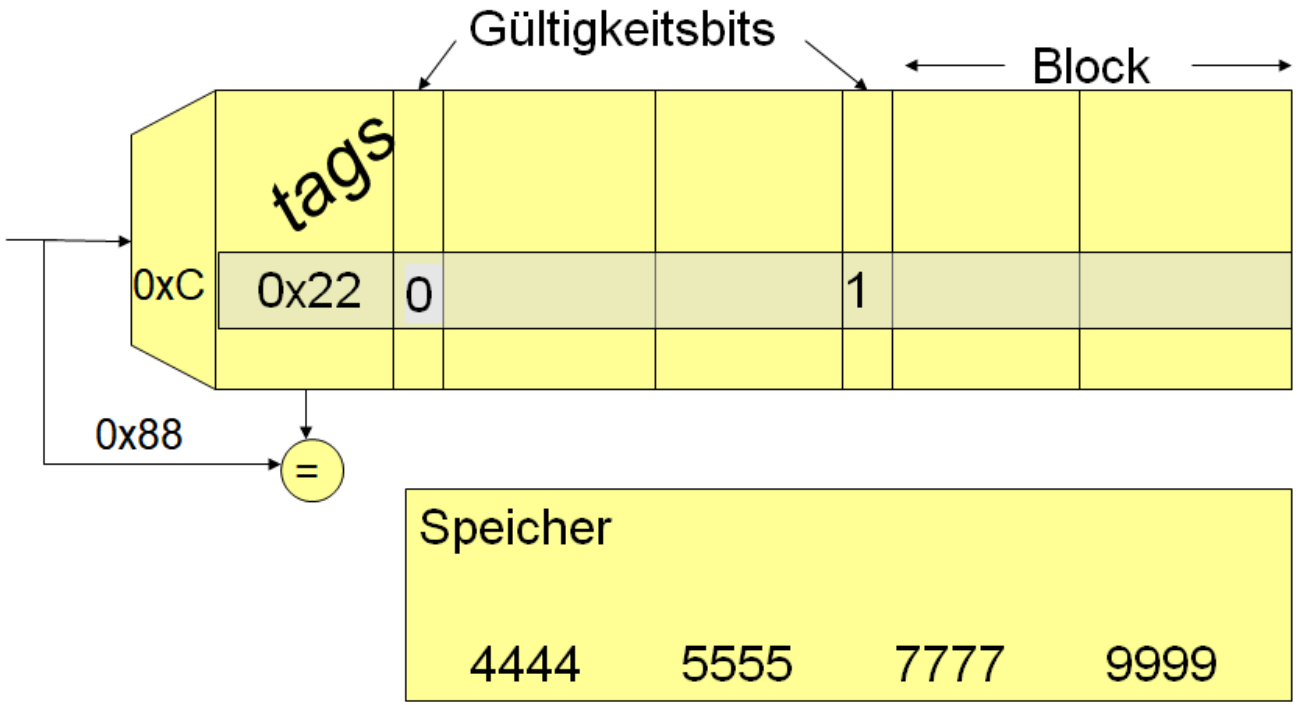
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



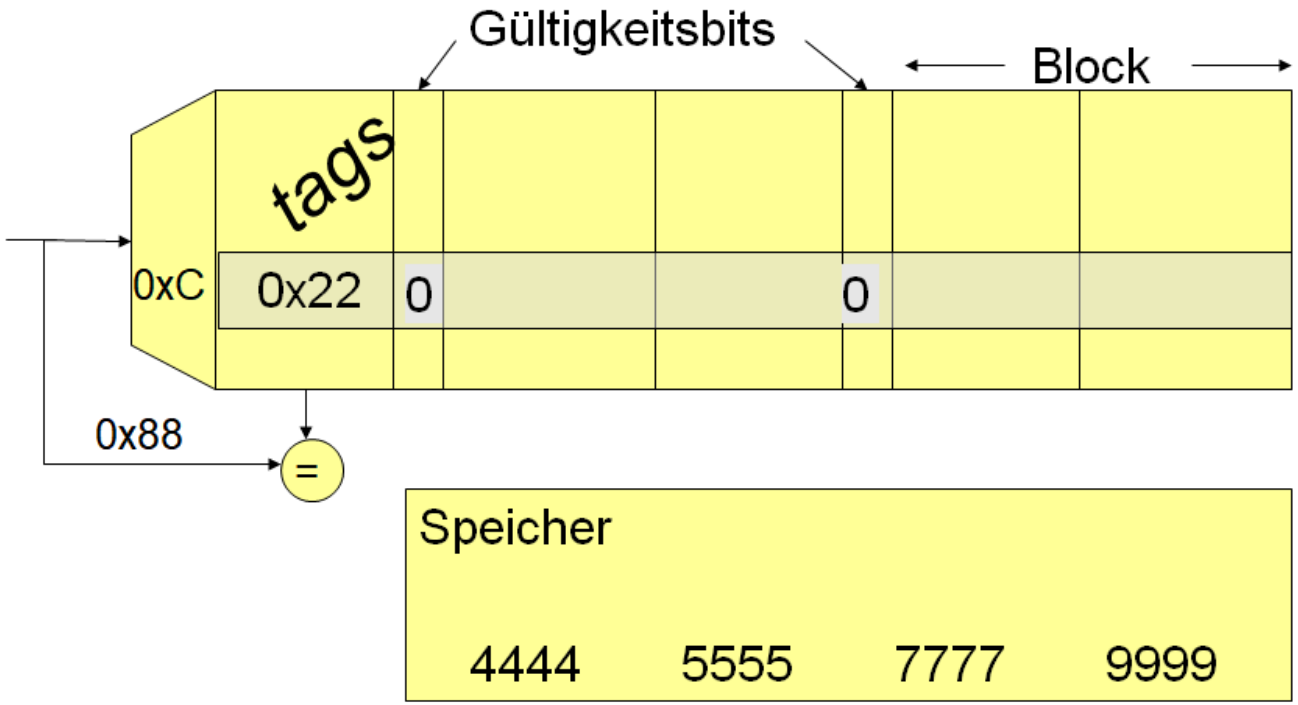
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



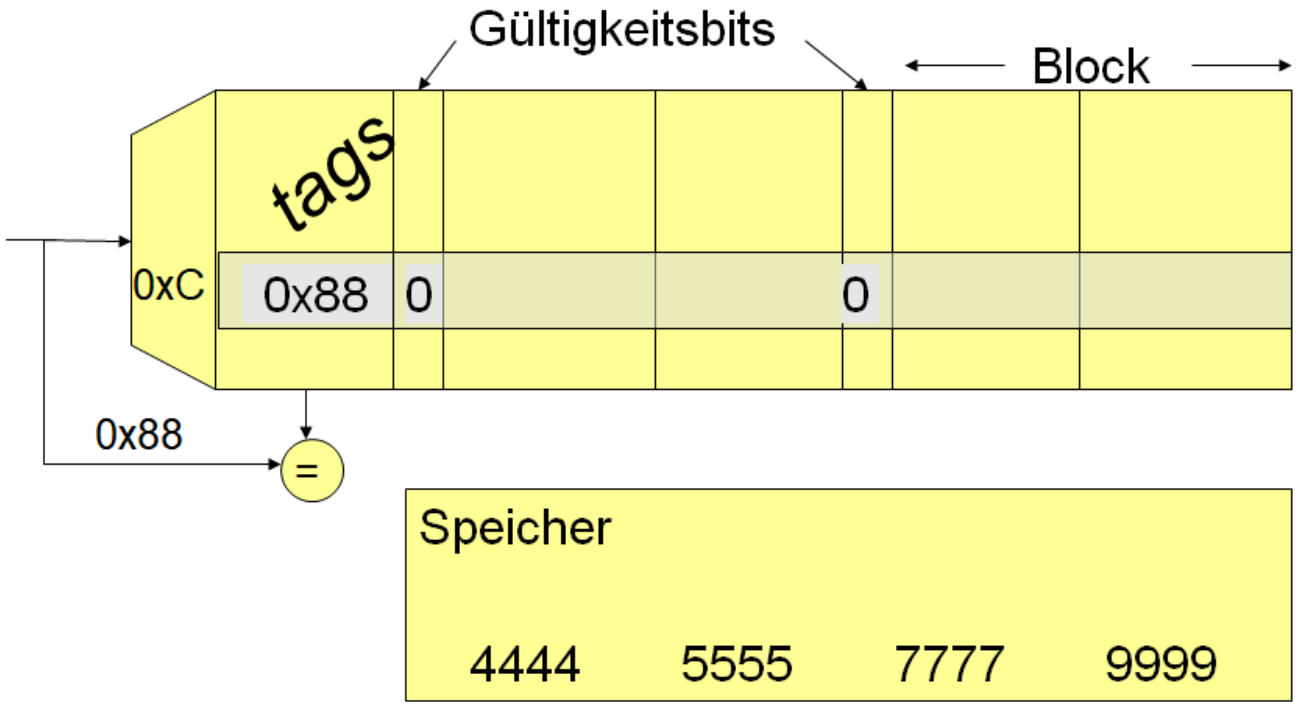
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



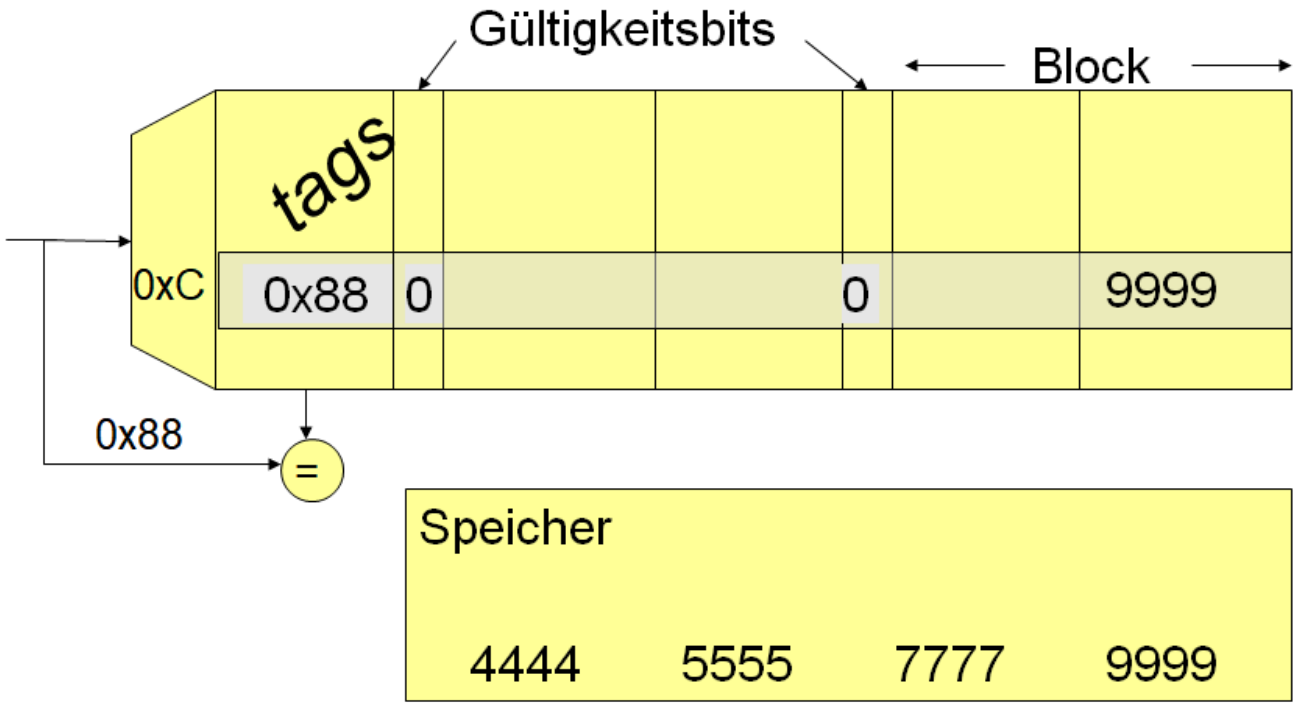
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



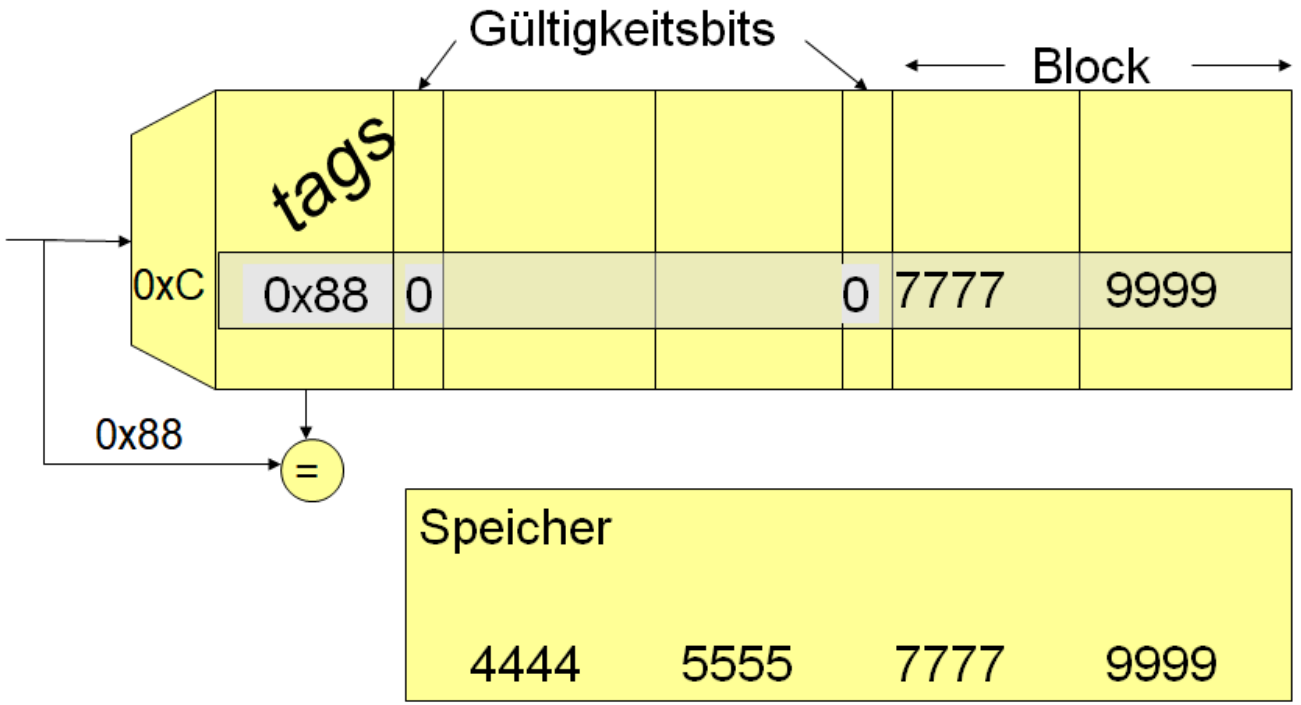
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



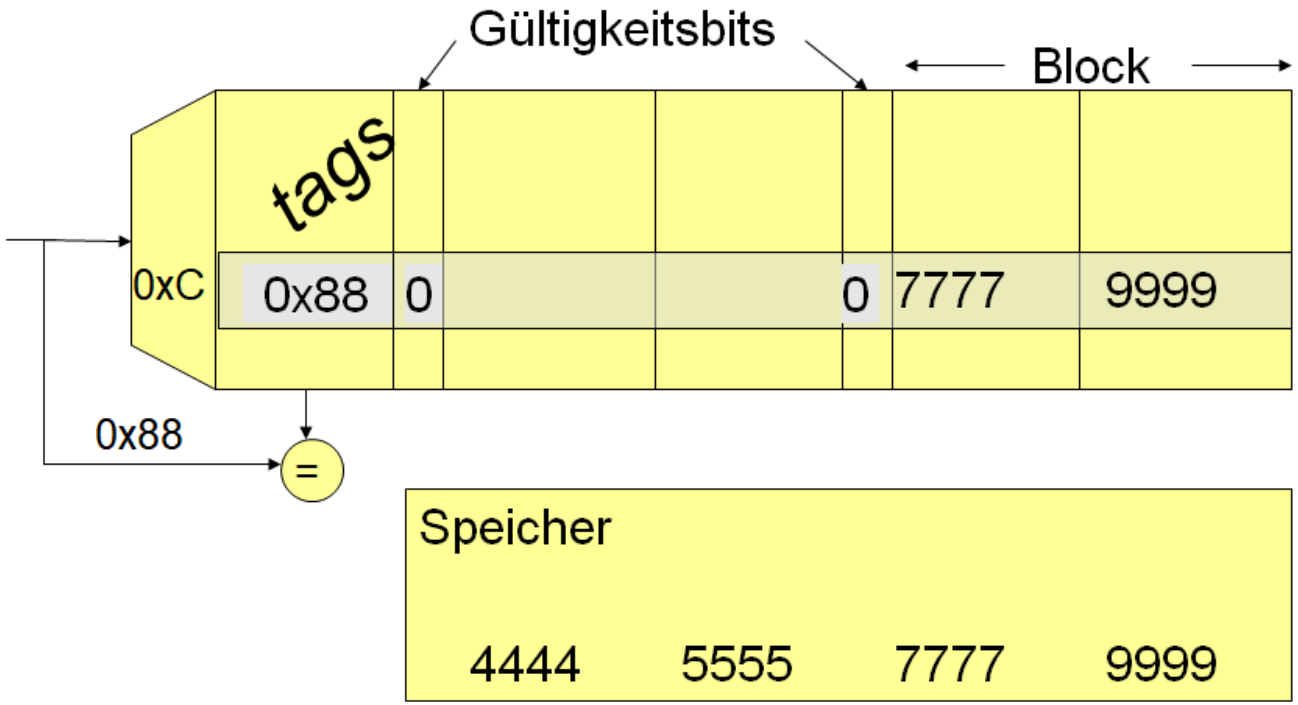
Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2



Cache-Blöcke (cache blocks) (2)

- Wenn block size < line size, dann sind zusätzliche Gültigkeitsbits erforderlich
- Beispiel: Blockgröße = line size / 2

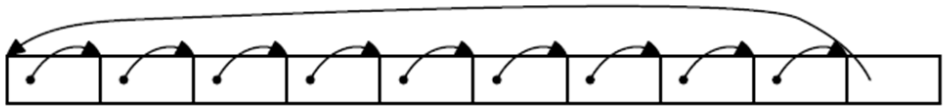


Organisationsformen von Caches

- **Direct mapping**
  Für caching von Befehlen besonders sinnvoll, weil bei Befehlen aliasing sehr unwahrscheinlich ist.
- **Set associative mapping** Sehr häufige Organisationsform, mit Set-Größe=2 oder 4, selten 8.
- **Associative mapping** Wegen der Größe eines Caches kommt diese Organisationsform kaum in Frage (Grund, mit TLBs zu starten).

Warum sollte man sich mit TLBs und Caches beschäftigen?

- Untersuchung der Laufzeit beim Durchlauf durch eine im Array abgelegte verkettete Liste mit Einträgen von (NPAD+1)*8 Byte

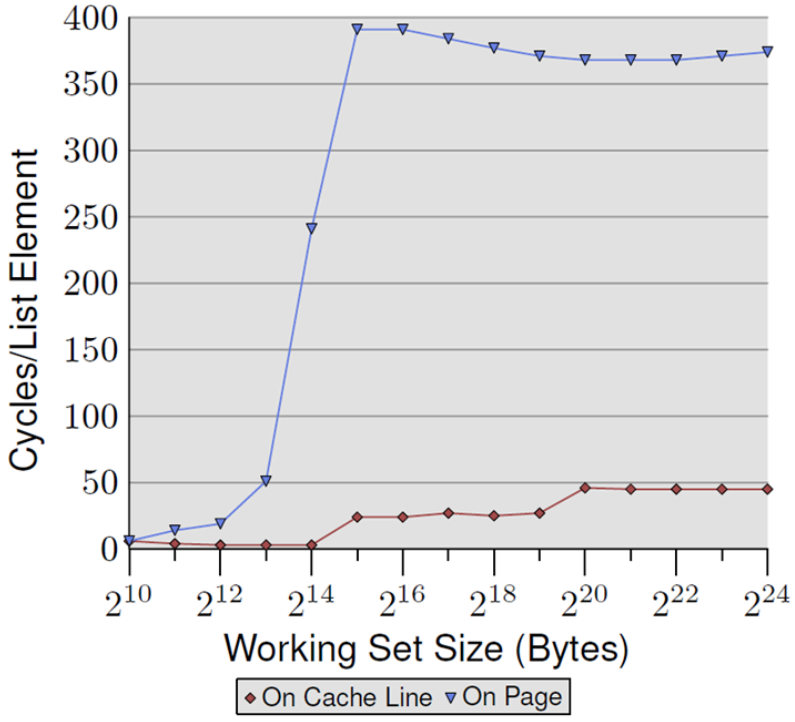


- Pentium P4
- 16 kB L1 Daten-Cache mit 4 Zyklen/Zugriff
- 1 MB L2 Cache mit 14 Zyklen/Zugirff
- Hauptspeicher mit 200 Zyklen/Zugriff

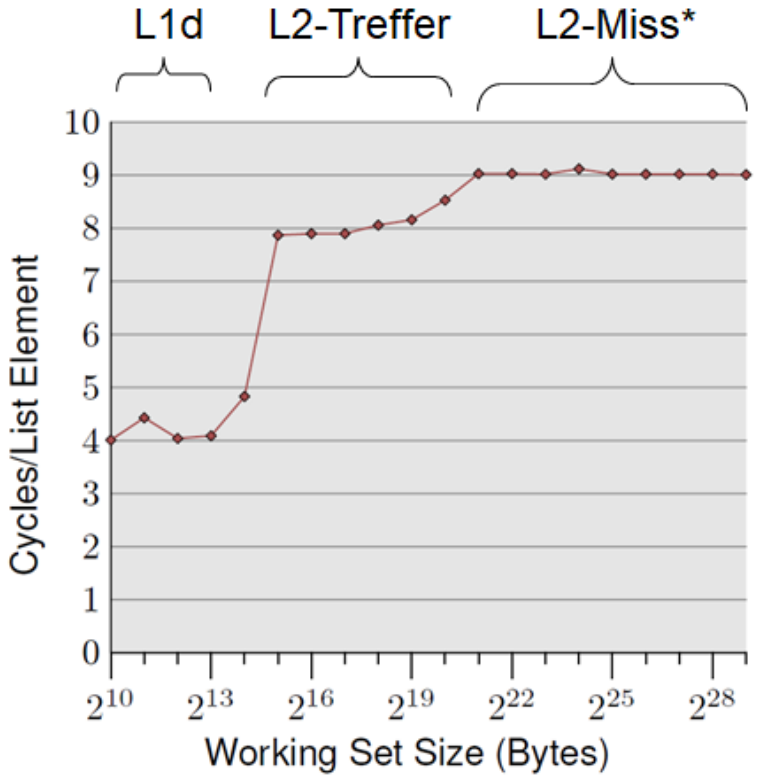
U. Drepper: What every programmer should know about memory, *2007*, *http://www.akkadia. org/drepper/cpumemory.pdf*; *Dank an Prof. Teubner (LS6) für Hinweis auf diese Quelle* In Anlehnung an das Papier „David Goldberg, What every programmer should know about floating point arithmetic, ACM Computing Surveys, 1991 (auch für diesen Kurs benutzt).

Großer Einfluss von TLBs und Caches auf die Laufzeit

TLB: Elemente auf ≠ Seiten; Laufzeit ↑, wenn TLB-Größe überschritten ist



Cache: NPAD=0, sequent. Zugriff

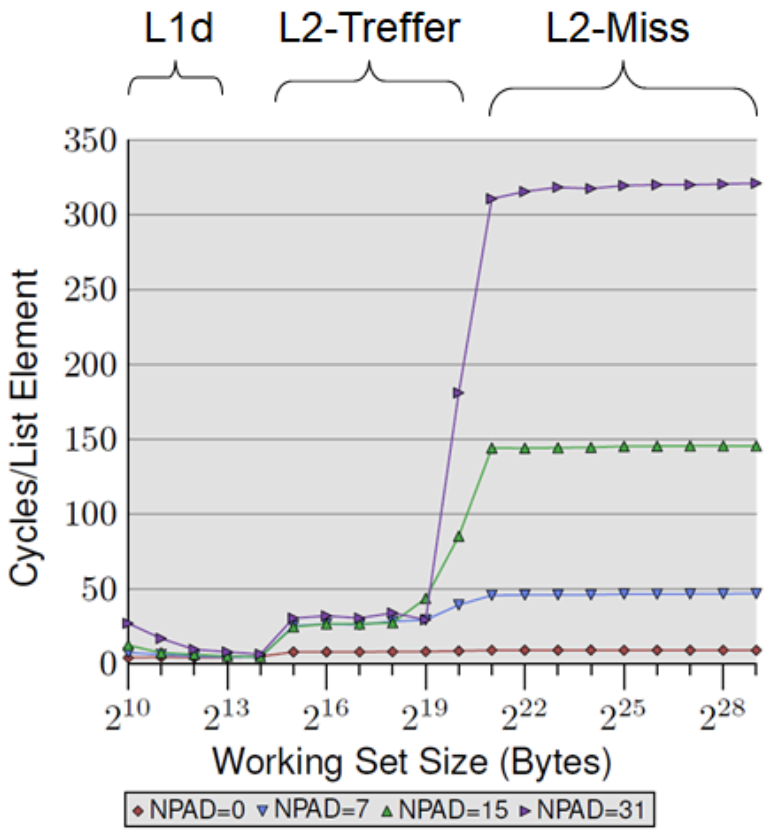


*Funktionierendes Prefetching

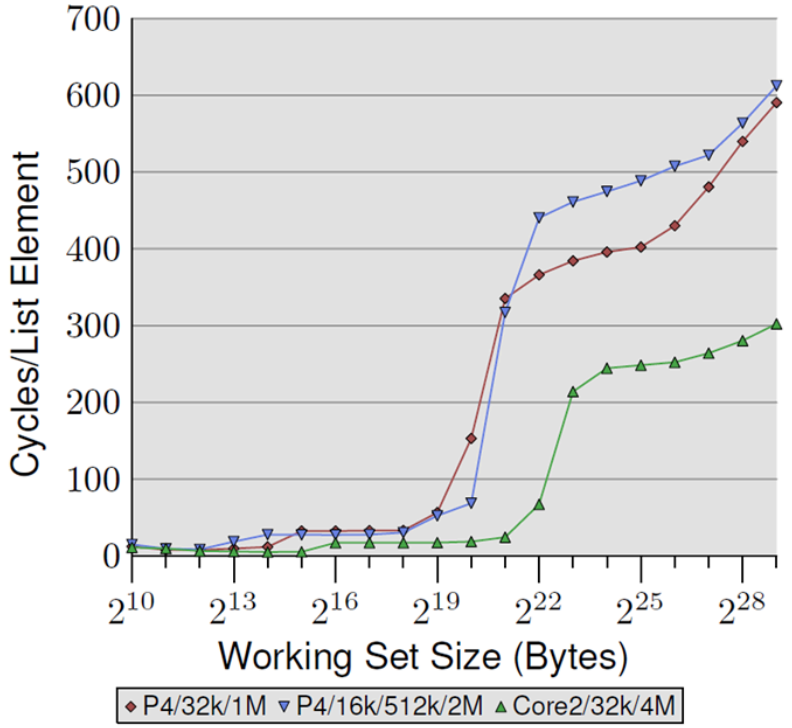
- NPAD=0 heißt 8 Byte Elemente in einer Liste

Dramatischer Einfluss auf die Laufzeit!

Cache: NPAD=0, Prefetching misslingt



Größere **Chaches** verschieben Stufen

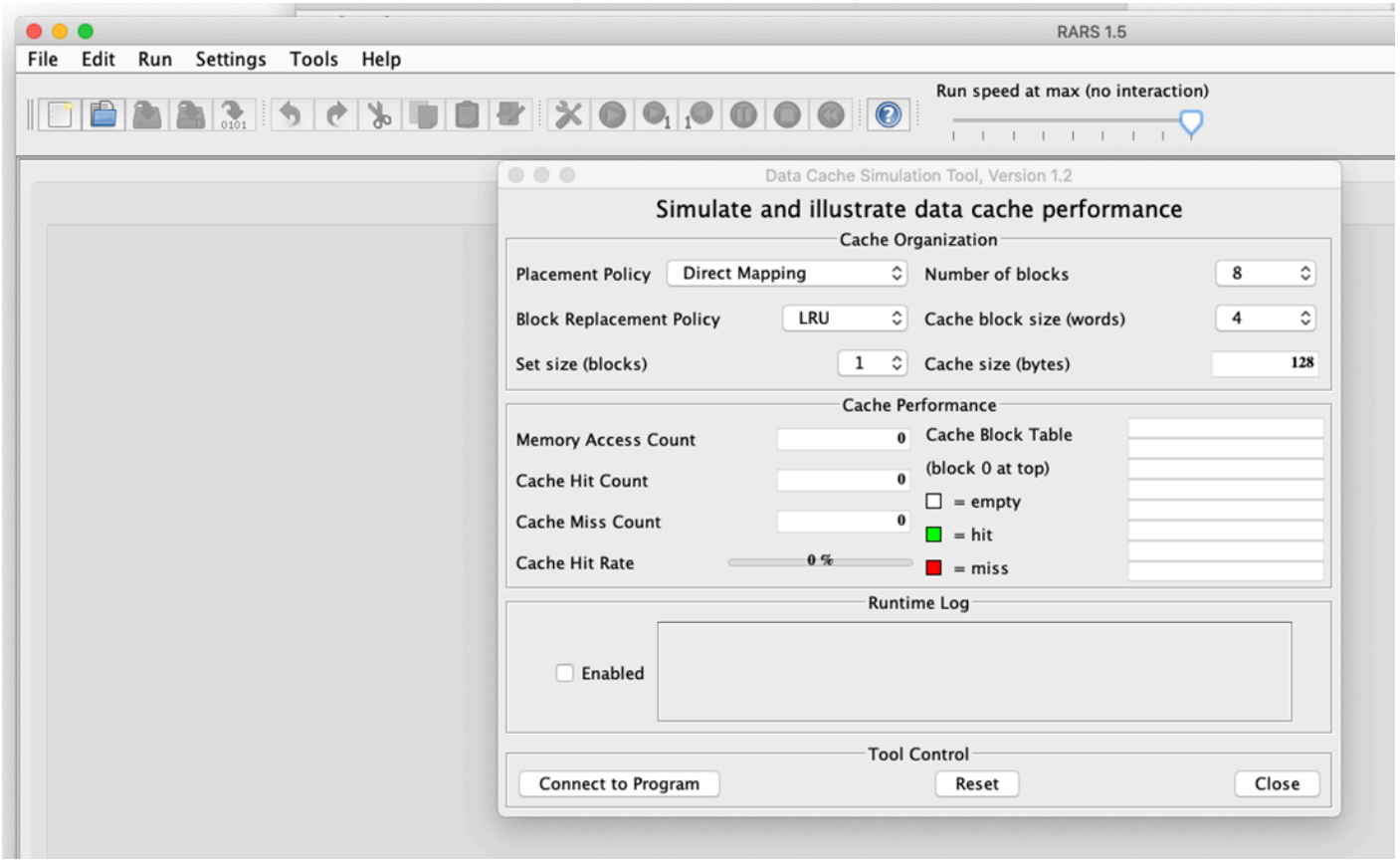


→ Jetzt TLBs & Caches ansehen!

Cache-Simulation in RARS

RARS erlaubt die Cache Simulation.
Achtung:

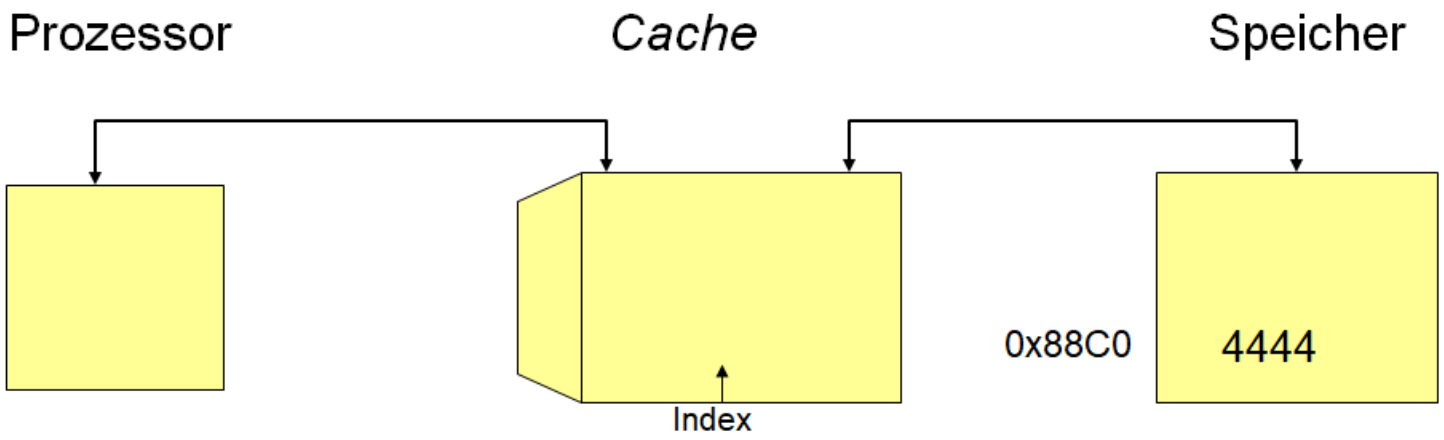
- block size = line size
- set size = Assoziativität



Schreibverfahren

Strategien zum Rückschreiben Cache → (Haupt-) Speicher

1. Write-Through (durchschreiben)
Jeder Schreibvorgang in den Cache führt zu einer unmittelbaren Aktualisierung des (Haupt-)Speichers

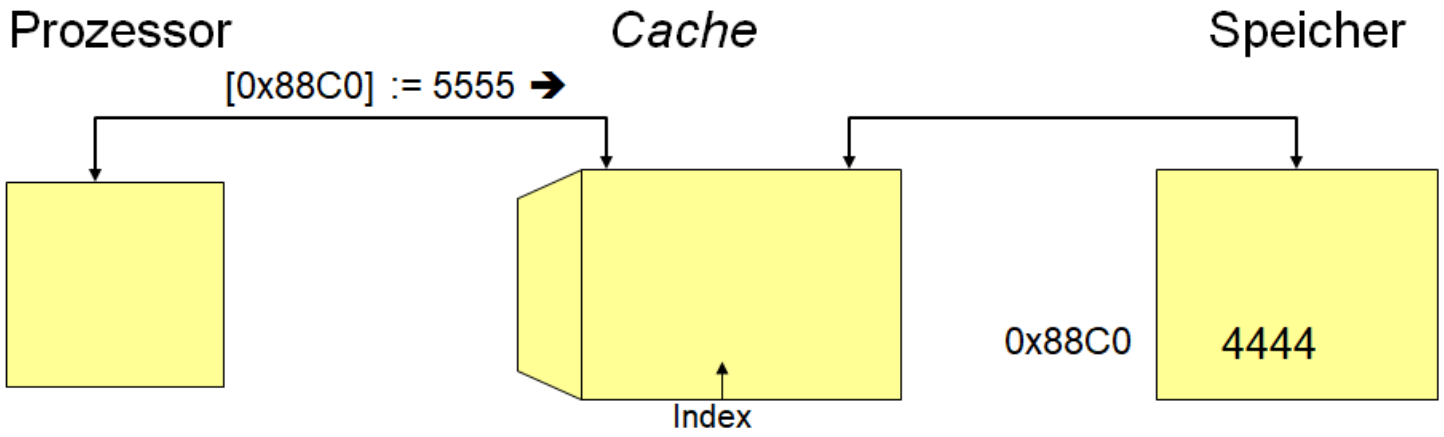


Speicher wird Engpass, es sei denn, der Anteil an Schreiboperationen ist klein oder der (Haupt-) Speicher ist nur wenig langsamer als der Cache.

Schreibverfahren

Strategien zum Rückschreiben Cache → (Haupt-) Speicher

1. Write-Through (durchschreiben)
Jeder Schreibvorgang in den Cache führt zu einer unmittelbaren Aktualisierung des (Haupt-)Speichers

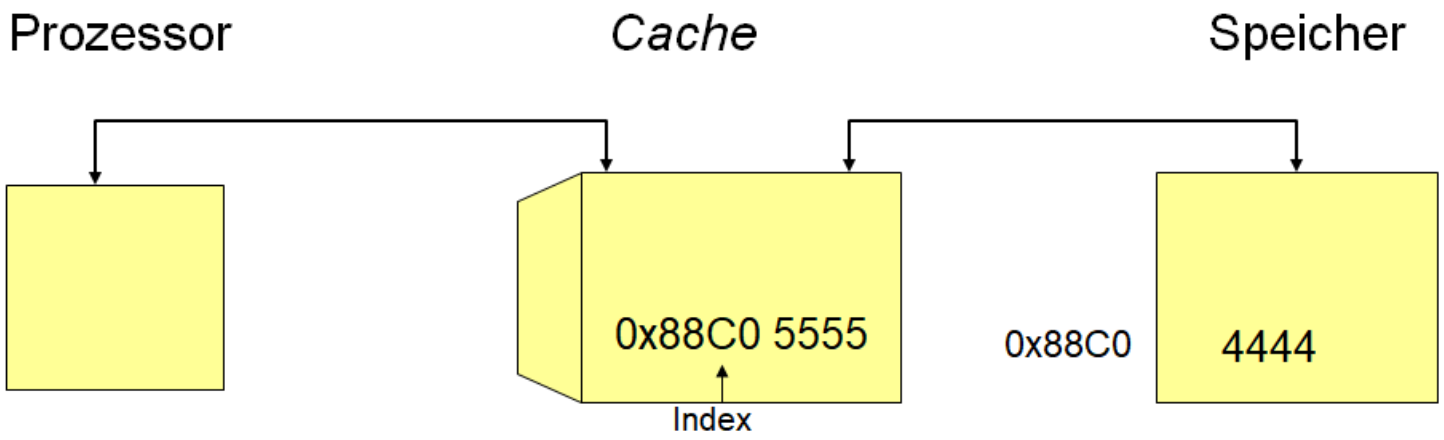


Speicher wird Engpass, es sei denn, der Anteil an Schreiboperationen ist klein oder der (Haupt-) Speicher ist nur wenig langsamer als der Cache.

Schreibverfahren

Strategien zum Rückschreiben Cache → (Haupt-) Speicher

1. Write-Through (durchschreiben)
Jeder Schreibvorgang in den Cache führt zu einer unmittelbaren Aktualisierung des (Haupt-)Speichers

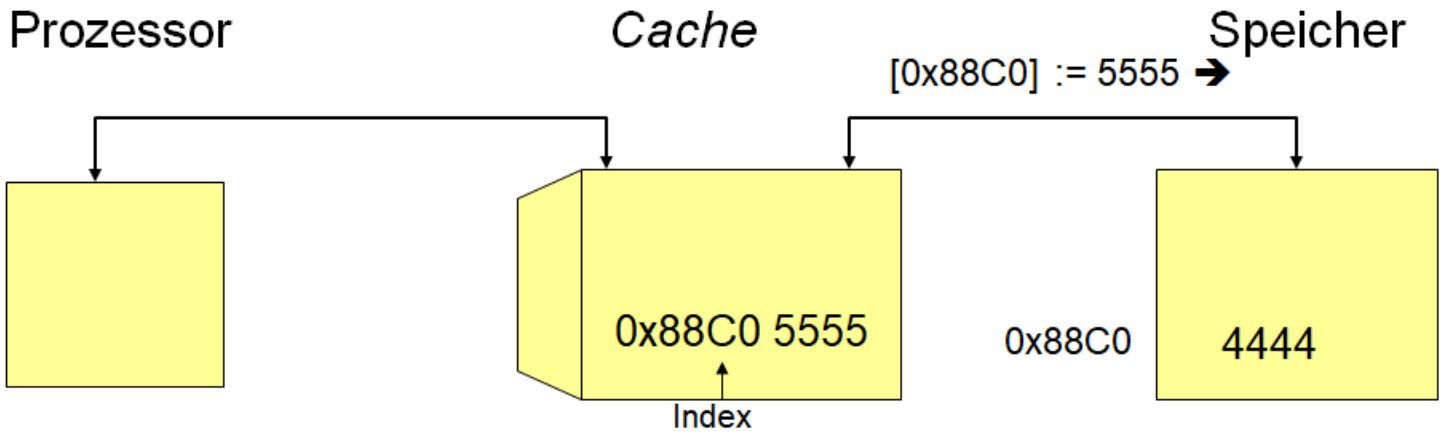


Speicher wird Engpass, es sei denn, der Anteil an Schreiboperationen ist klein oder der (Haupt-) Speicher ist nur wenig langsamer als der Cache.

Schreibverfahren

Strategien zum Rückschreiben Cache → (Haupt-) Speicher

1. Write-Through (durchschreiben)
Jeder Schreibvorgang in den Cache führt zu einer unmittelbaren Aktualisierung des (Haupt-)Speichers

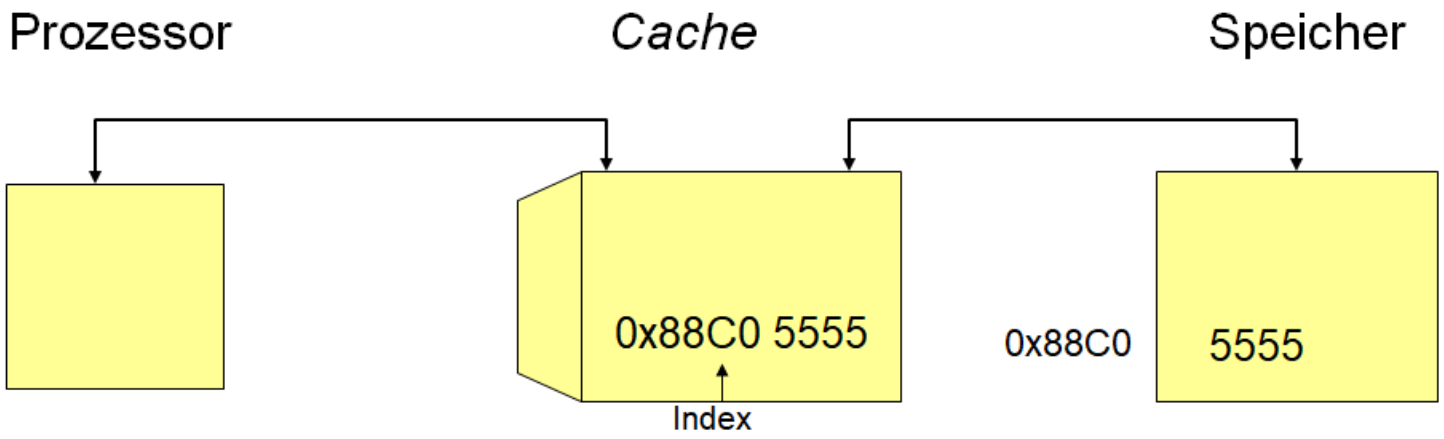


Speicher wird Engpass, es sei denn, der Anteil an Schreiboperationen ist klein oder der (Haupt-) Speicher ist nur wenig langsamer als der Cache.

Schreibverfahren

Strategien zum Rückschreiben Cache → (Haupt-) Speicher

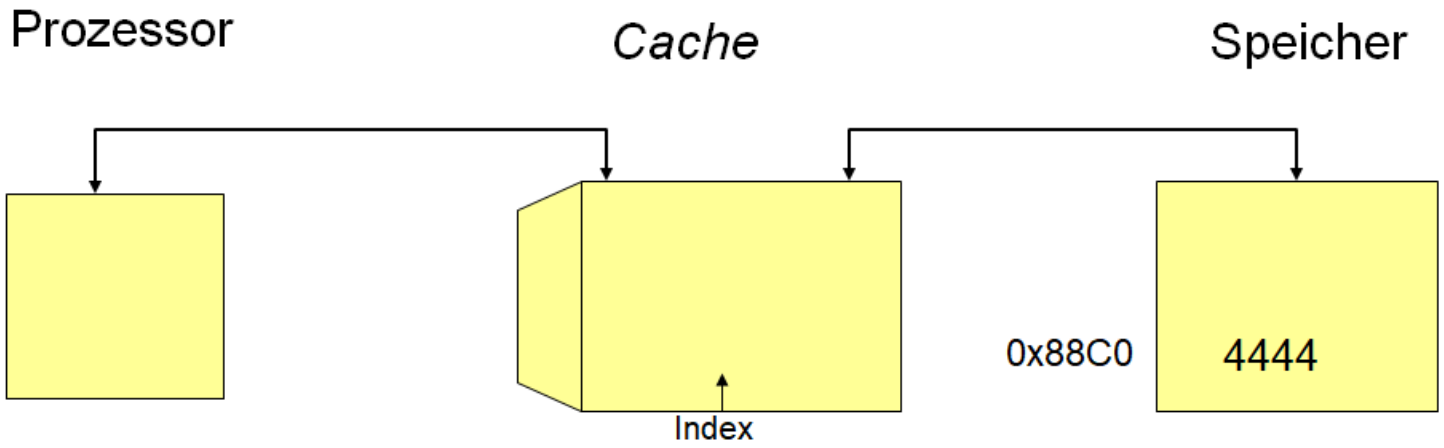
1. Write-Through (durchschreiben)
Jeder Schreibvorgang in den Cache führt zu einer unmittelbaren Aktualisierung des (Haupt-)Speichers



Speicher wird Engpass, es sei denn, der Anteil an Schreiboperationen ist klein oder der (Haupt-) Speicher ist nur wenig langsamer als der Cache.

Schreibverfahren

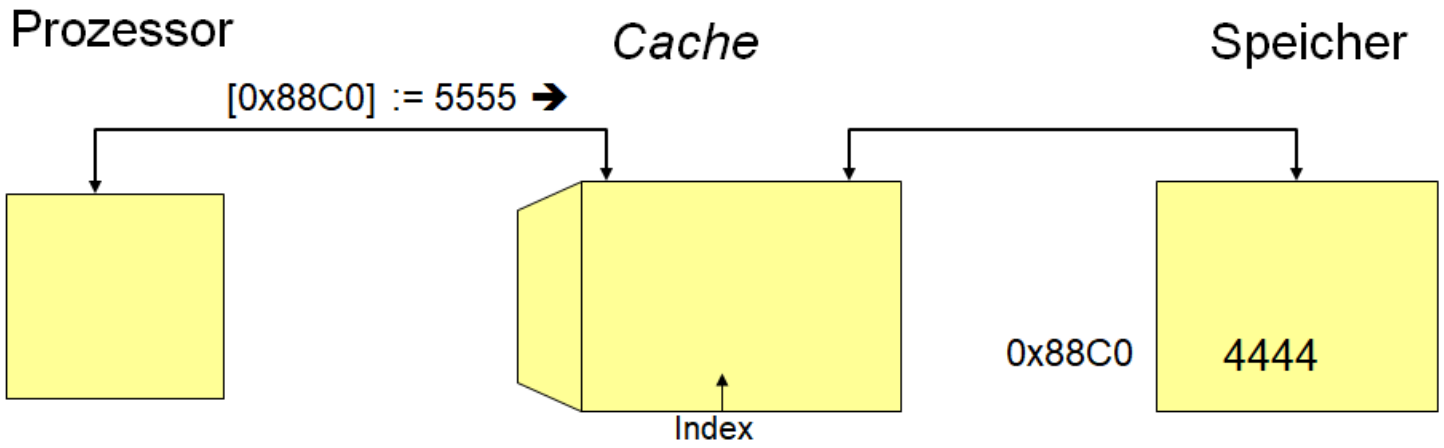
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

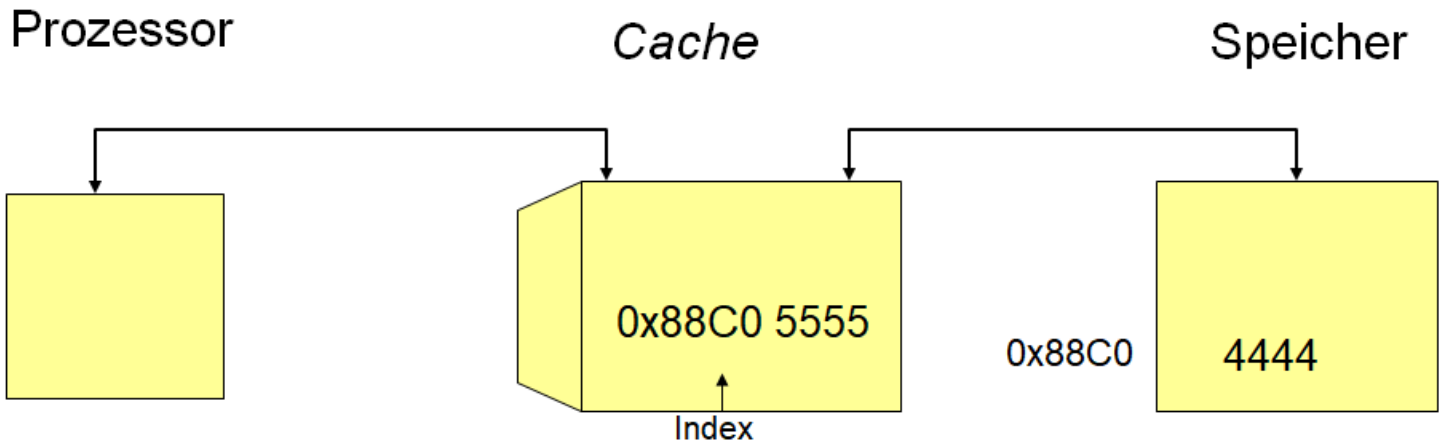
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

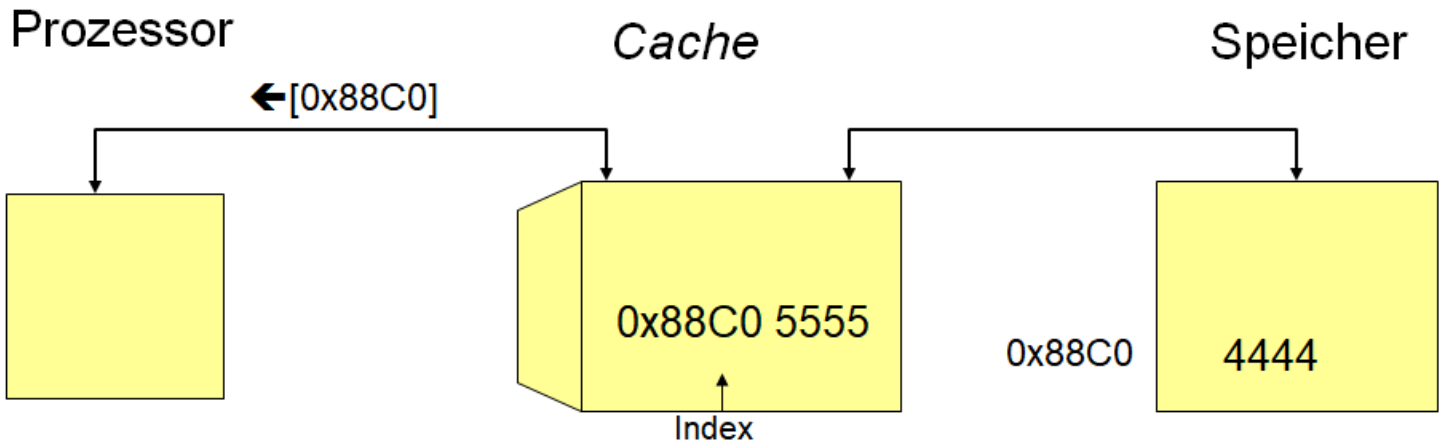
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

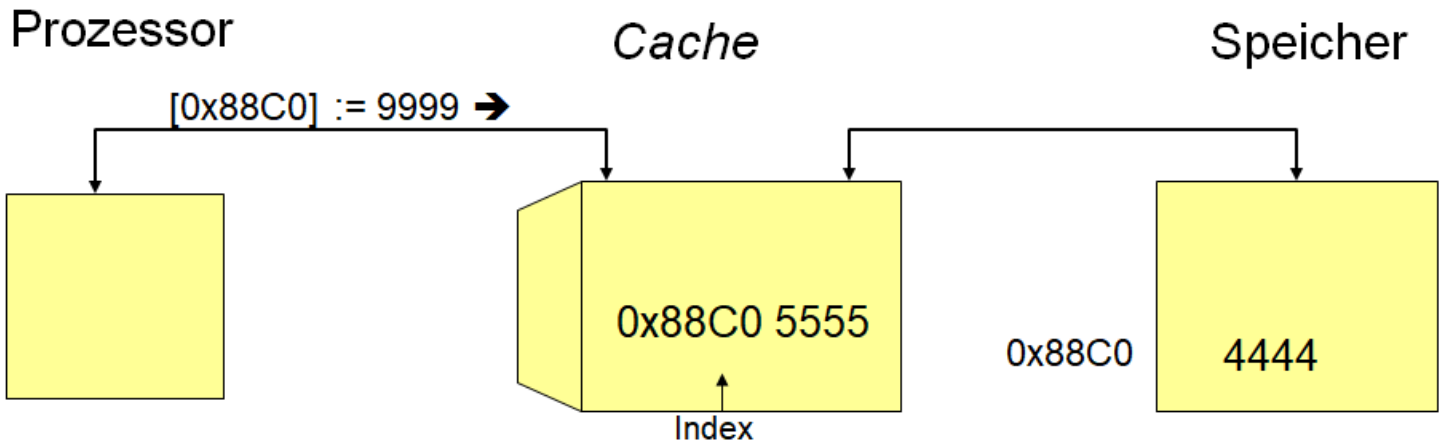
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

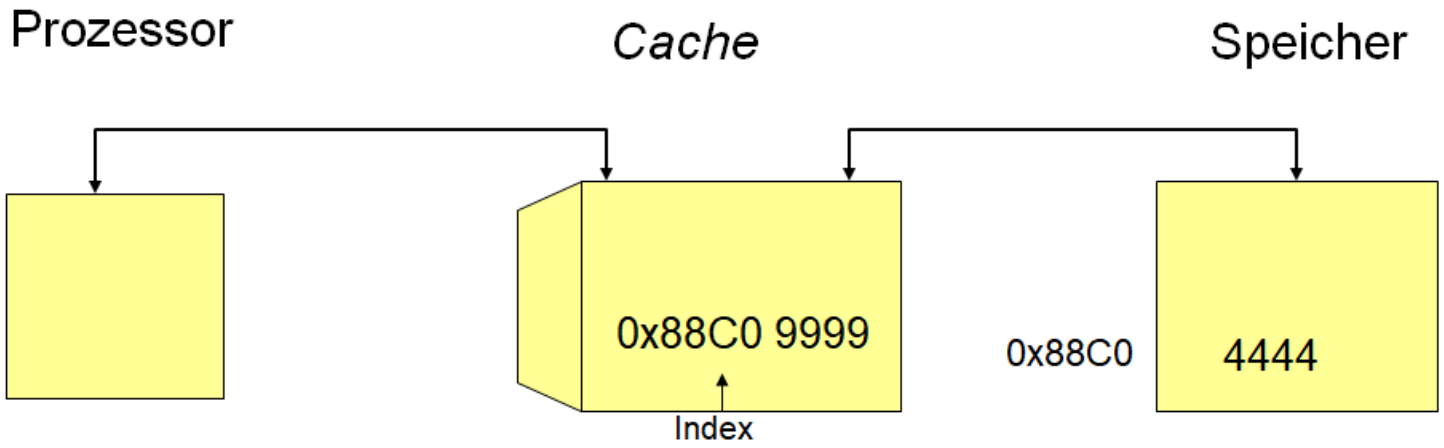
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

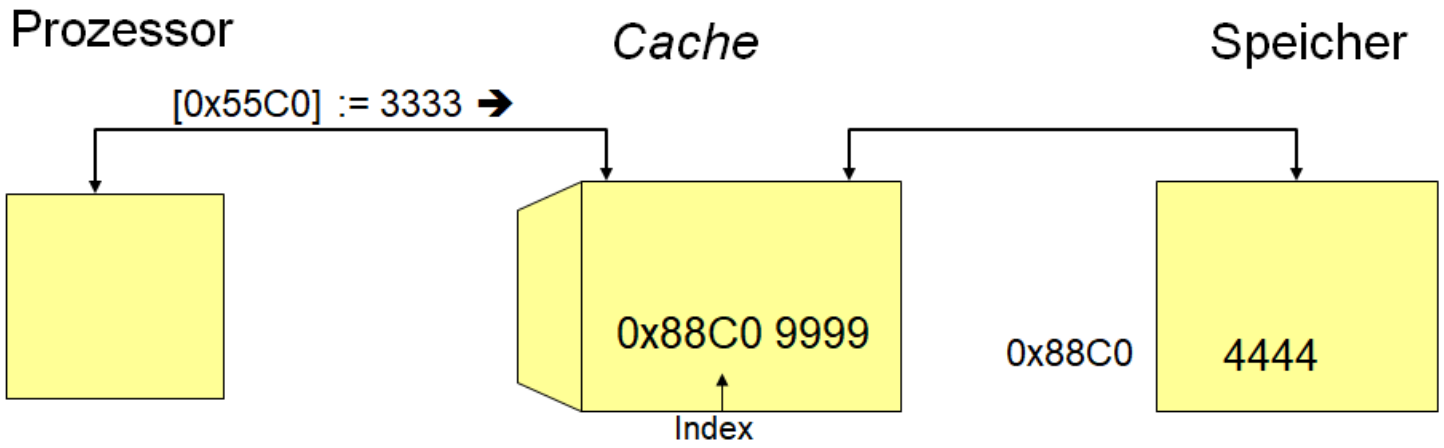
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

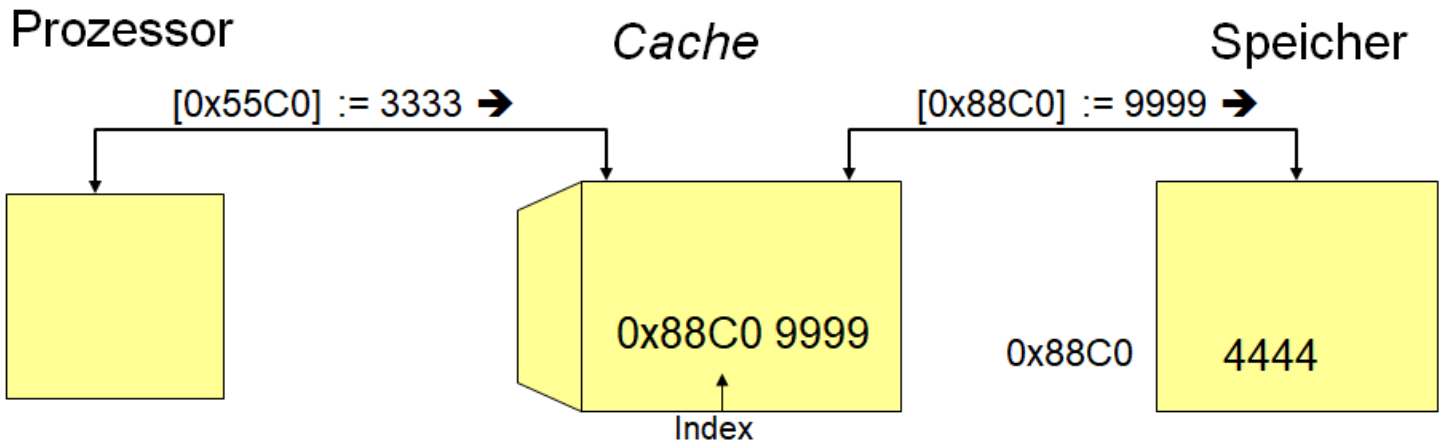
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

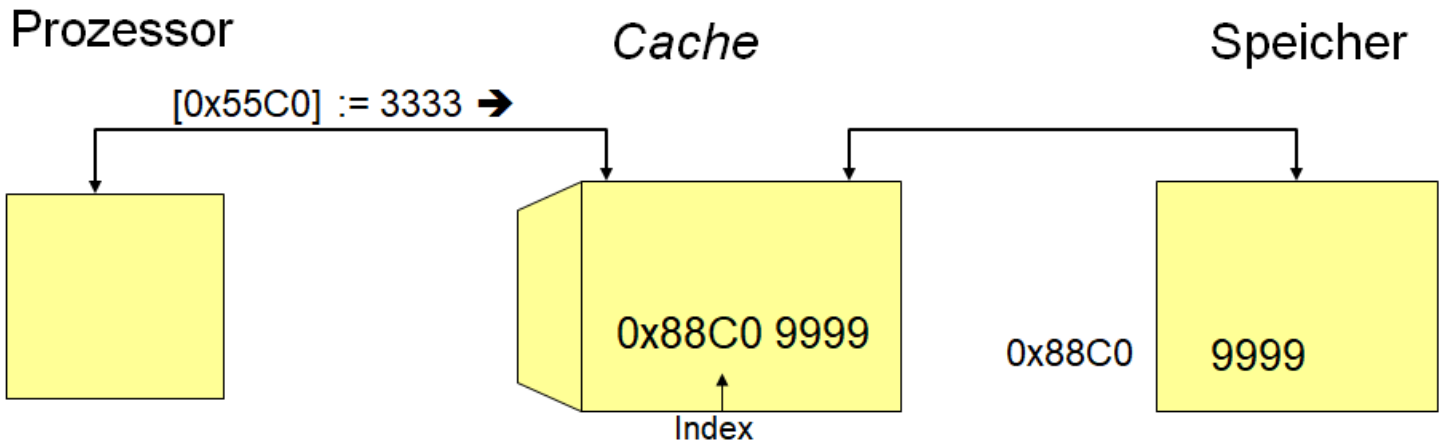
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

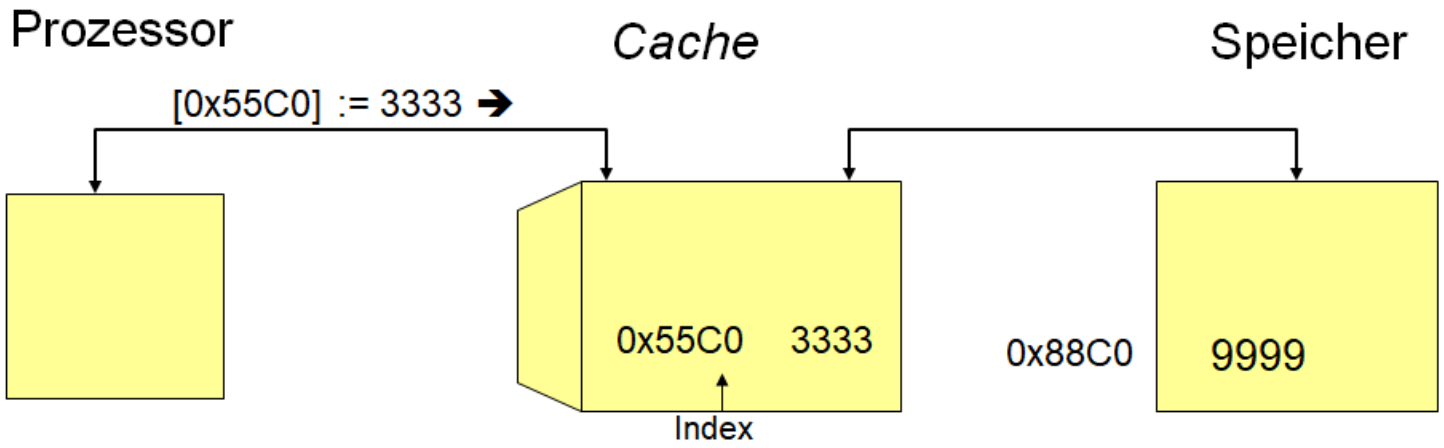
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Schreibverfahren

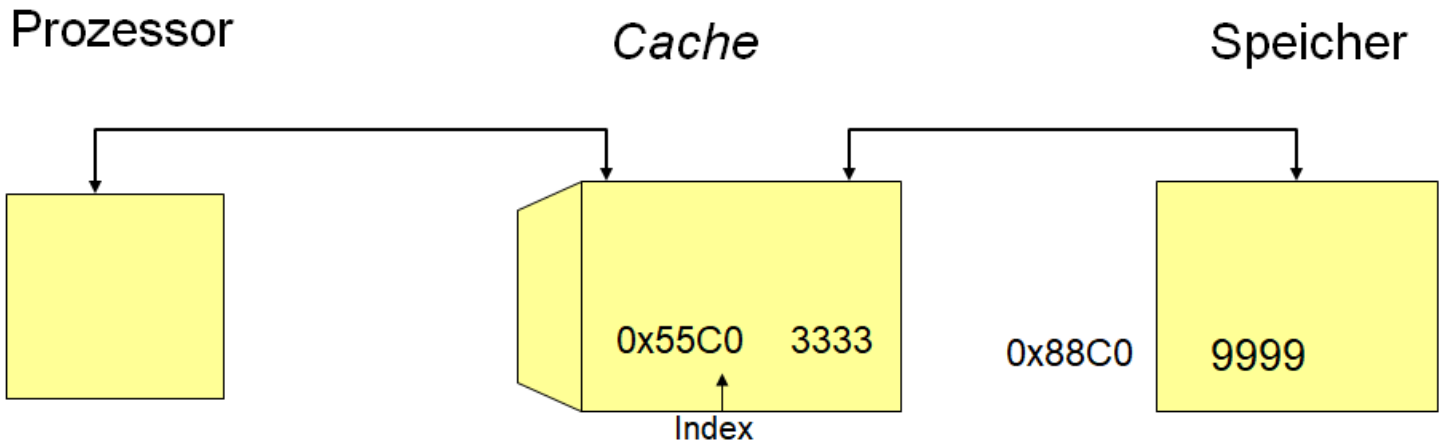
2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

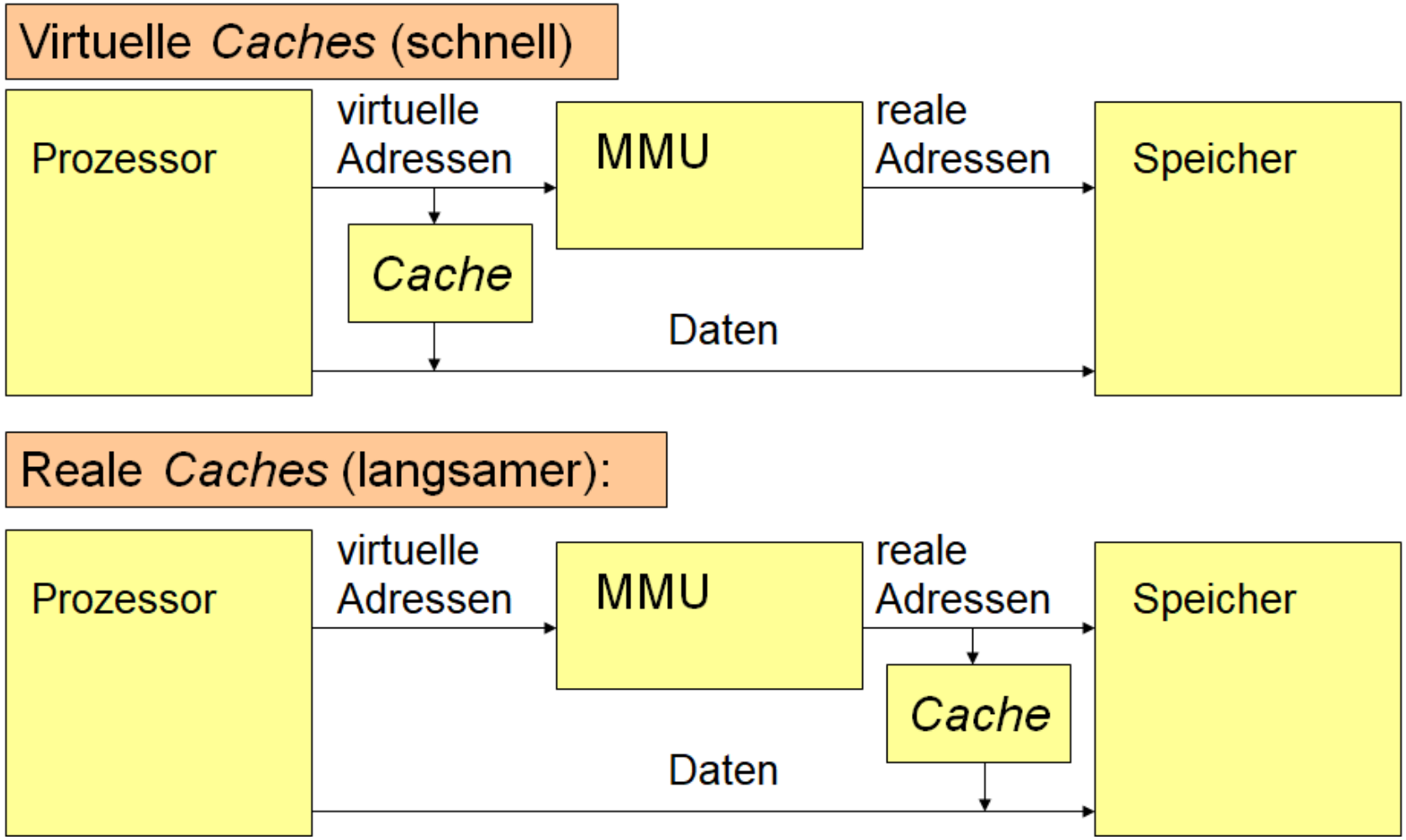
Schreibverfahren

2. Copy-Back, conflicting use write back:
Rückschreiben erfolgt erst, wenn die Cache-Zeile überschrieben wird.



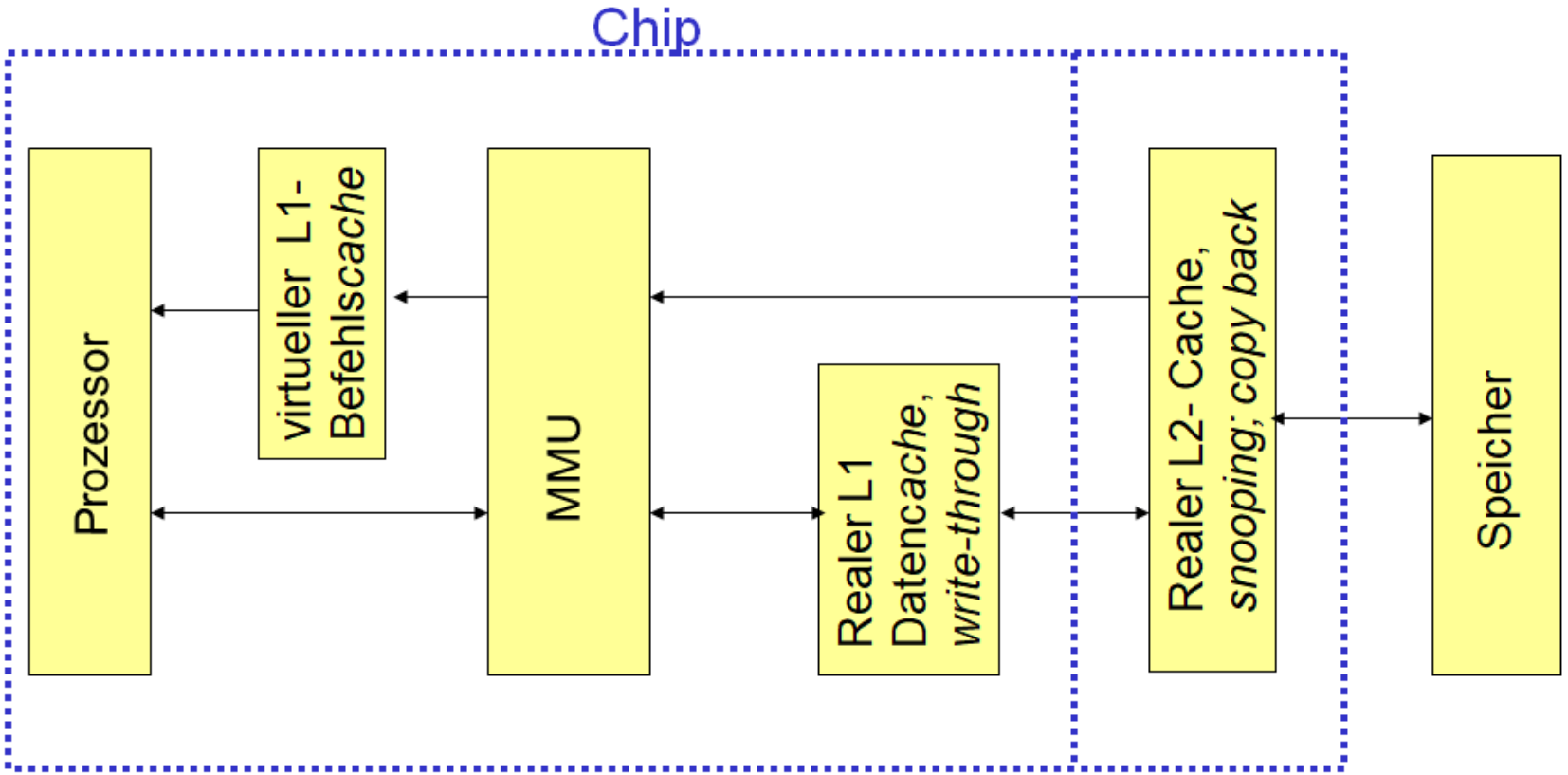
Funktioniert auch bei großen Geschwindigkeitsunterschieden zwischen Cache und Speicher. Vorkehrungen erforderlich, damit keine veralteten Werte aus dem Speicher kopiert werden.

Virtuelle und reale Caches



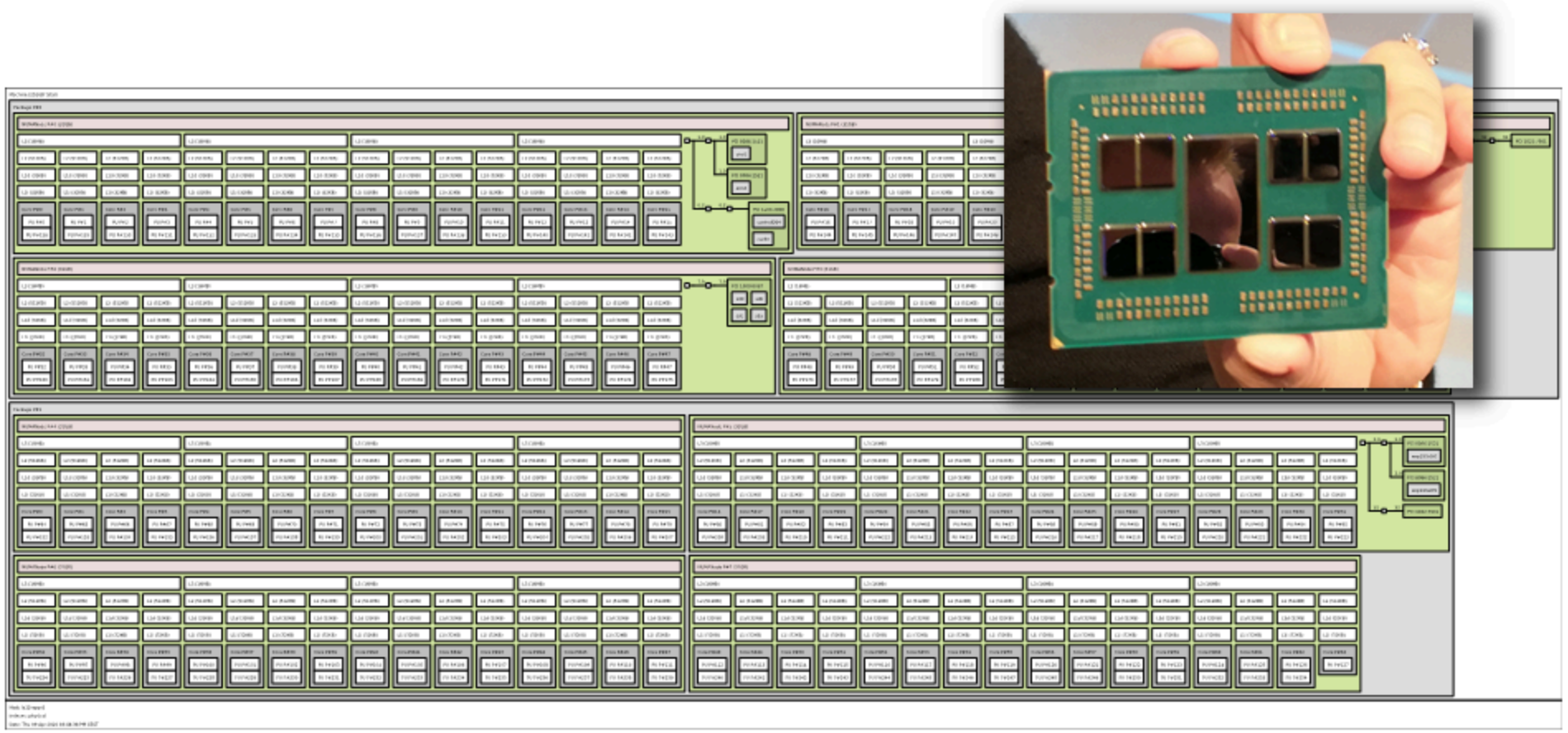
Systeme mit mehreren Caches

Beispiel:



Beispiel: AMD Prozessoren

Beispiel: AMD EPYC 7742 64-Core Processor:



Austauschverfahren

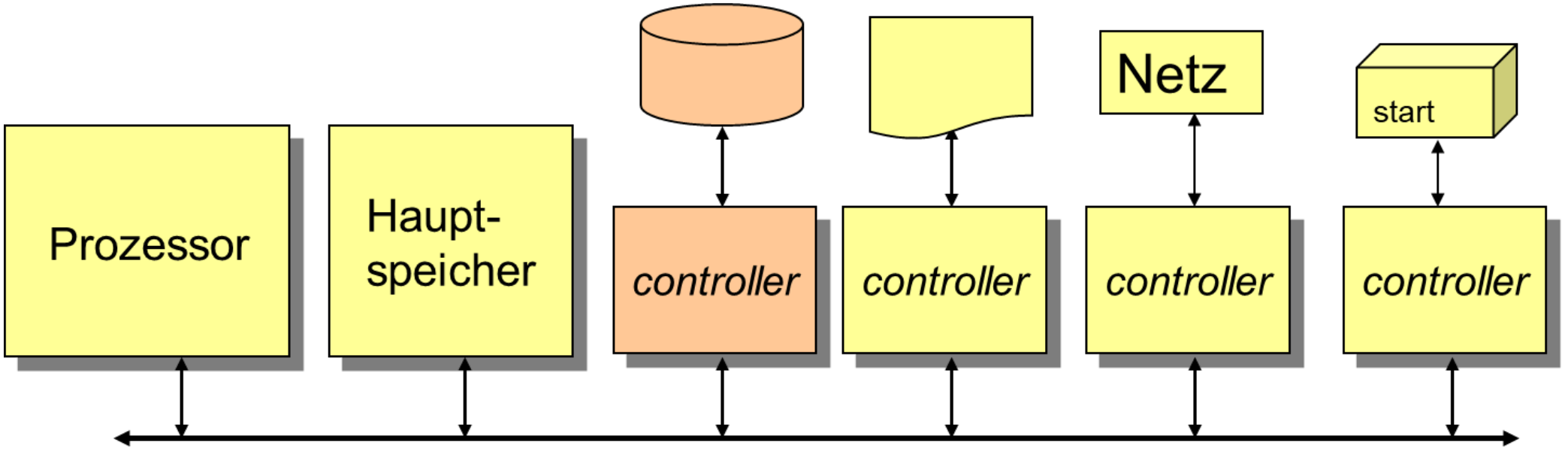
Überschreiben von Einträgen in TLB, Caches, Kacheln:

1. **Random- bzw. Zufallsverfahren**
   Ein zufällig ausgewählter Eintrag wird überschrieben.
2. **NRU: not recently used**
   Einteilung von Einträgen in 4 Klassen:
   a. **Nicht benutzte, nicht modifizierte Einträge**
   b. **Nicht benutzte, modifizierte Seiten**
      Benutzt-Kennzeichnung wird nach einiger Zeit zurückgesetzt.
   c. **Benutzte, nicht modifizierte Seiten**
   d. **Benutzte, modifizierte Seiten**
      Überschrieben wird ein zufällig ausgewählter Eintrag der niedrigsten Klasse, die nicht-leer ist.
3. **LRU: least recently used**
   Überschreiben des Eintrags, der am längsten nicht benutzt wurde.

Zusammenfassung

- Cache-Zeilen: Einheit der Gültigkeitsprüfung
- Cache-Blöcke: Einheit des Nachladens
- Organisationsformen:
  - direct mapping
  - set associative mapping
- Unterscheidung zwischen virtuellen und realen Caches
- Moderne Systeme besitzen mehrere Cache-Ebenen

Massenspeicher



- Plattenspeicher
  - Disc-Arrays
- Flash-Speicher

- CD-ROM
- DVD
- Bandlaufwerke
- Weitere

Mögliche Stufen der Speicherhierarchie und derzeit eingesetzte Technologien

Tertiärspeicher (Optisch, Bänder, Cloud) (persistent)
Sekundärspeicher (Platten, Flash) (persistent)
Plattencaches (DRAM, Flash)
Haupt- bzw. Primärspeicher (DRAM)
Caches (SRAM) & TLBs
Register (SRAM)

Redundant arrays of inexpensive discs (RAID)
auch: Redundant arrays of independent discs

- Verbesserung der E/A-Geschwindigkeit durch paralleles Lesen/Schreiben auf mehrere Platten;
- Einsatz von Redundanz, damit dadurch nicht die Zuverlässigkeit leidet.

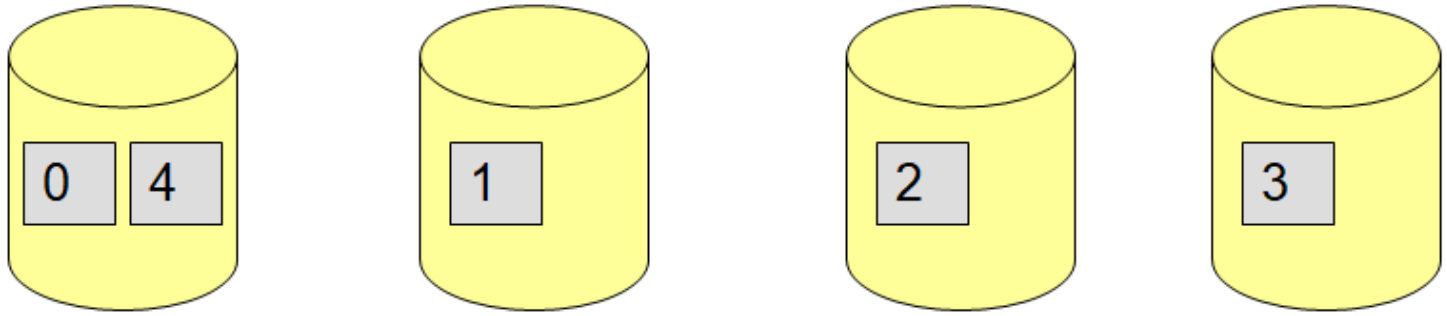
→ Disc arrays, drive arrays

RAID 0 (Striping)

Dateien werden unterteilt in so genannte Stripes.

Striping-Parameter = Länge eines Stripes.

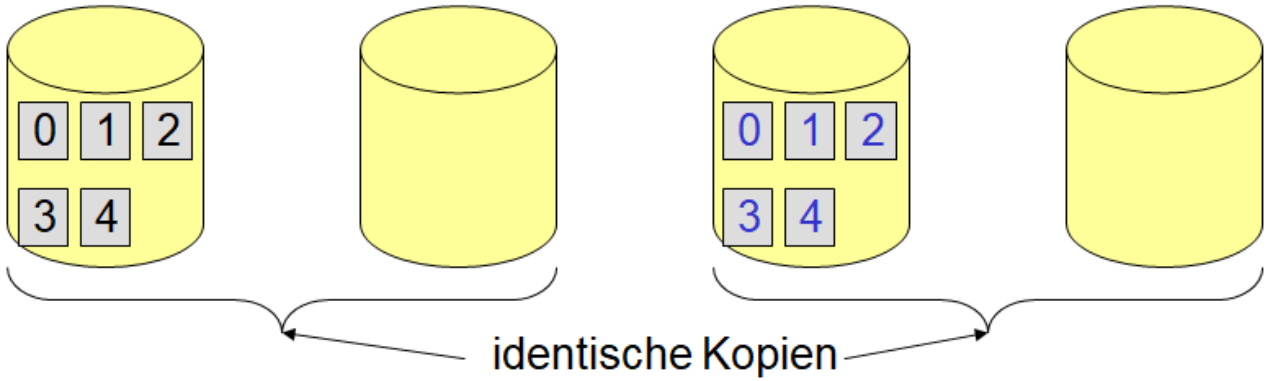
Diese Stripes werden auf den Platten verteilt.



- Keine Redundanz
- Nur Verbesserung der Übertragungsrate

RAID 1 (Mirroring)

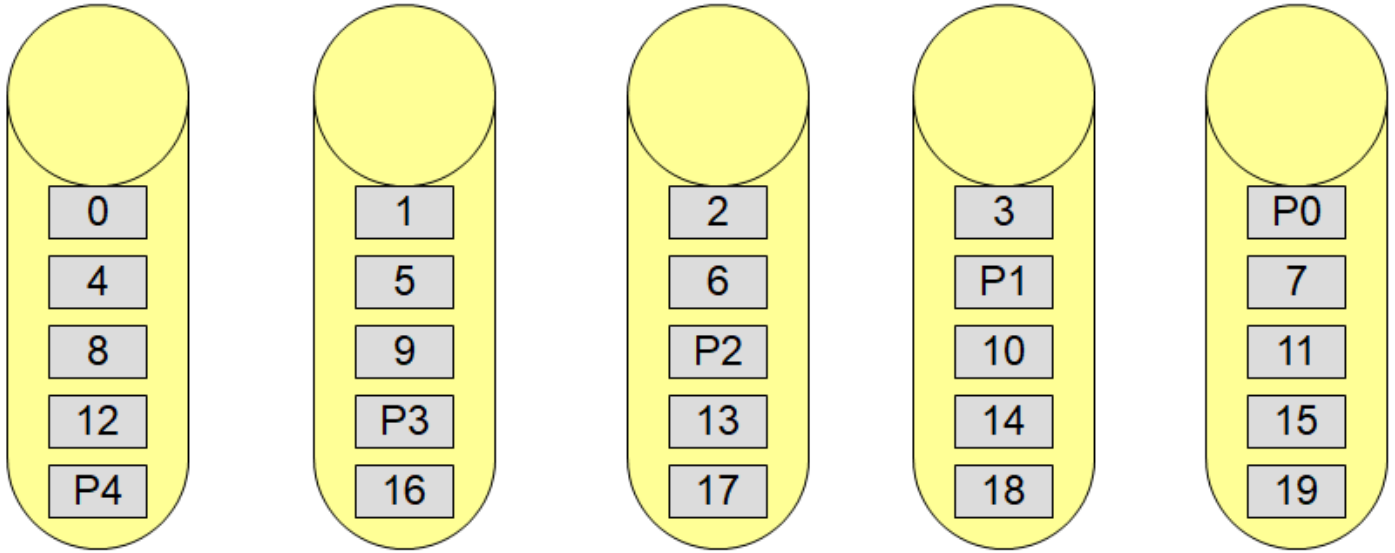
Dieselben Informationen werden auf zwei Platten geschrieben (gespiegelte Platten, mirrored discs)



- Erhöhte Fehlersicherheit
- Stark erhöhter Aufwand (x2)
- Es werden eigentlich nur 2 Plattenlaufwerke genutzt

RAID 5 (distributed parity)

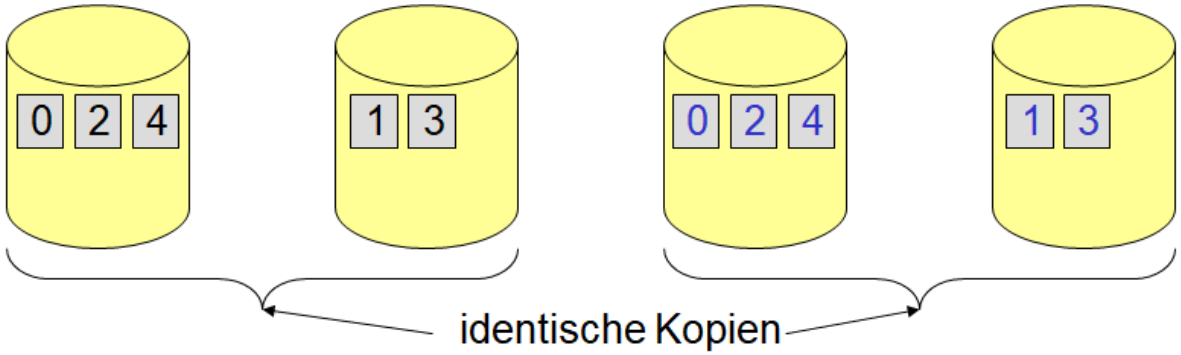
Die Paritätsinformation wird über verschiedene Platten verteilt



- Paritätsplatte nicht weiter ein Eingpass
- Fehler während der (langen) re-build-Zeit problematisch

RAID 0+1, RAID 01 (mirrored stripes)

- Hierarchien
- Gespiegelte Platten (RAID1), mit verteilten Stripes (RAID0) → *mirrored stripes*



- Lesegeschwindigkeit erhöht
- erhöhte Fehlersicherheit
- stark erhöhter Aufwand

RAID 1+0, RAID 10 (striped mirrors)

Striped mirrors: striping von gespiegelten Platten

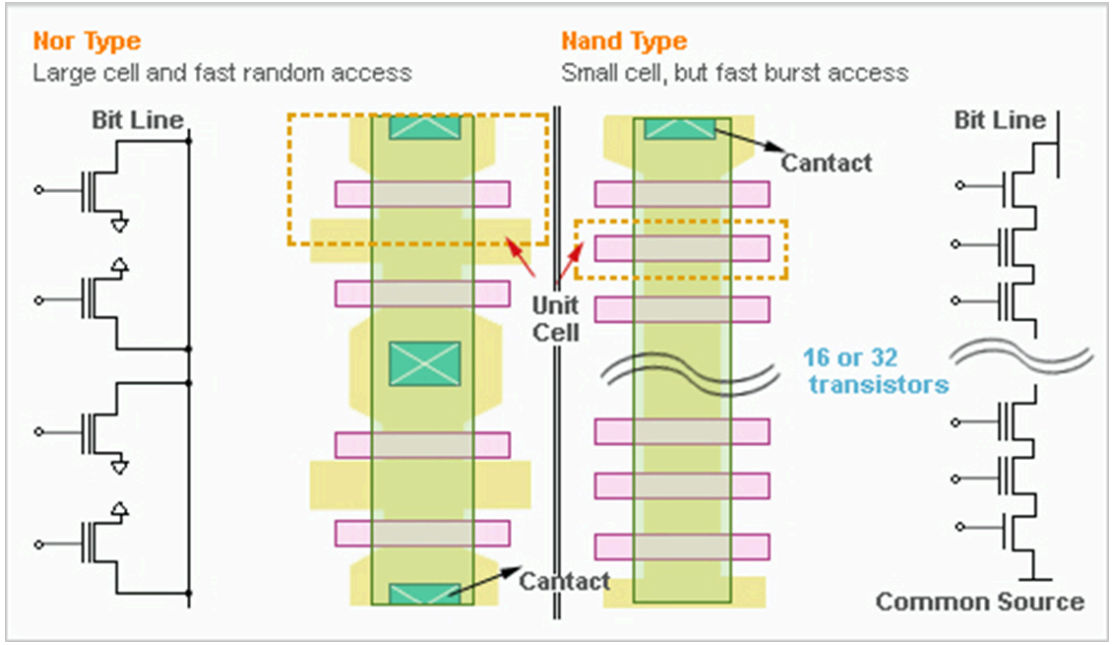
- Lesegeschwindigkeit erhöht,
- erhöhte Fehlersicherheit,
- stark erhöhter Aufwand.

Übersicht

RAID Variante	Technik	Fehlertoleranz	Daten-Platten	Prüfplatten
0	non-redundant	0	8	0
1	mirrored	1	8	8
0+1	Komb. v. 0 & 1	1	8	8
2	memory-style ECC	1	8	4
3	bit-interleaved parity	1	8	1
4	block-interleaved parity	1	8	1
5 **(Bevorzugt)**	dto, distributed parity	1	8	1
6	P+Q redundancy	2	8	2

NOR- und NAND-Flash

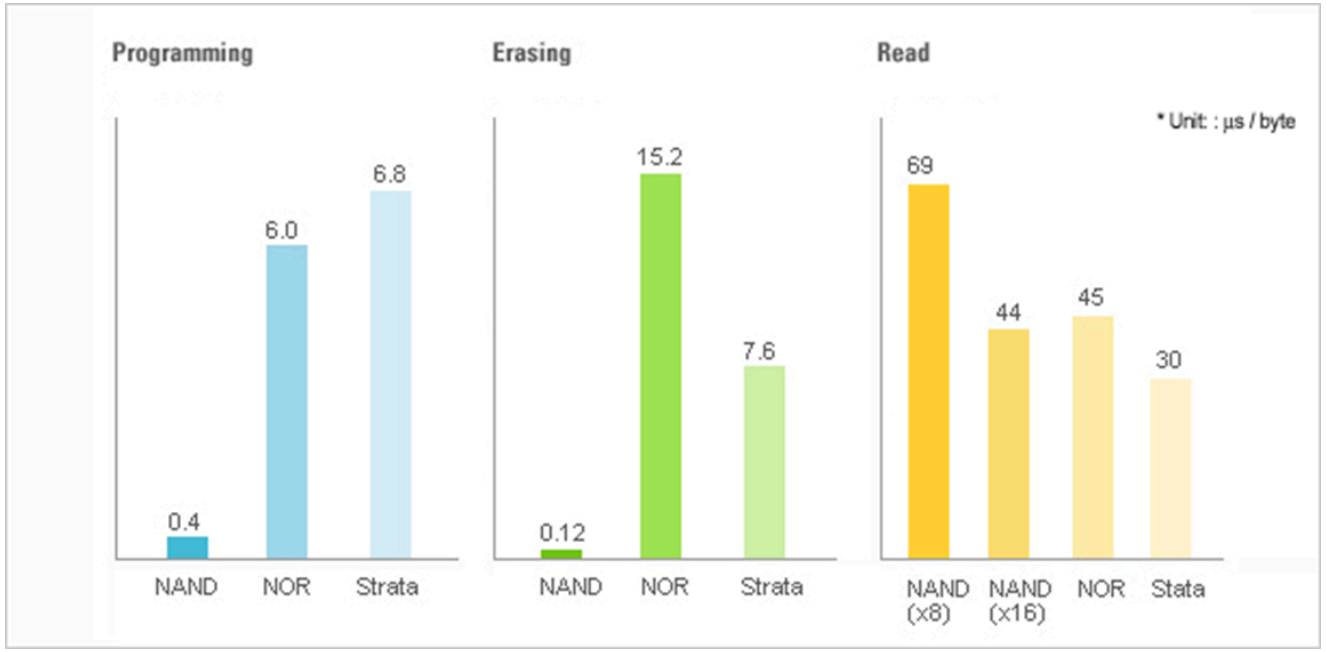
- **NOR**: 1 Transistor zwischen Bitleitung und Masse
- **NAND**: >1 Transistor zwischen Bitleitung und Masse



was at [www.samsung.com/Products/ Semicon-ductor/ Flash/FlashNews/ FlashStructure. htm] (2007)

Eigenschaften von NOR- und NAND-Flash- Speichern

Type/Eigenschaft	NOR	NAND
Wahfreier Zugang	Ja	Nein
Block löschen	Langsam	Schnell
Zellgröße	Groß	Klein
Zuverlässigkeit	Größer	Kleiner
Direktes Ausführen	Ja	Nein
Anwendungen	Codespeicherung, boot flash, set top box	Datenspeicher, USB, Sticks, Speicherkarten



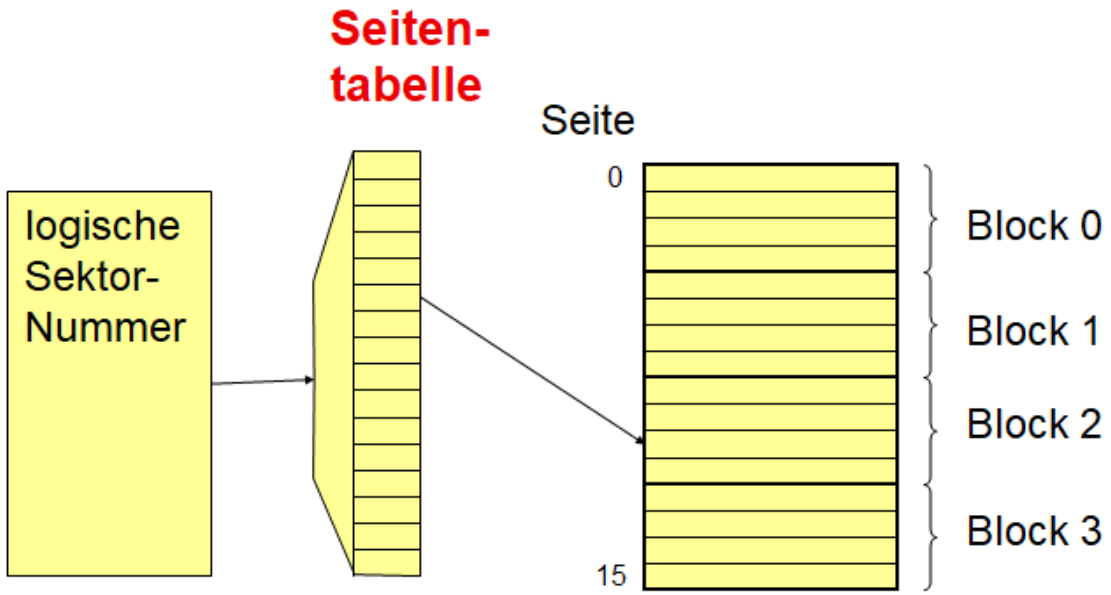
Charakteristische Eigenschaften von NAND Flash Speicher

- Speicher aufgeteilt in Blöcke (typ. 16-256 KB)
- Blöcke unterteilt in Seiten (typ. 0.5-5 KB)
- Schreib-/Lesevorgänge jeweils auf Seiten

	1 Bit/Zelle (SLC)	**> 1Bit/Zelle (MLC)**
Lesen (Seite)	25 µs	>> 25µs
Schreiben (Seite)	300 µs	>> 300 µs
Löschen (Block)	2 ms	1.5 ms

J. Lee, S. Kim, H. Kwin, C. Hyun, S, Ahn, J. Choi, D. Lee, S.Noh: Block Recycling Schemes and Their Cost-based Optimization in NAND Flash Memory Based Storage System, EMSOFT'07, Sept. 2007

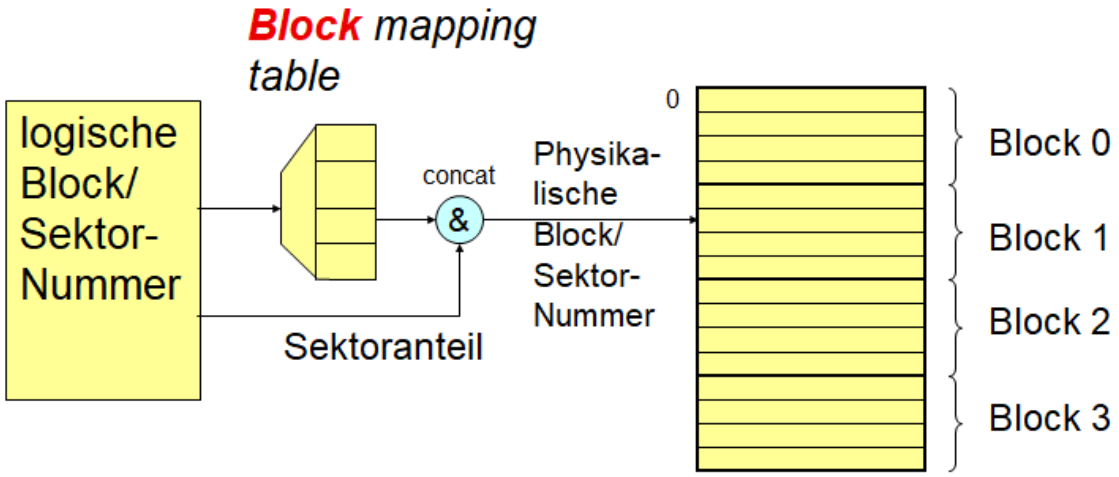
Seiten-/ Sektorabbildung mit Flash translation layer (FTL)



- Invertierte Seitentabelle im Flashspeicher gespeichert (Extra Bits)
- “Normale Seitentabelle” während der Initialisierung erzeugt
- Seitentabelle kann sehr groß werden; Wird in kleinen NOR Flash-Speichern benutzt.

Sektor ≈ Seite + Extra Bits

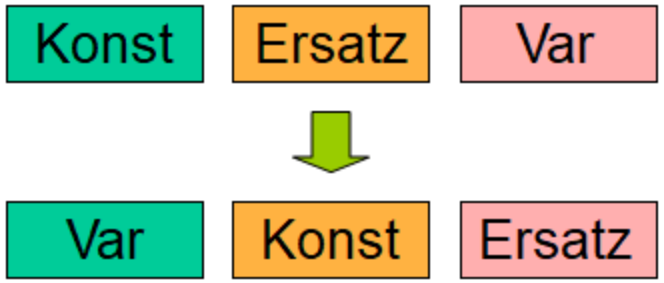
Block mapping flash translation layer (FTL)



- Abbildungstabellen kleiner als bei Seiten-basierten FTLs
- In großen NAND Flash-Speichern benutzt
- Einfache Realisierung
- Wiederholtes Schreiben erfordert Kopieren auf einen neuen Block
- Schlechte *Performance* bei wiederholtem und zufälligem Schreiben
- Hybride Verfahren

Ausgleich der Abnutzung (wear levelling)

Beispiel (Lofgren et al., 2000, 2003):



- Jede *erase unit* (Block) besitzt Löschzähler
- 1 Block wird als Ersatz vorgehalten
- Wenn ein häufig genutzter Block frei wird, wird der Zähler gegen den des am wenigsten benutzten Blocks verglichen. Wenn der Unterschied groß ist:
  - Inhalt wenig genutzten Blocks (≈ Konstanten) → Ersatz
  - Inhalt häufig genutzten Blocks → am wenigsten genutzter Block
  - Häufig genutzter Block wird zum Ersatzblock

Source: Gal, Toledo, ACM Computing Surveys, June 2005

Zusammenfassung

Nicht flüchtiger Speicher:

- Plattenspeicher sind traditionell preisgünstiges Speichermedium
- RAID-Speicher bieten benötigte Zuverlässigkeit
- Flash-Speicher bieten schnelle Speicheroptionen, aber mit begrenzter Anzahl von Schreiboperationen

Appendix: Caches

Realer Cache bei Prozesswechsel (context switch)

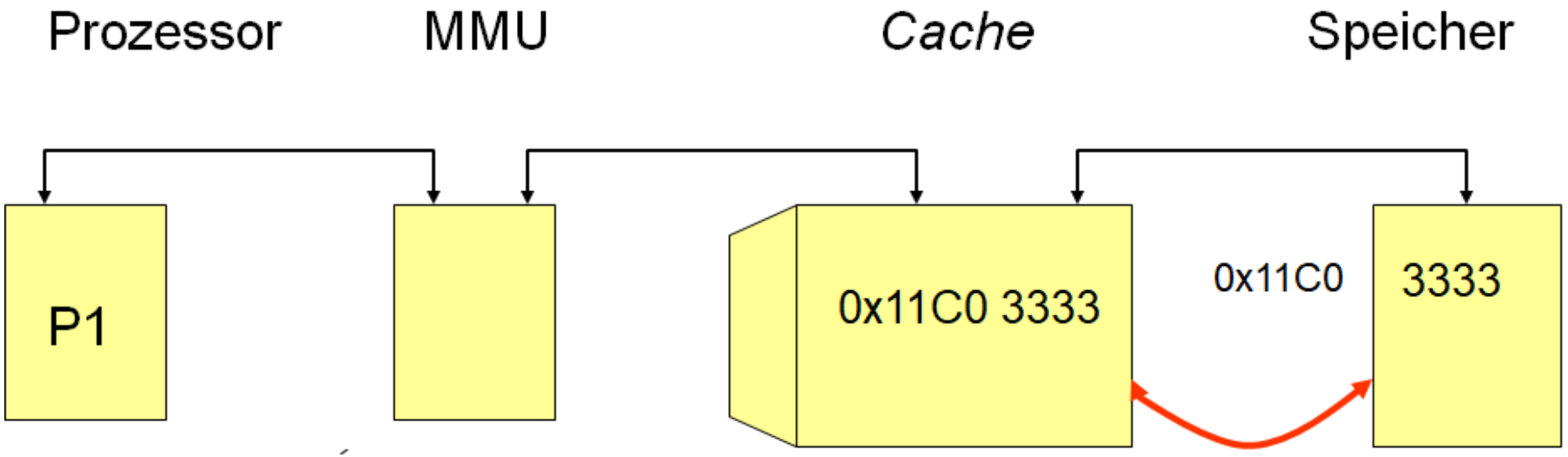
- Zu einer bestimmten Adresse im Cache gehört vor und nach dem Prozesswechsel dieselbe Zelle im Speicher

Realer Cache bei Seitenfehler (Seite muss per demand paging eingelagert werden)

- Zu einer bestimmten Adresse im Cache gehört vor und nach dem Aus- & Einlagern von Seiten eine andere Information
- Bei Seitenfehlern müssen alle Cache-Zeilen, die Kopien der verdrängten Seite enthalten, ungültig erklärt werden.

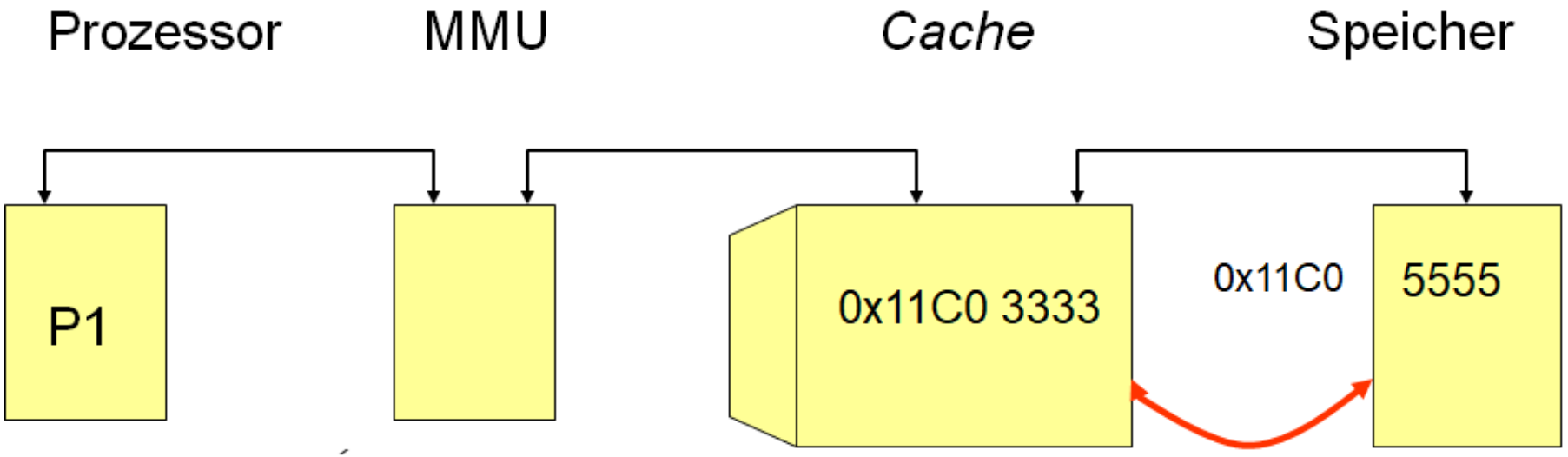
Bus snooping (Bus-Lauschen)

- Bei Seitenfehlern müssen alle *Chache*-Zeilen, die Kopien der verdrängten Seite enthalten, ungültig erklärt werden
- *Cache* “lauscht”, ob ein Schreibvorgang in den Speicher eine Zelle betrifft, von der er eine Kopie hat, wenn ja, dann erklärt er diese für ungültig



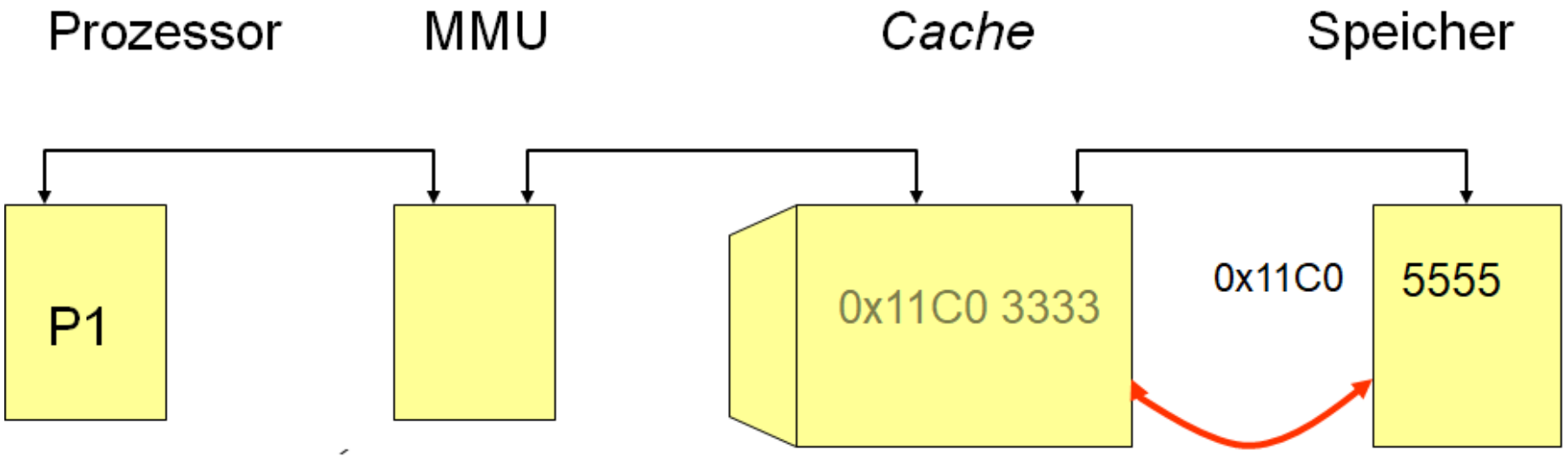
Bus snooping (Bus-Lauschen)

- Bei Seitenfehlern müssen alle *Chache*-Zeilen, die Kopien der verdrängten Seite enthalten, ungültig erklärt werden
- *Cache* “lauscht”, ob ein Schreibvorgang in den Speicher eine Zelle betrifft, von der er eine Kopie hat, wenn ja, dann erklärt er diese für ungültig



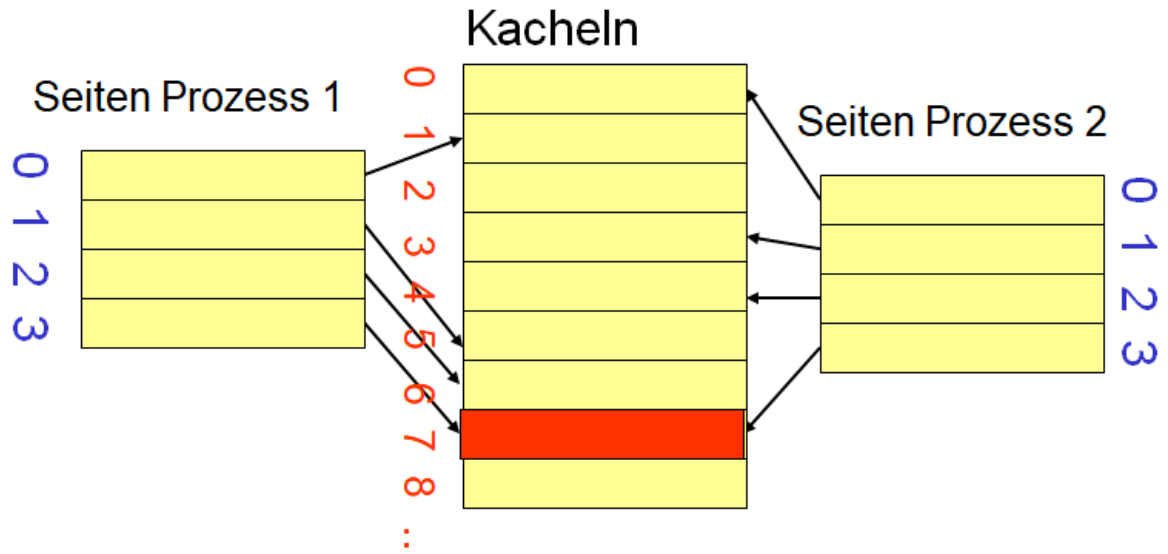
Bus snooping (Bus-Lauschen)

- Bei Seitenfehlern müssen alle *Chache*-Zeilen, die Kopien der verdrängten Seite enthalten, ungültig erklärt werden
- *Cache* “lauscht”, ob ein Schreibvorgang in den Speicher eine Zelle betrifft, von der er eine Kopie hat, wenn ja, dann erklärt er diese für ungültig



Gemeinsam genutzte Seiten (sharing)

- Für gemeinsame Daten oder gemeinsame Befehle: Eintrag derselben in Kachel in mehrere Seitentabellen



Realer Cache bei gemeinsam genutzter Seite

- Annahme *: Beide Prozesse benutzen dieselbe virtuelle Adresse für den gemeinsamen Speicherbereich

Virtueller Cache bei Prozesswechsel (context switch)

- Zu einer bestimmten Adresse im Cache gehört vor und nach dem Prozesswechsel eine andere Zelle im Speicher.

Virtueller Cache bei Seitenfehler

- Zu einer bestimmten Adresse im Cache gehört vor und nach dem Seitenfehler dieselbe Information.

Virtueller Cache bei gemeinsam genutzter Seite (sharing)

- Annahme: Beide Prozesse benutzen dieselbe virtuelle Adresse für den gemeinsamen Speicherbereich

Virtueller Cache mit PIDs bei Prozesswechsel (context switch)

- Virtuelle Adresse wird um Prozessnummer (process identifier, PID) erwei-tert. Zu einer bestimmten Adresse im Cache gehört vor und nach dem Prozesswechsel wieder dieselbe Zelle.

Virtueller Cache mit PIDs bei Seitenfehler

- Zu einer bestimmten Adresse im Cache gehört vor und nach dem Seitenfehler dieselbe Information

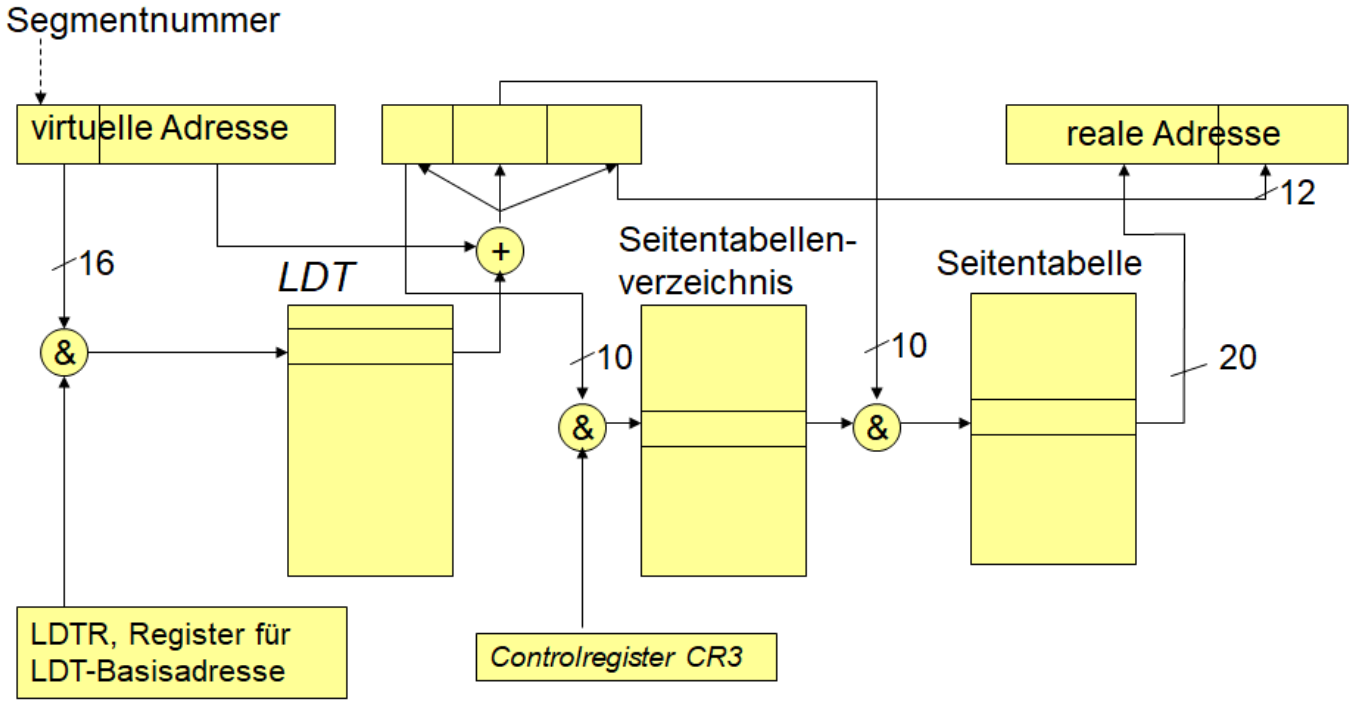
Virtueller Cache mit PIDs bei gemeinsam genutzter Seite (sharing)

Cache-Kohärenz

	Realer Cache	Virtueller Cache mit PIDs	Virtueller Cache
Inhalt nach Prozesswechsel	gültig	gültig, falls es ausreichend viele PIDs gibt	ungültig
Inhalt nach Seitenfehler	modifizierte Kachel ungültig	gültig* (verletzt *inclusion property*)	gültig* (verletzt *inclusion property*)
Geschwindigkeit	langsam	schnell	schnell
Automatische Kohärenz bei Sharing	ja	nein	ja, aber Code muss neu geladen werden
primäre Anwendung	große Chaches	kleine Caches; Befehlscaches, falls dynamisches Überschreiben von Befehlen verboten ist.	

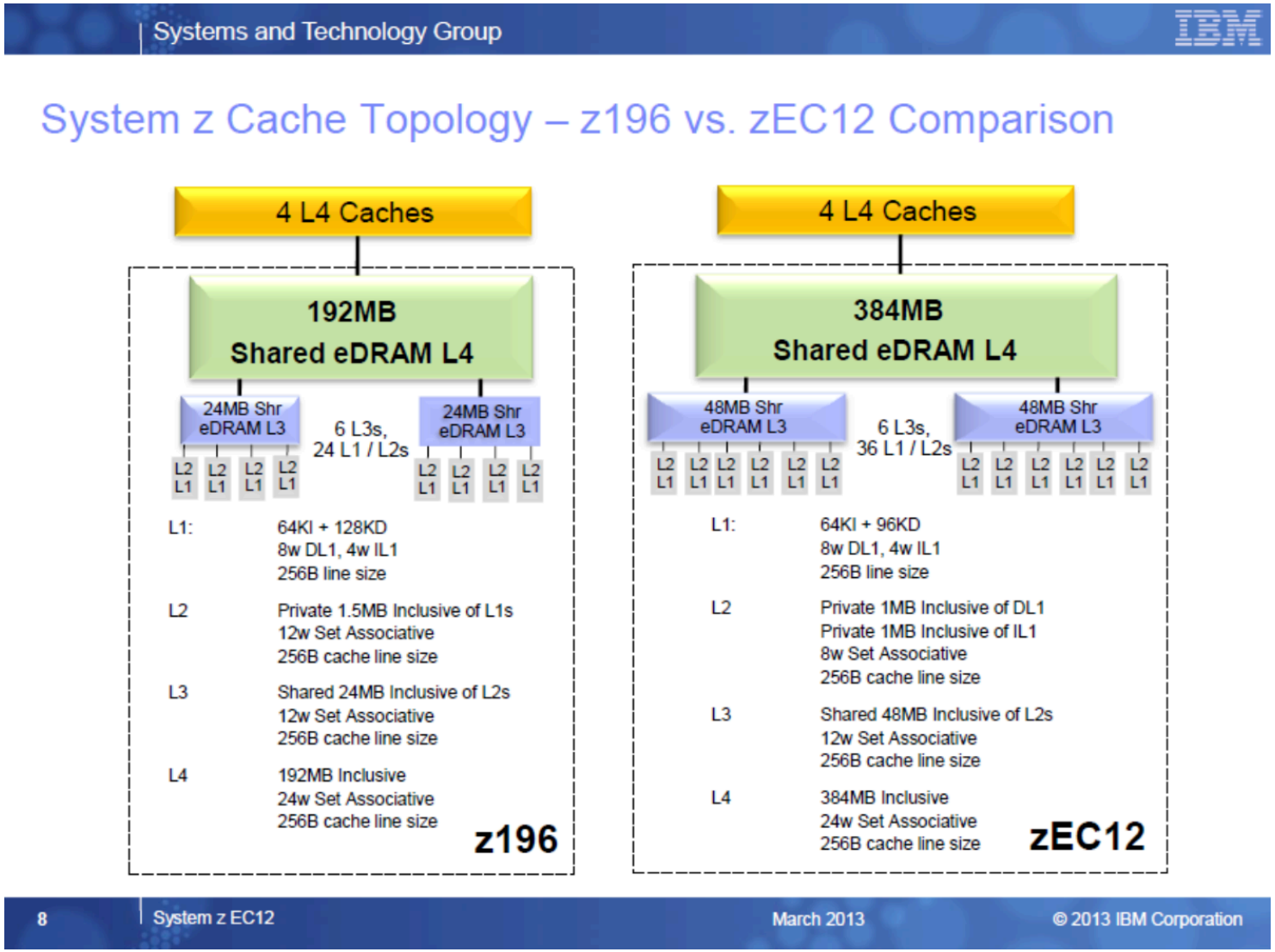
* Potenzielle Schwierigkeiten, wenn Cacheinhalte $\not\subset$ Hauptspeicherinhalte

Adressabbildung beim Intel 386

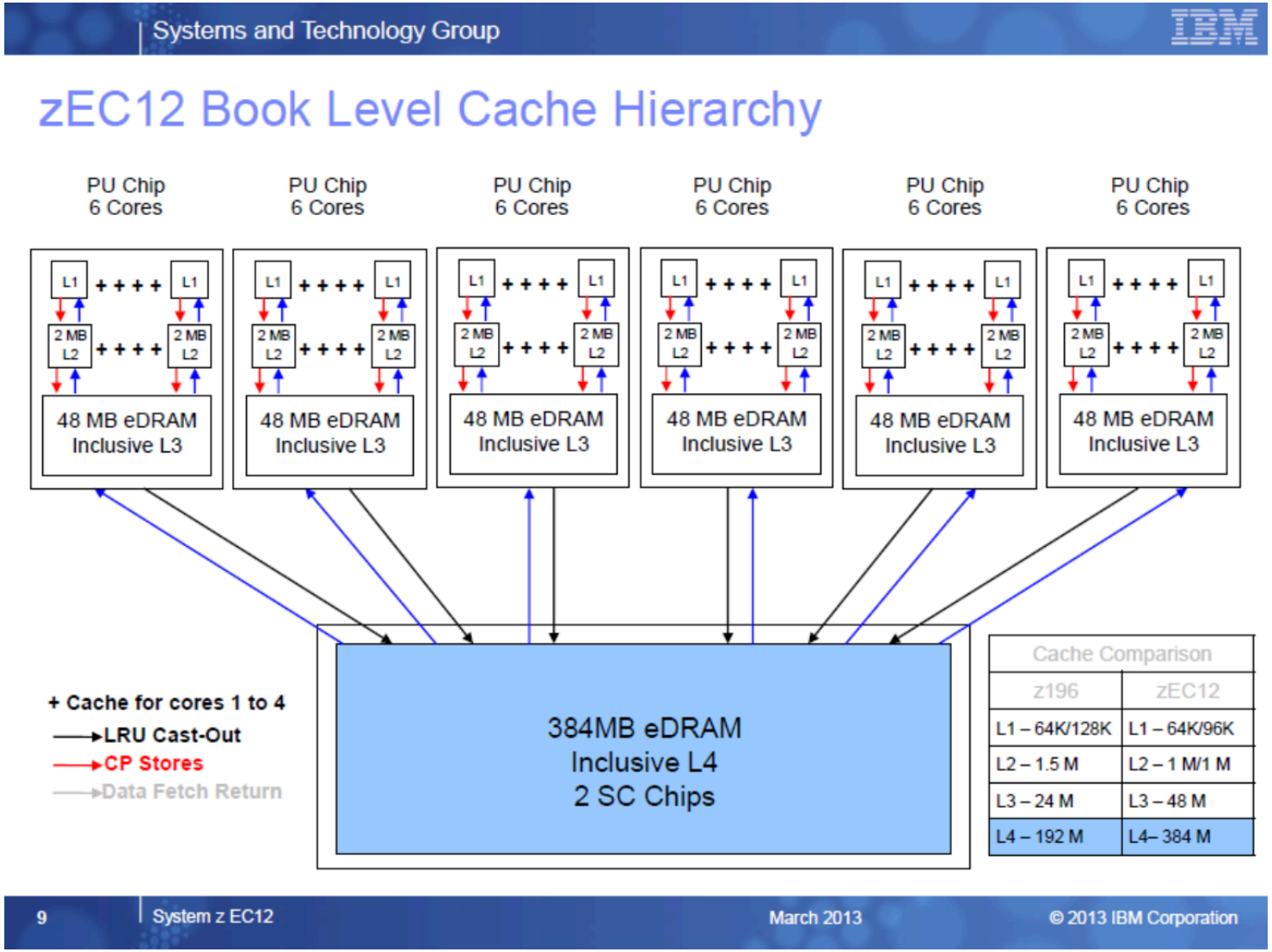


- Die *Local Descriptor Table LDT* benötigt Einträge für den Bereich möglicher Segmentnummern;
- Tabellen für unbenutzte Seiten sind nicht erforderlich. Diese haben selbst wieder die Größe einer Seite.
- Adressübersetzungstabellen können teilweise auf die Platte ausgelagert werden
- In 64-Bit Varianten nur rudimentäre Bedeutung

Beispiel: IBM Großrechner



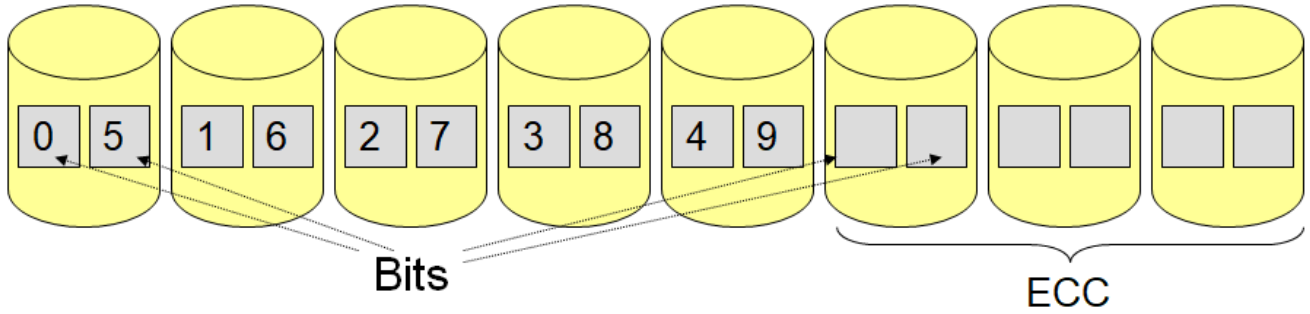
Beispiel: IBM Großrechner



Anhang: RAID

RAID 2 (ECC)

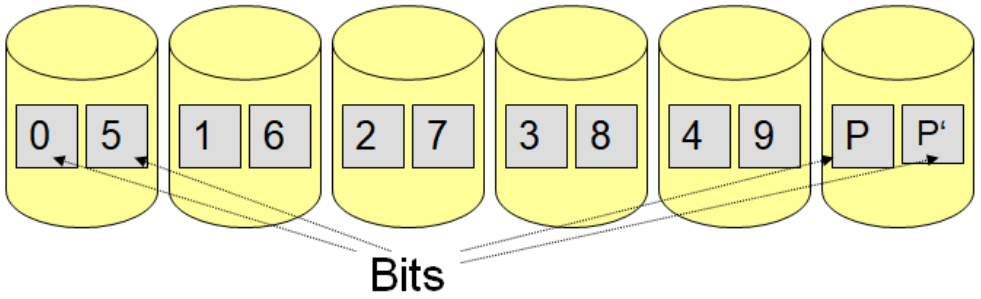
- Zusätzlich werden Prüfbits auf spezielle Platten geschrieben, Verwendung von fehlerkorrigierenden Codes (error correcting code, ECC).
- Die Bits werden auf mehreren Platten verteilt.



- Geringere Redundanz als RAID 1, aber wegen Prüfbit-Erzeugung und Last auf Prüfbitplatten langsamer.
- Erfordert spezielle Platten, kommerziell nicht verfügbar.

RAID 3 (dedicated parity)

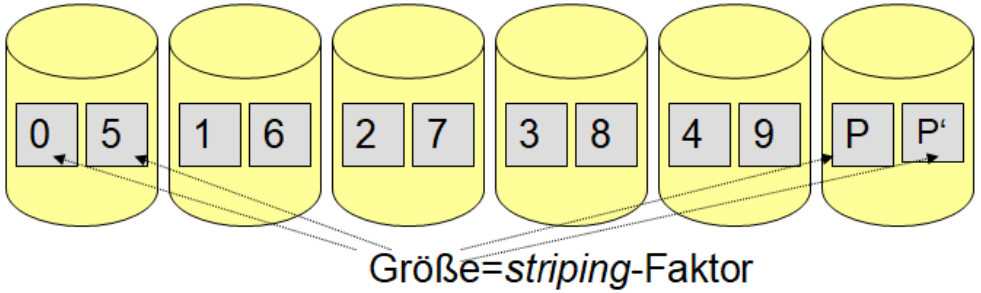
- Es nur ein einzelnes Paritäts-Stripe auf einer Parity-Platte abgelegt.
- Häufig nur der Fall Stripe-Parameter=1 Bit betrachtet. Die Stripes eines Datenblocks werden auf Platten verteilt.



- Selbst für kleinste zu schreibende/lesende Blöcke alle Platten aktiv. Keine gute Performance.

RAID 4

- Wie RAID-3, jedoch mit einem Striping-Faktor von einem Block und mehr.

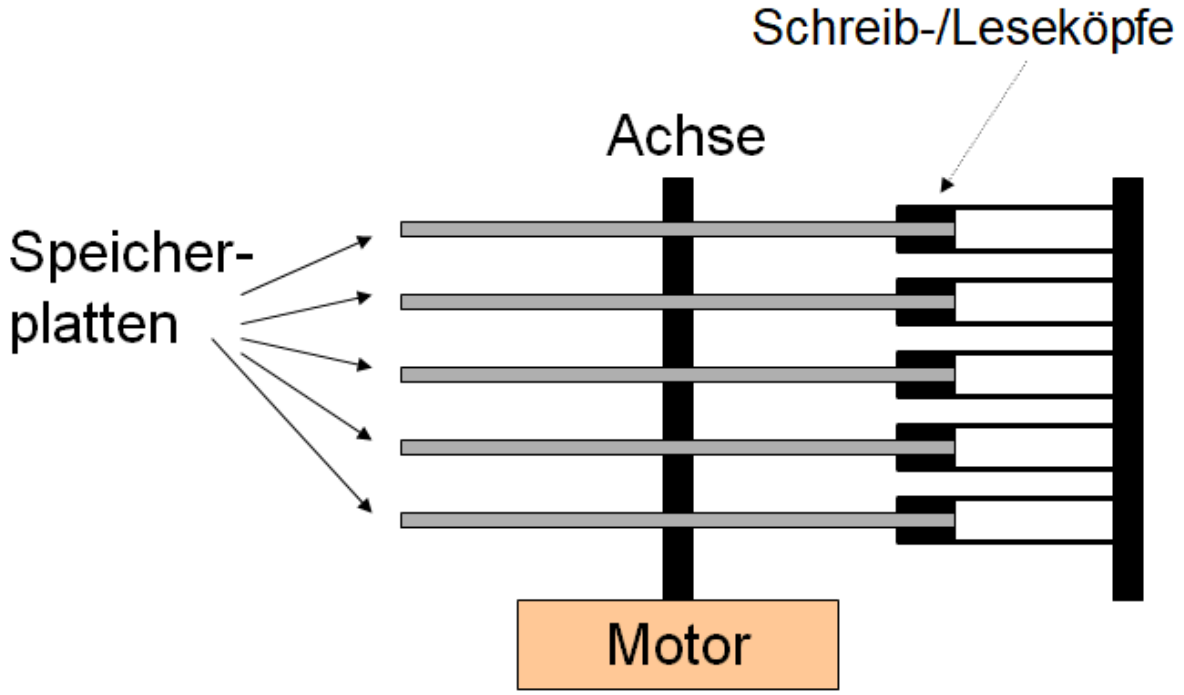


- Besserer wahlfreier Zugriff als RAID 3. Paritätsplatte bleibt ein Engpass.

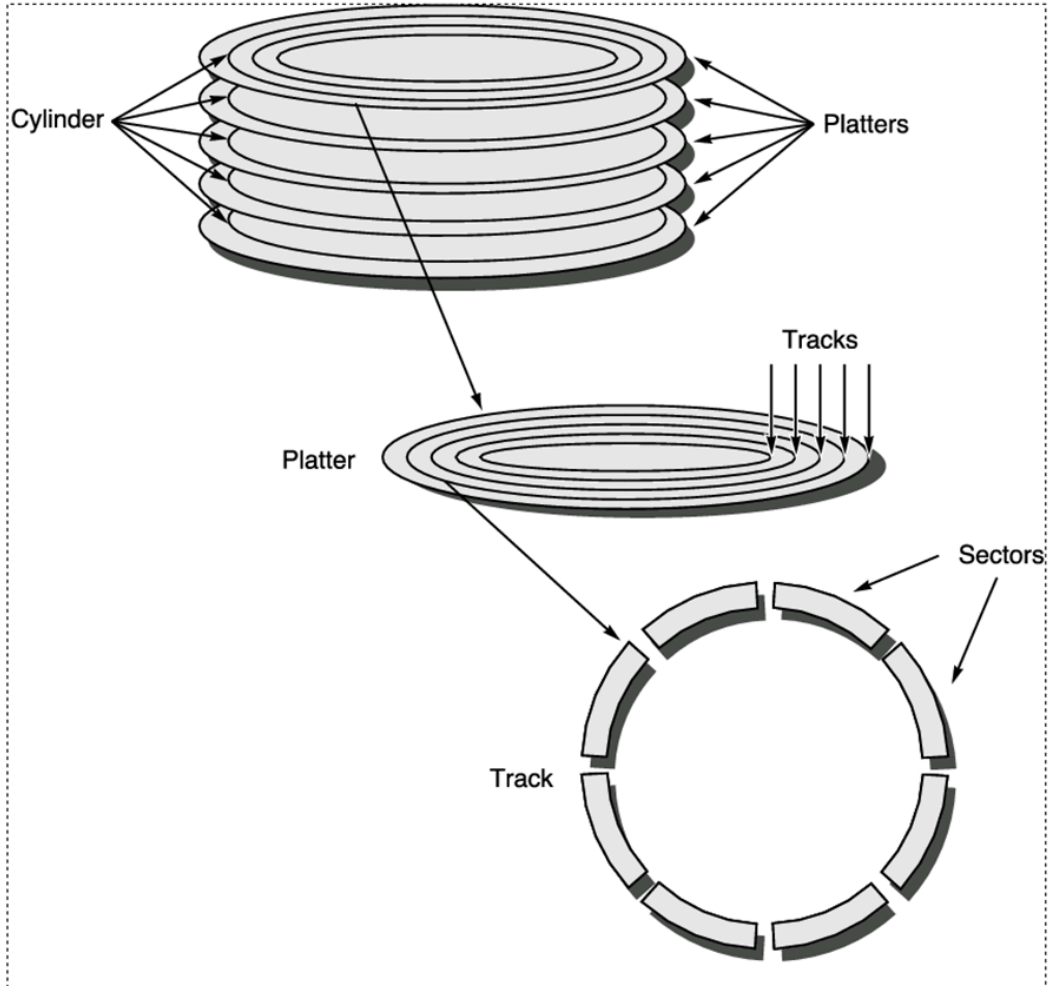
Anhang: Platten, Flash, CD, etc.

Schematischer Aufbau eines Plattenlaufwerks

- Motor dreht Plattenlaufwerk um Achse
- Schreib/Leseköpfe bewegen sich vorwärts richtung Achse und wieder zurück



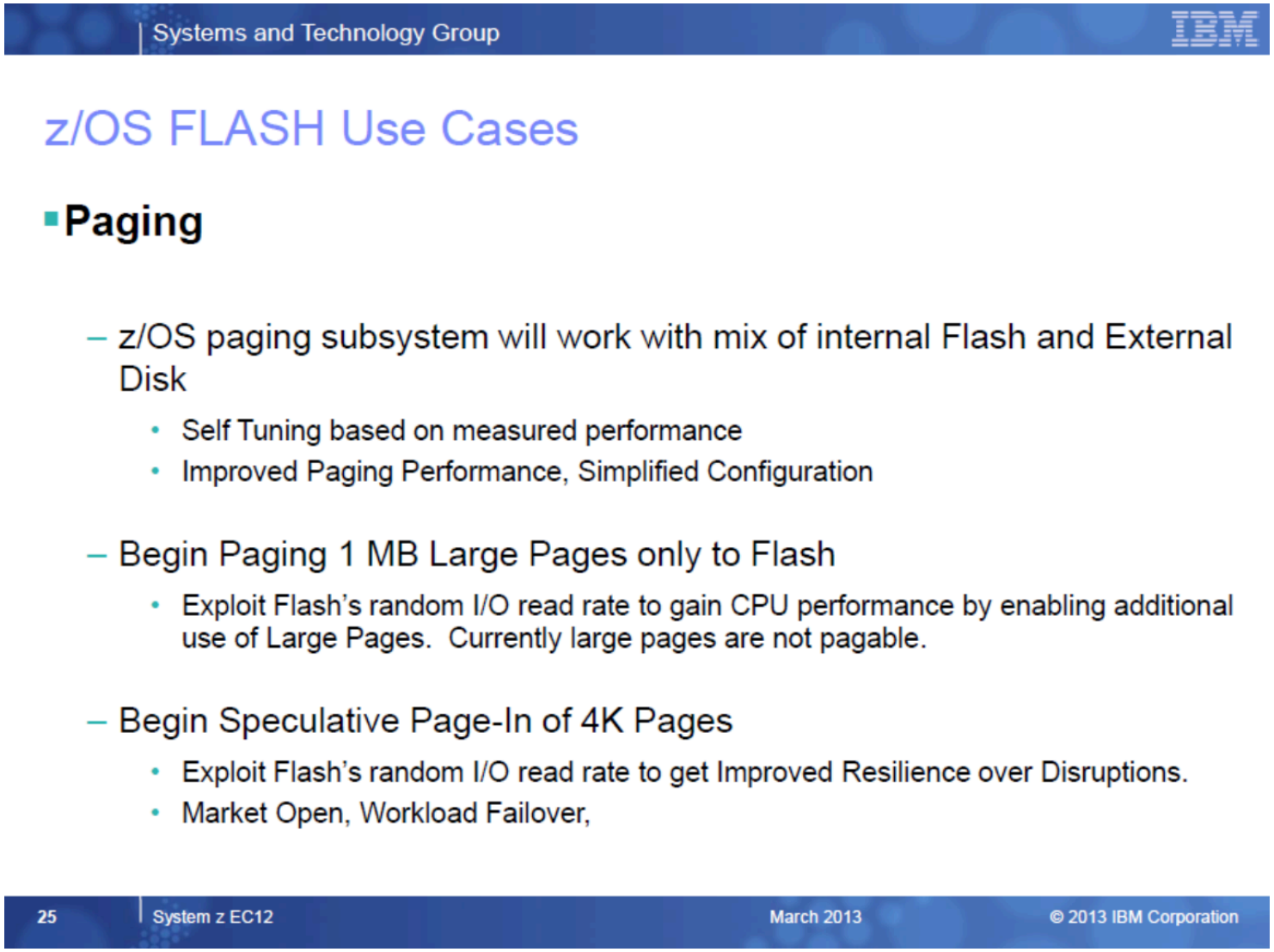
Einteilung der Platten in Sektoren, Spuren und Zylinder



Daten einiger Plattenlaufwerke (2011)

	Seagate Barracuda XT	Seagate Momentus Thin	Toshiba MK1214GAH
Durchmesser[”]	3,5	2,5	1,8
Kapazität[GB]	2000	7200	5400
Umdrehungen[RPM]	7200	7200	5400
Mittlere Latenz [ms]	4.17	4.17	5,56
Average seek [ms]	8,5 read/9.5 write	11	15
Übertragungsrate [MB/s]	Max. 600, 138 dauerhaft	300	300
Leistung [W] [passiv/aktiv]	6,39/9,23	0,66/1,60	0,40/1,30
Puffer [MB]	64	16	16
Gb/sq.in	347	425	?
Schock-Toleranz [in Betrieb/außer Betrieb]	86G/300G	350G/1000G	500G/1500G
Interface	SATA, 6 Gb/s	SATA, 3 Gb/s	SATA, 3 Gb/s

Flashspeicher als „paging device“



Flash-spezifische Dateisysteme

- Zwei Ebenen können ineffizient sein:
  - FTL bildet Magnetplatte nach
  - Standard-Dateisystem basiert auf Magnetplatten
    Beispiel: Gelöschte Sektoren nicht markiert nicht wieder verwendet
- Log-strukturierte Dateisysteme fügen nur neue Informationen zu
  - Für Magnetplatten
    - Schnelle Schreibvorgänge
    - Langsames Lesen (Kopfbewegungen für verteilte Daten)
  - Ideal für Flash-basiertes Dateisystem:
    - Schreibvorgänge in leere Sektoren
    - Lesen nicht langsam, da keine Köpfe bewegt werden

→ Spezifische log-basierte Flash-Dateisysteme (JFFS2 (NOR), YAFFS (NAND))

Vergleich Harddisc/Flash-Speicher (2011)

	Flash	**HDD**
Zugriffszeit (random) [ms]	~0,1	5-10
Kosten [$/GB]	1,2-2 (Fixanteil gering)	0,05-0,1 Fixanteil!
Kapazität [GB]	Typ. < 120	1000-3000
Leistungsaufnahme [W]	Ca. 1/3-1/2 der HDD-Werte	Typ 12-18, Laptops: ~2
Defragmentierung	unwichtig	Zu beachten
Zeitverhalten	Relativ deterministisch	Von Kopfbewegung abhängig
Anzahl der Schreibvorgänge	begrenzt	unbegrenzt
Verschlüsselung	Überschreiben unverschlüsselter Info schwierig	Überschreiben einfach
Mechanische Empfindlichkeit	Gering	Stoßempfindlich
Widerbenutzung von Blöcken	Erfordert Extra-Löschen	Überschreiben

Vergleich Flash/Microdrive (2002)

	Sandisk Type I Flash	Sandisk Type II Flash	IBM Microdrive DSCM-10340
Kapazität [MB]	64	300	340
El. Leistung [W] (standby/operating)	0,15/0,66	0,15/0,66	0,07/0,83
Schreibzyklen	300.000	300.000	unbegrenzt
Mittlerer Abstand zwischen Fehlern [h]	>1.000.000	>1.000.000	service-life=min(5J, 8800 h operating)
Fehlerraten, unkorrigierbar	< 1 per 10^{14}	< 1 per 10^{14}	< 1 per 10^{13}
Maximale Zahl Einschaltvorgänge	unbegrenzt	unbegrenzt	300.000
Schock-Toleranz	2000G; 2000G	2000G	175G; 1500G
Verschlüsselung	Überschreiben unverschlüsselter Info schwierig	Überschreiben einfach	
Mechanische Empfindlichkeit	Gering	Stoßempfindlich	
Widerbenutzung von Blöcken	Erfordert Extra-Löschen	Überschreiben	

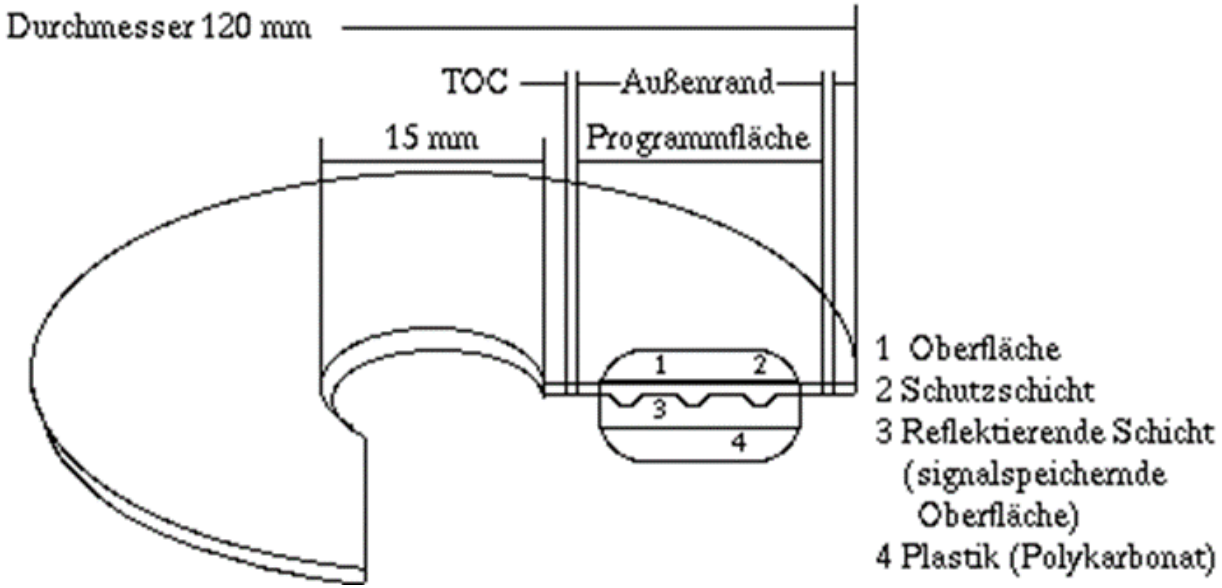
Source: Hennessy/Patterson, Computer Architecture, 2002

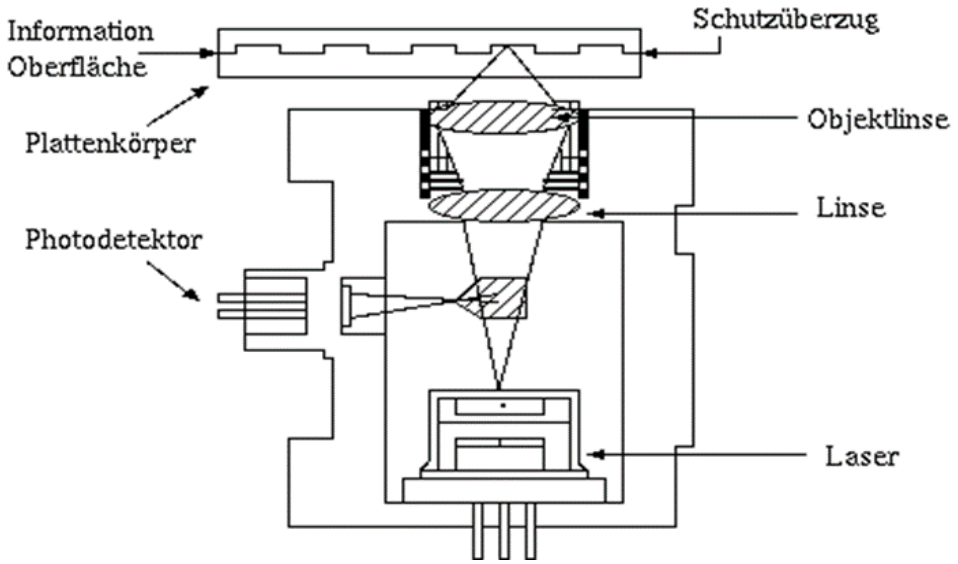
Mögliche Stufen der Speicherhierarchie und derzeit eingesetzte Technologien

Tertiärspeicher (Optisch, Bänder, Cloud) (persistent)
Sekundärspeicher (Platten, Flash) (persistent)
Plattencaches (DRAM, Flash)
Haupt- bzw. Primärspeicher (DRAM)
Caches (SRAM) & TLBs
Register (SRAM)

CD-ROM

- Datenträger auf der Basis der Technik von Audio-CDs
- Informationen als Vertiefung in einer reflektierenden Oberfläche gespeichert
- Abtastung mit 780 nm Laser

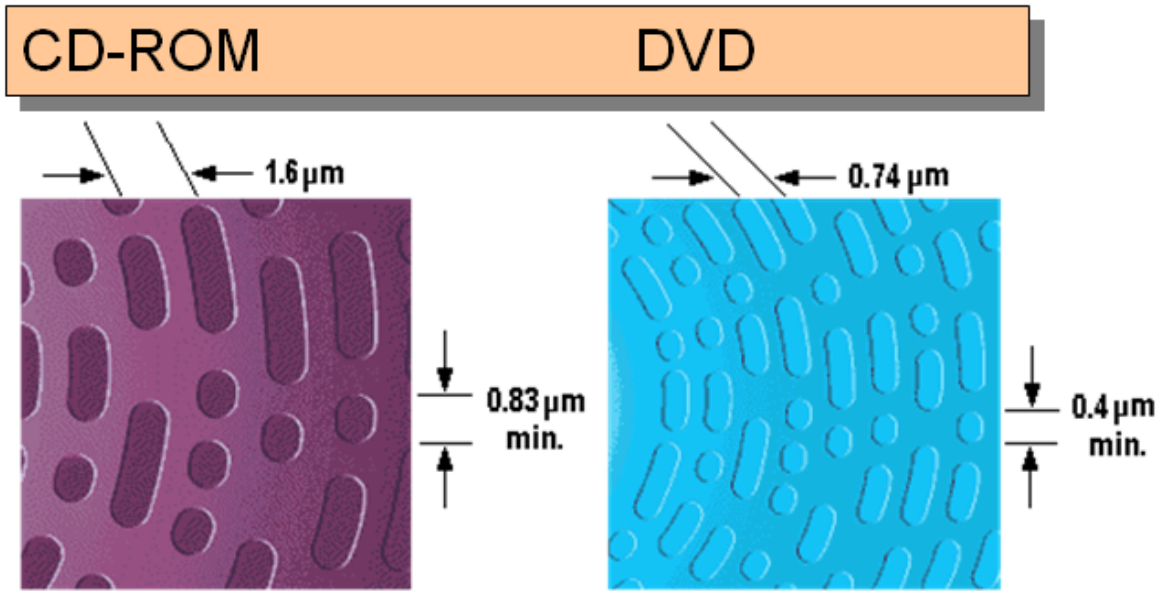




DVD-Laufwerke (1)

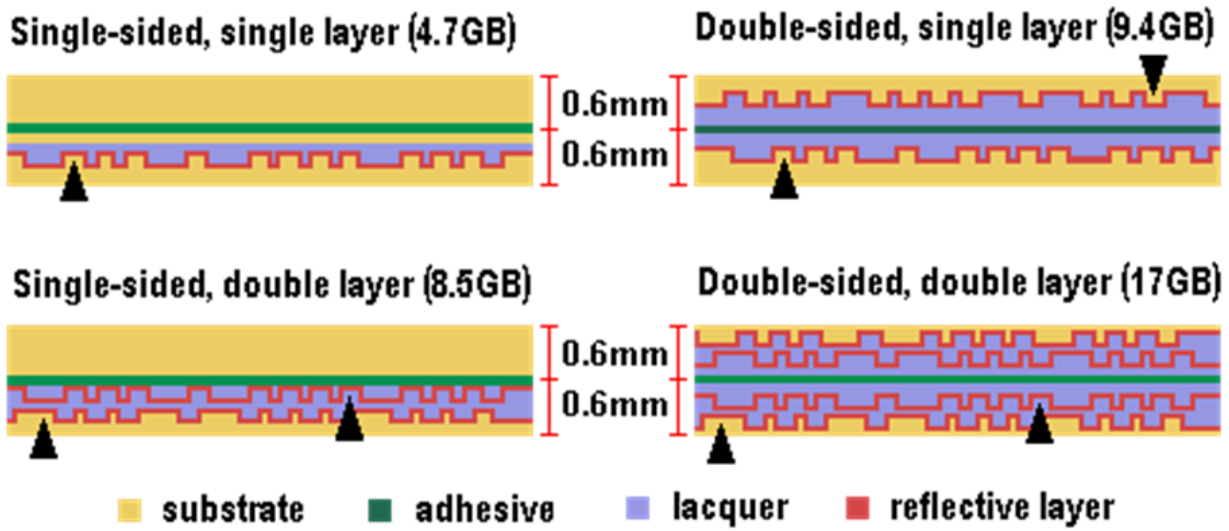
Kapazität gegenüber CDs erhöht:

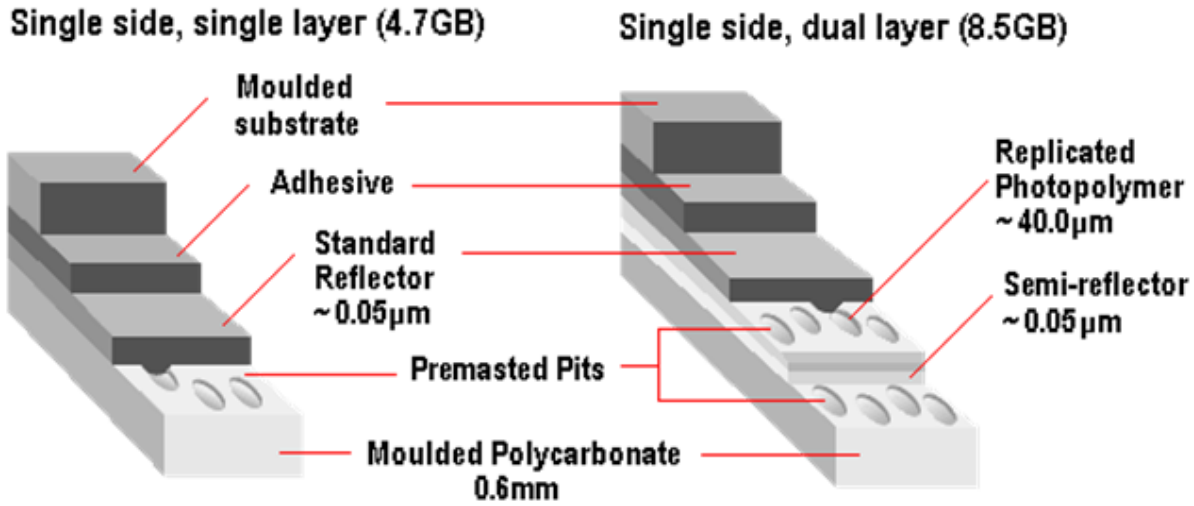
- 1. Abstände und Größe der Vertiefungen werden reduziert, Abtastung mit 650 nm (rotem) Laserstrahl → Erhöhung der Kapazität um Faktor 4



DVD-Laufwerke (2)

2. Informationen können in zwei Ebenen und auf beiden Seiten gespeichert werden.





- Datenrate 1,25 MB/s bei einfacher Geschwindigkeit, MPEG-2 bei Speicherung von Filmen.

Blu*-ray disc (BD) : Motivation”

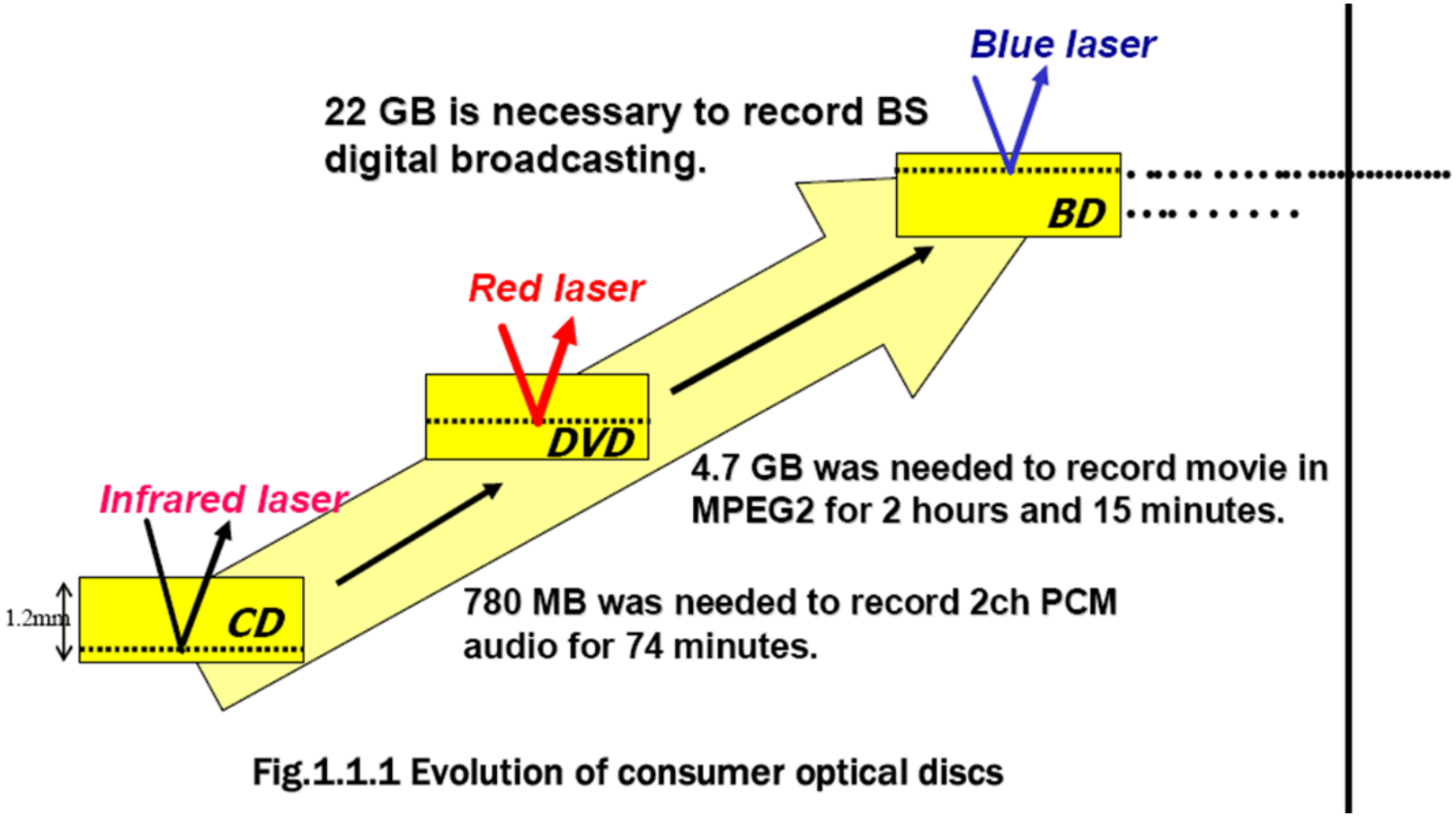


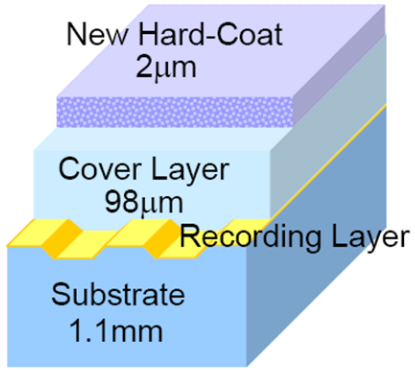
Fig.1.1.1 Evolution of consumer optical discs

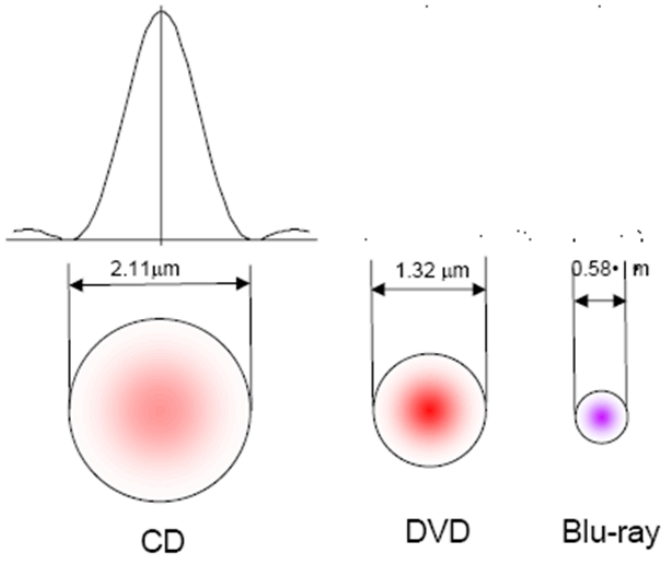
*Falsche Schreibweise, damit Eintrag als Markenzeichen möglich

http://www.blu-raydisc.com/assets/downloadablefile/ general_bluraydiscformat-12834.pdf

Blu-ray: Eigenschaften

- Verkürzung der Wellenlänge auf 405 nm (blau)
- Reduktion der Dicke der transparenten Schicht auf 100 µm, zur besseren Fokussierung des Laserstrahls, bessere Fehlerkorrektur; zusätzliche schmutzresistente Schicht

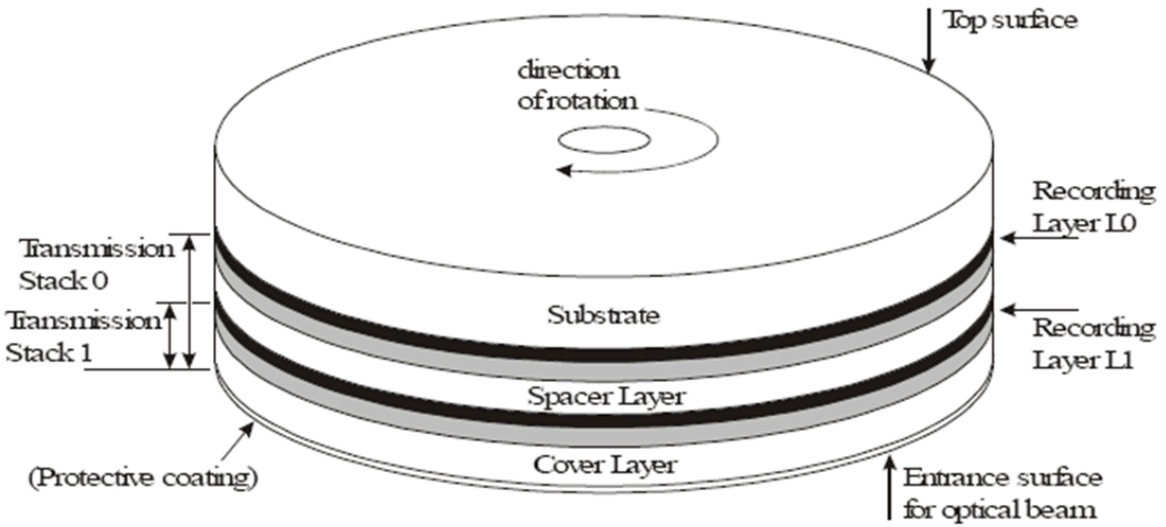




http://www.blu-raydisc.com/assets/downloadablefile/ general_bluraydiscformat-12834.pdf

Blu-ray: Eigenschaften (2)

- 23.3/25/27 GB in einer Speicherebene
- 46.6/50/54 GB in zwei Speicherebenen
- Experimentell 200 GB



Vergleich Übertragungsgeschwindigkeiten

Annahme: Transport einer Blu-ray, als *Dual Layer* beschrieben (Kapazität=50 GB) auf der Strecke Dortmund - München (Strecke ~ 500 km) mit einer Fahrrad-Staffel mit 36km/Std im Mittel. Gleichzeitig DSL-Übertragung mit Übertragungsrate 8 M Bit/s (download-Rate (!)).
Wie viel % der maximalen Rate muss die DSL-Strecke erreichen, damit nicht die Radfahrer gewinnen ?
(Vernachlässigung von Extra-Bits)

$x \times$ Übertragungsrate $\times$ Zeit des Radfahrers $=$ Kapazität

$x =$ Kapazität $\times$ Geschwindigkeit $/$ (Übertragungsrate $\times$ Strecke)

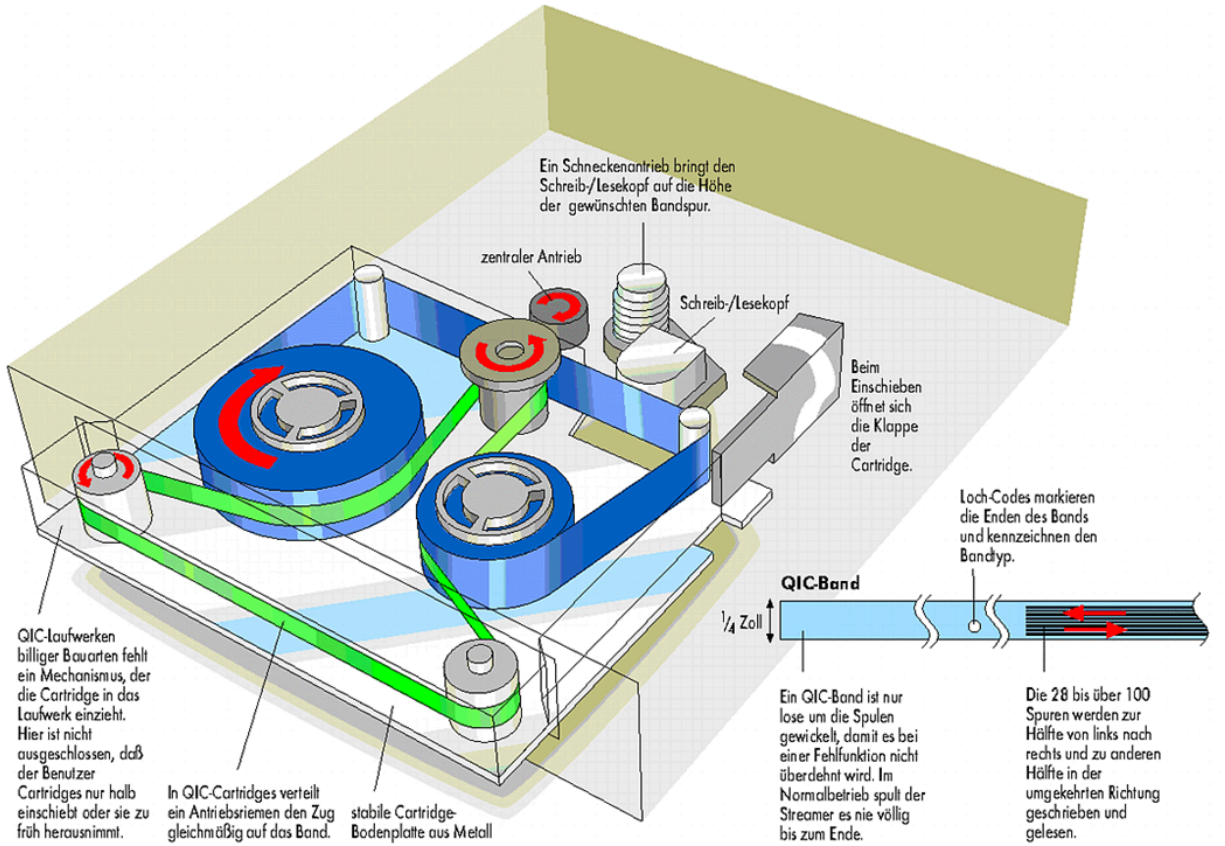
$$x = 50 \times 10^9 \times 8 \times 36 / (8 \times 10^6 \times 60 \times 60 \times 500)$$

$$x = 50 \times 10^9 \times 8 \times 36 / (50 \times 10^9 \times 8 \times 36)$$

$$x = 1$$

Die DSL-Strecke müsste 100 % der max. Übertragungsrate erreichen.

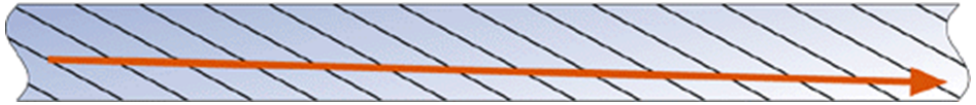
QIC-Laufwerk (Quarter inch tape)

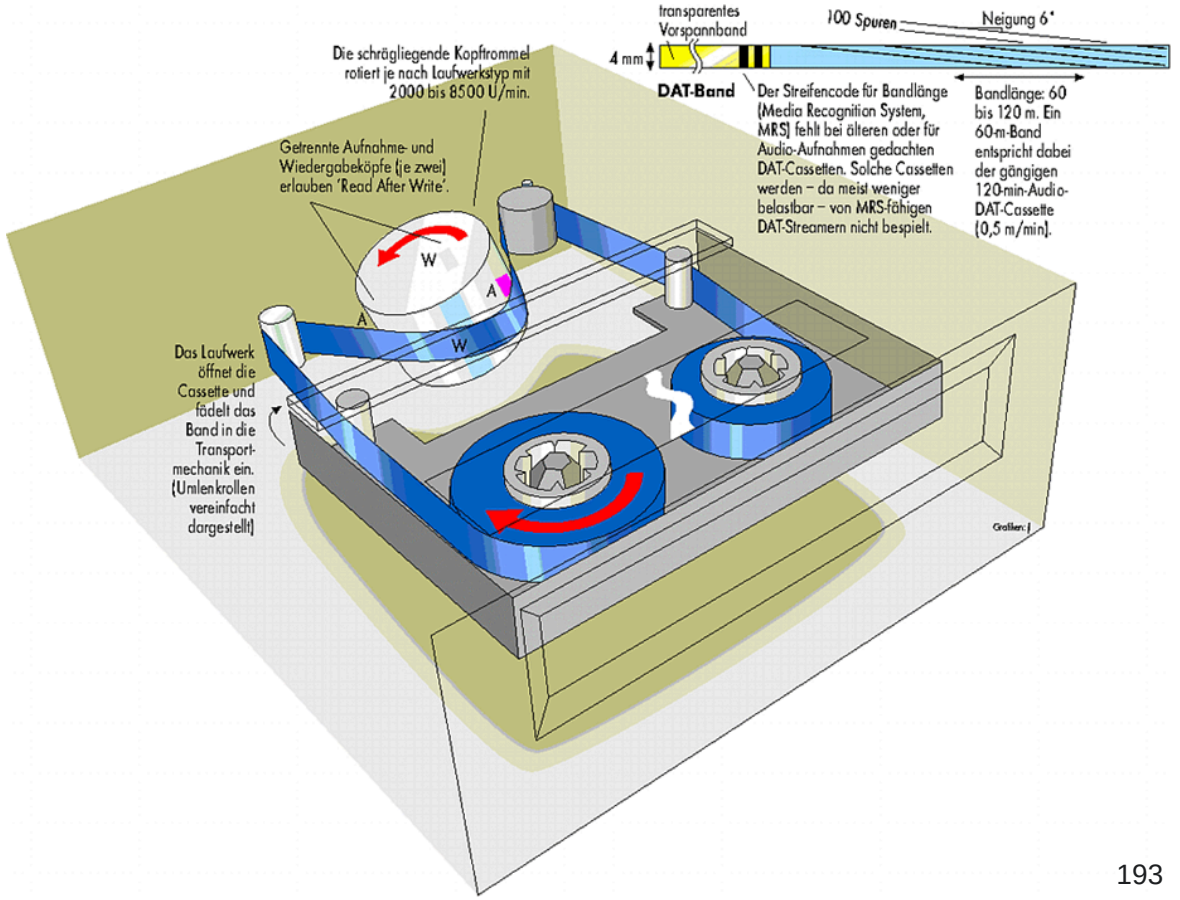


- Horizontale Aufzeichnung in mehreren (bis zu 100) Spuren nacheinander
- optimiert auf streaming-Betrieb mit hoher Bandgeschwindigkeit
- bis ~5 GB (20 GB bei Weiterentwicklung Travan-5)

DAT-Laufwerk (digital audio tape)

- Technik der digital audio tapes
- Schrägspur-aufzeichnung
- Weiterentwicklung DDS-4
- Bis 20 GB (40GB komprimiert);





Linear Tape Open (LTO) - 4



[www.wikipedia.de]

Linear Tape Open (LTO) - 4

Übersicht der Generationen								
Laufwerks-Generation	Bandkapazität bei gleicher Generation		maximale Transferrate		kompatible Bänder		Verfügbarkeit	
	ohne Kompression	mit Kompression	ohne Kompression	mit Kompression	Schreiben	Lesen	Lizenzen seit	Produkte seit
Ultrium 1	100 GB [5]	200 GB [5]	20 MB/s	40 MB/s [5]	Ultrium 1	Ultrium 1	1998 [5]	2000 [5]
Ultrium 2	200 GB [5]	400 GB [5]	40 MB/s	80 MB/s [5]	Ultrium 1 Ultrium 2	Ultrium 1 Ultrium 2	April 2002 [5]	Ende 2002 [5]
Ultrium 3	400 GB [5]	800 GB [5]	80 MB/s	160 MB/s [5]	Ultrium 2 Ultrium 3	Ultrium 1 Ultrium 2 Ultrium 3	Juli 2004 [5]	Ende 2004 [5]
Ultrium 4	800 GB [5]	1.600 GB [5]	120 MB/s	240 MB/s [5]	Ultrium 3 Ultrium 4	Ultrium 2 Ultrium 3 Ultrium 4	Ende 2006 [5]	Frühjahr 2007 [6]
Ultrium 5	1.500 GB	3.000 GB [5]	140 MB/s	280 MB/s [5]	Ultrium 4 Ultrium 5	Ultrium 3 Ultrium 4 Ultrium 5	Sommer 2009 [7]	Frühjahr 2010 [8]
Ultrium 6	3.200 GB	8.000 GB [5]	210 MB/s	525 MB/s [5]	-	-	-	-
Ultrium 7	6.400 GB	16.000 GB [5]	315 MB/s	788 MB/s [5]	-	-	-	-
Ultrium 8	12.800 GB	32.000 GB [5]	472 MB/s	1180 MB/s [5]	-	-	-	-

[www.wikipedia.de]

Bewertung von Band-basierten Medien

Lange Suchzeiten, geringe Transferraten.

Abnutzung

- Schrägspur: „Hunderte von Durchläufen“
- Parallelspur: „Tausende bis Millionen von Durchläufen“

Ursprünglich deutlich kostengünstiger als Platten, da nur das Speichermedium repliziert wird.

Hennessy/Patterson: Gegenwärtiger Kostenvorteil?

IRB (2011): ~22 € für Archivierung von 800 GB über LTO-4; bietet immer noch Kosten- und Archivierungsvorteile